1. Python 계단밟기
1. https://wikidocs.net/20480

제 01장 첫번째 계단밟기

Python은 크로스 플랫폼 프로그래밍 언어입니다. 즉, Windows, Mac OS X, Linux, Unix와 같은 여러 플랫폼에서 실행되며 Java 및 .NET 가상 시스템으로 포팅되었습니다. 그것은 자유롭고 오픈 소스입니다.
오늘날의 Linux와 Mac의 대부분은 Python이 사전 설치되어 있지만 버전이 오래되었을 수 있습니다. 따라서 항상 최신 버전을 설치하는 것이 좋습니다.
여기에서는 Python을 설치하고 기본 사용법을 익혀보도록 하겠습니다.
파이썬을 시작하는 다양한 방법이 있습니다.

1. 즉시 모드

python명령 행에 입력 하면 즉시 모드에서 인터프리터가 호출됩니다. Python 표현식을 직접 입력하고 Enter 키를 눌러 출력을 얻을 수 있습니다.

```
>>> 1+
2
>>>
```

파이썬 프롬프트입니다. 통역사가 우리의 의견을 기다릴 준비가되었음을 알려줍니다. 입력을 시도 1 + 1하고 Enter 키를 누릅니다. 우리 2는 결과물을 얻습니다 . 이 프롬프트는 계산기로 사용할 수 있습니다. 이 모드를 종료하려면 exit()또는 quit()을 입력하고 Enter 키를 누릅니다.

2. 스크립트 모드

이 모드는 파일로 작성된 Python 프로그램을 실행하는 데 사용됩니다. 이러한 파일을 스크립트 라고합니다 . 나중에 사용할 수 있도록 스크립트를 디스크에 저장할 수 있습니다. 파이썬 스크립트는 .py 확장자를 갖습니다.
예 : helloWorld.py
이 파일을 스크립트 모드로 실행하려면 python helloWorld.py명령 프롬프트에 간단하게 작성 하십시오.

3. 통합 개발 환경 (IDE)

텍스트 편집 소프트웨어를 사용하여 Python 스크립트 파일을 작성할 수 있습니다.
.py확장 기능 으로 저장하면됩니다 . 하지만 IDE를 사용하면 훨씬 쉽게 삶을 살 수 있습니다.
IDE는 응용 프로그램 개발을 위해 프로그래머에게 코드 힌트, 구문 강조 표시 및 확인, 파일 탐색기 등과 같은 유용한 기능을 제공하는 소프트웨어입니다.
IDE를 사용하면 중복 작업을 제거하고 응용 프로그램 개발에 필요한 시간을 크게 줄일 수 있습니다.
IDEL은 Python 프로그래밍 언어와 함께 설치할 수있는 그래픽 사용자 인터페이스 (GUI)이며 공식 웹 사이트에서 구할 수 있습니다.
우리는 또한 우리의 취향에 따라 다른 상용 또는 무료 IDE를 사용할 수 있습니다. 우리는

PyCharm IDE를 사용할것 입니다.
마지막 편집일시 : 2018년 7월 6일 10:16
---

1. 01. Python 설치하기

가. Python 설치하기

웹 브라우저로 "http://www.python.org"에 접속합니다.

Downloads 메뉴를 선택합니다. 화면을 아래로 이동하여 Python 3.x.x 를 선택합니다.

나타난 화면에서 다시 화면을 아래로 이동시킵니다. 자신의 컴퓨터 플랫폼에 맞는 파일의 링크를 클릭하여 다운로드를 받습니다.

window 64 비트일 경우는 "Windows x86-64 executable installer"를 다운받습니다.

window32 비트일 경우는 "Windows x86 executable installer"를 다운받습니다.

MacOSX 일 경우는 “Mac OS X 64-bit/32-bit installer”를 다운받습니다.

Linux 일 경우는 기본적으로 파이썬(Python)이 설치되어 있을 것입니다. 어느 버전의 파이썬이 설치되어 있는지 확인하려면, 명령 창을 열고 아래의 스크립트를 실행하세요.

$ python —version

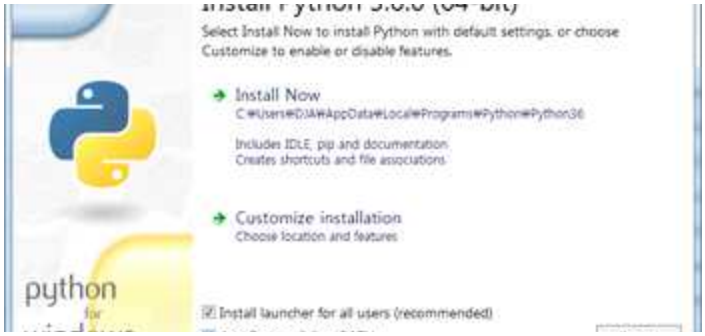
새로운 버전의 파이썬을 설치하려면 매뉴얼을 참조 하시기 바랍니다.

이제 파이썬을 설치하도록 하겠습니다. 여기에서는 windows 에 설치하는 것을 예로 들겠습니다.

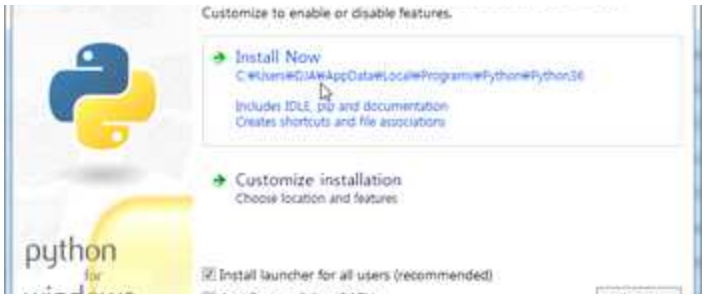
다운로드 받은 파일을 더블클릭하여 실행을 합니다.

나타난 설치 화면에서 하단의 체크 박스를 선택합니다.

그러면 환경변수 path 에 자동으로 파이선의 경로가 추가 됩니다.

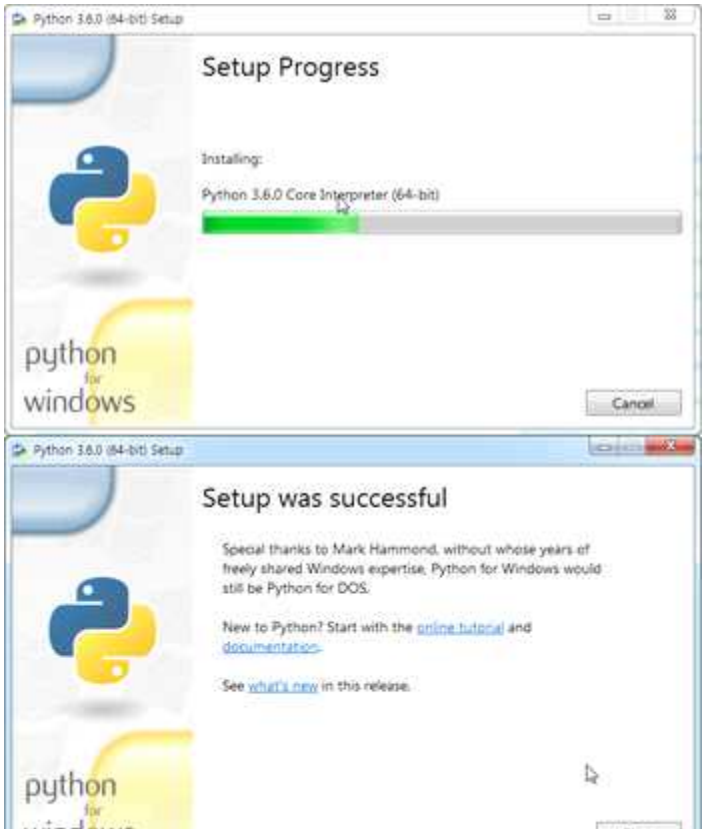


그리고 "Install Now"를 클릭합니다.

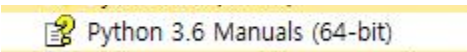


설치가 진행되고 설치가 완료되면 설치 성공화면이 나타납니다.

[Close]버튼을 클릭하여 설치를 완료합니다.

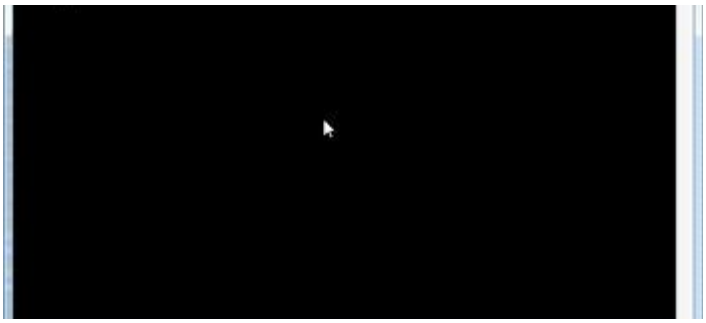


시작 메뉴를 클릭하여 찾아보면 다음과 같이 설치가 완료되어 등록된 것을 확인할 수 있습니다.

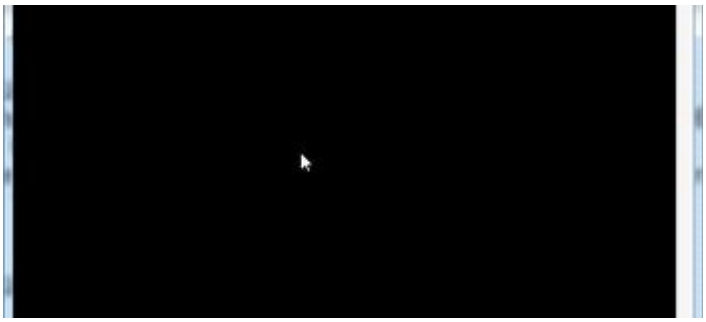


"Python3.6(64-bit)"를 선택하면 다음과 같이 콘솔 창의로 실행화면이 나타납니다.

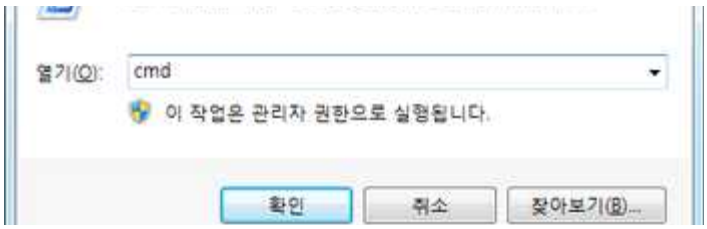
프롬프트에서 [Ctrl + Z]를 누르고 [Enter]키를 누르면 창이 닫히며 종료됩니다.



또는 프롬프트에서 “exit()”를 타이핑하고 [Enter]키를 누르면 창이 닫히며 종료됩니다.

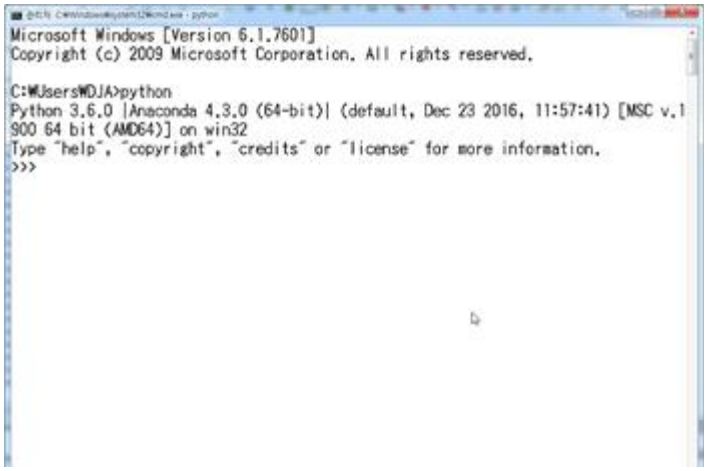


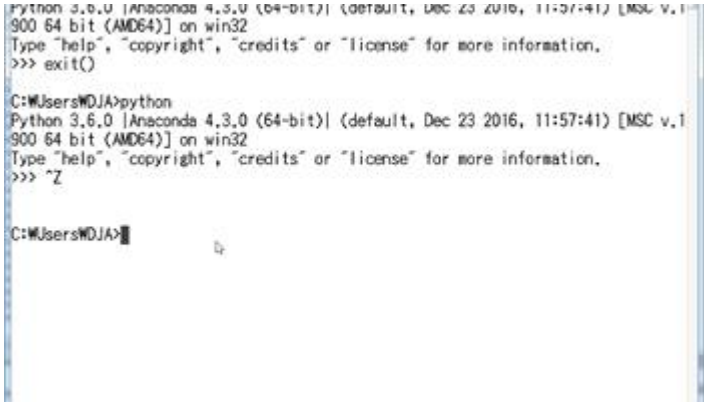
[window + R]키를 눌러 “cmd”를 타이핑하고 콘솔 창을 열어서 실행해 보겠습니다.



프롬프트에서 “python”이라고 입력하고 엔터키를 누르면 실행이 됩니다.

파이썬의 종료 방법은 동일합니다. 콘솔 창을 닫으려면 “exit”를 입력하고 엔터키를 누르면 됩니다.





버전을 확인하려면 다음과 같이 "python –V" 또는 "python —version"으로 확인이 가능합니다.

파이썬에서 제공하는 "IDLE"를 이용해도 됩니다.

파이썬의 종료 방법은 "exit()"를 입력하고 엔터를 칩니다.

나타난 대화상자에서 [확인] 버튼을 클릭하여 종료합니다.

당연히 윈도우 창을 닫아도 종료가 됩니다.

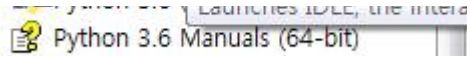
Ctrl+D 를 눌러도 종료합니다.

Ctrl+Q 를 눌러도 종료합니다.

마지막 편집일시 : 2018 년 7 월 6 일 10:07

IDLE 사용해 보기

시작메뉴에서 [IDLE]을 선택합니다.



다음의 화면이 나타납니다.

Ctrl+Space 키를 누르면 자동으로 함수를 선택하는 화면이 나타납니다.

아래와 같이 입력을 해봅니다.

">>>"는 프롬프트입니다. 명령의 입력을 대기하고 있는 상태입니다. 프롬프트 모양 뒤의 내용만 입력합니다. 그럼 바로 결과 확인이 가능합니다.

```
>>> print('Hello World!!')

Hello World!!

>>> print('한글은?');

한글은?

>>> name = input('이름 : ')

이름 : 한사람

>>> name

'한사람'

>>> print(name)

한사람

>>> exit()
```

exit() 함수는 종료를 합니다.

파일로 파이선 파일을 작성하기 위해서는 [File] - [New File]을 선택합니다.

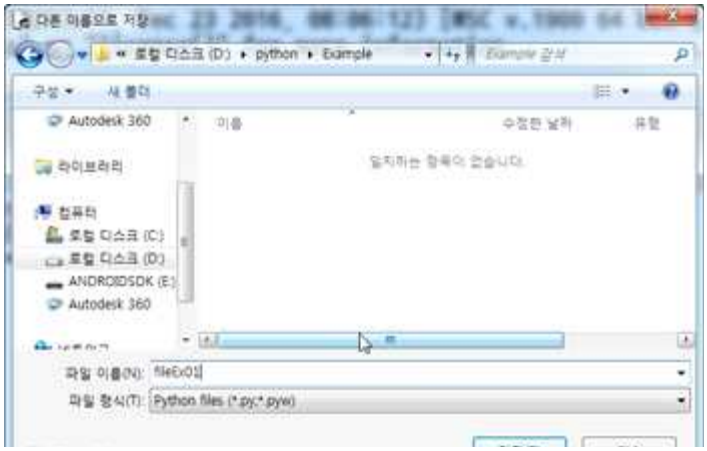
다음의 코드를 입력해봅니다. 파이선에서는 들여쓰기가 아주 중요한 역활을 합니다.
세번째 줄의 내용은 반드시 안으로 들여쓰기가 되어야 합니다.

```
height = int(input('몇줄짜리?'))

for i in range(1, height+1):

    print("☆" * i)

print()
```

[File] - [Save]를 차례대로 선택합니다.

파일 저장 대화상자에서 파일명을 입력하고 저장을 합니다. 파일의 확장자는 py 입니다.

[Run] - [Run Module]을 선택합니다.



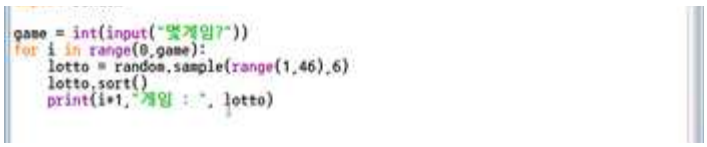
"몇줄짜리?"란 물음에 "5"를 입력해 봅니다. IDLE 윈도우로 실행결과가 나타납니다.

```
몇줄짜리?5

☆

☆☆

☆☆☆

☆☆☆☆

☆☆☆☆☆
```

새로운 파일을 만들면 새로운 창이 나타납니다.

다음의 코드를 입력하고 lotto.py 로 저장을 합니다.

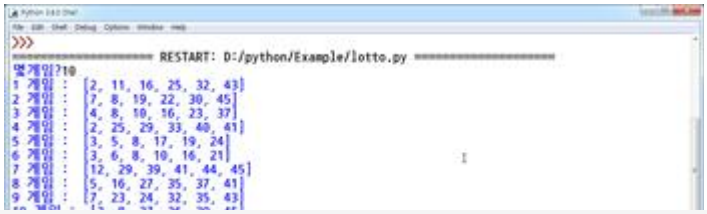
```
import random



game = int(input("몇게임?"))

for i in range(0,game):

    lotto = random.sample(range(1,46),6)

    lotto.sort()

    print(i+1,"게임 : ", lotto)
```



F5 키를 눌러서 실행을 해봅니다.

"몇 게임?"이란 물음에 "10"을 입력해 봅니다.

결과는 다음과 같습니다.




```
몇게임?10

1 게임 :  [3, 13, 24, 31, 35, 39]

2 게임 :  [6, 8, 13, 16, 17, 44]

3 게임 :  [4, 12, 23, 25, 28, 38]

4 게임 :  [5, 6, 20, 24, 36, 41]

5 게임 :  [12, 15, 23, 33, 37, 40]

6 게임 :  [7, 10, 20, 22, 26, 37]

7 게임 :  [3, 6, 13, 23, 31, 35]

8 게임 :  [14, 20, 33, 34, 42, 45]

9 게임 :  [7, 8, 12, 14, 36, 40]

10 게임 :  [8, 19, 22, 28, 33, 40]
```

마지막 편집일시 : 2018 년 7 월 5 일 5:53

---

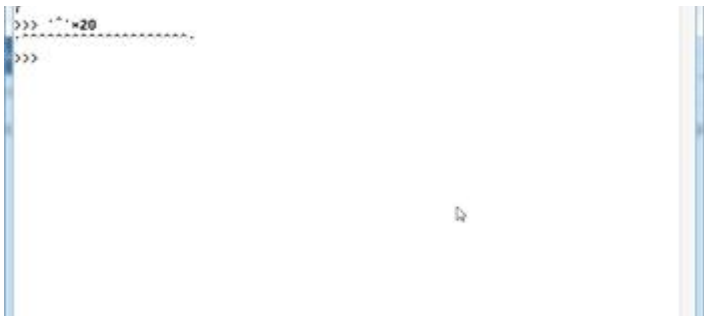
2. 03. REPL 사용해 보기

가. REPL 사용해 보기

- REPL(Read Evaluate Print Loop) : 콘솔 화면에서 파이썬 구문을 입력하면 바로 결과를 반환하고 다시 입력할수 있는 도구를 말합니다.

- 바로 바로 결과 값을 확인 할 수 있습니다.

```
>>> '*' * 20 [Enter]
```



- 아래와 같이 변수 할당, 함수 정의 등은 결과 값이 출력되진 않지만 메모리에 저장되어 변수로 활용가능합니다.

```
>>> a = 10  [Enter]

>>> b = 20  [Enter]

>>> a + b  [Enter]

300
```

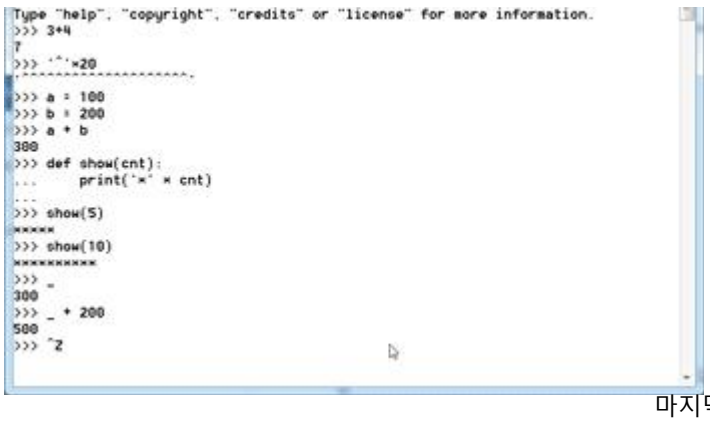
```
>>> def show(cnt):  [Enter]
        print('*' * cnt)  [Enter]

>>> show(5)  [Enter]
*****
>>> show(10)  [Enter]
**********
>>> _  [Enter]
300
>>> _ + 200  [Enter]
500
```

- _ 는 바로 직전 결과 값을 나타냅니다.

- REPL 종료

- Windows 는 CTRL + Z 를 누르고 [enter]를 치면 됩니다.

- Mac&Linux 는 CTRL + D 입니다.



마지막 편집일시 : 2018 년 7 월 5 일 5:52

---

3. 04. 파이참(PyCharm) 설치하기

가. 파이참(PyCharm) 설치하기

1) 1. 파이참(PyCharm) 이란?

파이참이란 파이선 프로그램을 쉽게 개발할 수 있도록 하는 IDE 입니다. IDE란 [Integrated Development Environment]의 약자로 통합 개발 환경을 말합니다. 통합개발 환경은 개발자가 소프트웨어를 개발하는 과정에 필요한 모든 작업을 하나의 소프트웨어에서 처리할 수 있도록 환경을 제공합니다. 코드 편집기, 디버거, 컴파일러, 인터프리터 등을 포함하고 개발자에게 제공합니다. 자주 사용되는 IDE의 종류로는 비주얼스튜디오, 이클립스, 파이참 등이 있습니다.

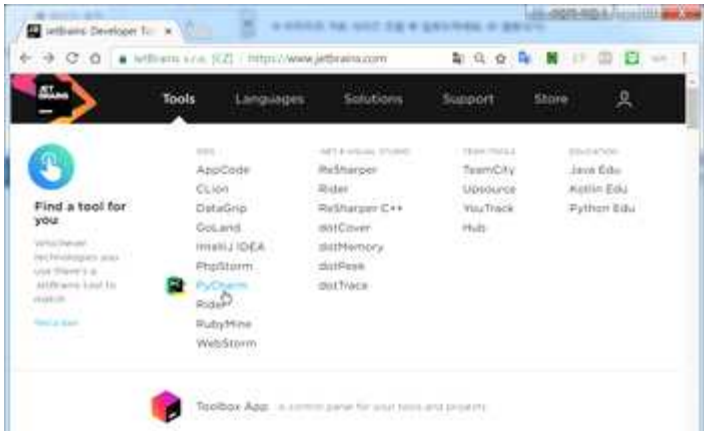
2) 2. 파이참(PyCharm) 설치

1. 공식홈페이지에 접속합니다.

접속하시면 아래와 같은 화면이 나옵니다.

상단의 [Tools]메뉴의 [PyCharm]을 선택합니다.

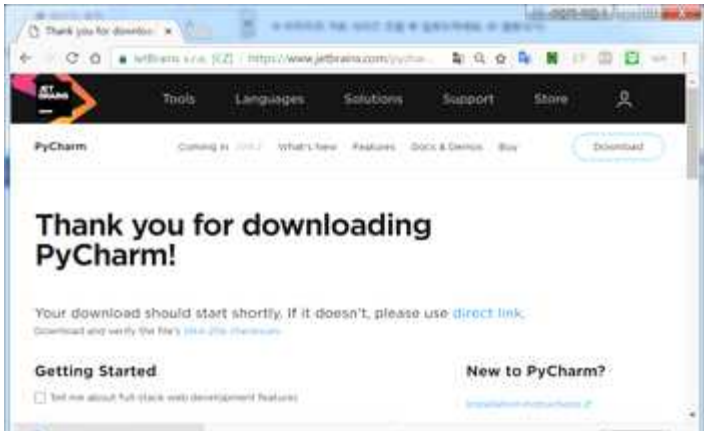
우측 상단의 [Download]버튼을 클릭합니다.



PyCharm은 유료인 Professional과 무료인 Community가 있습니다.

Community로 다운 받겠습니다.

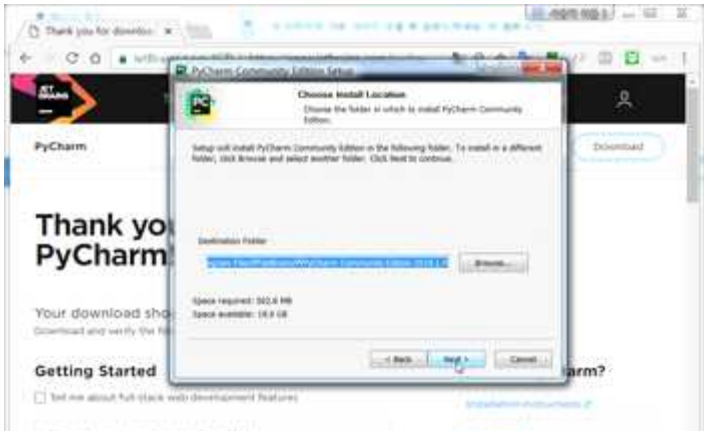
다운로드가 완료되면 더블클릭하며 실행을 합니다.



나타나는 화면에서 실행을 클릭합니다.

[Next]버튼을 클릭하여 설치를 진행합니다.

설치할 폴더를 지정합니다.



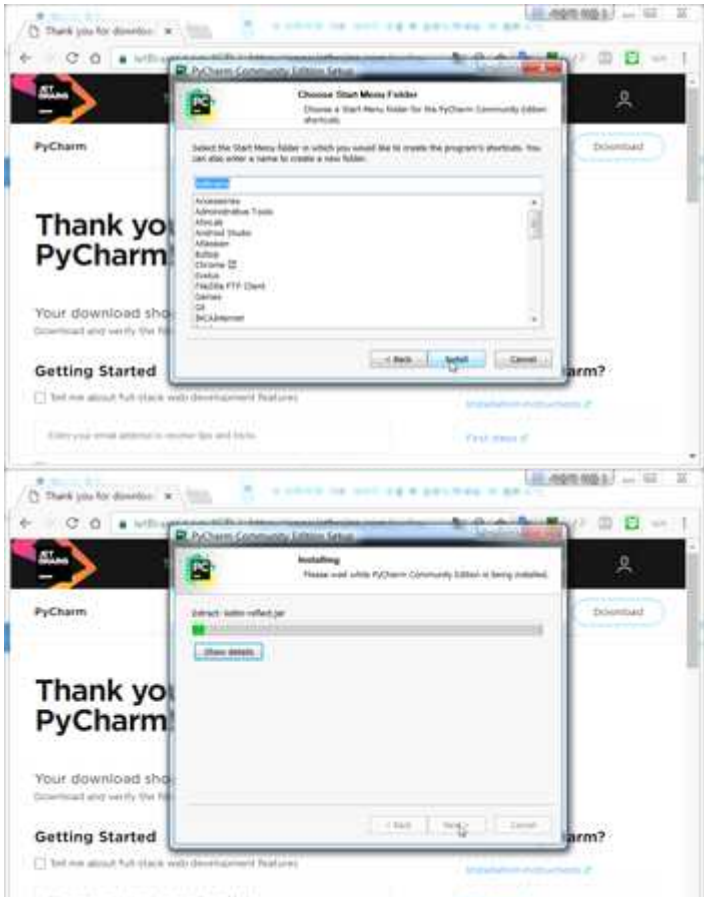
필요한 옵션을 선택하고 [Next]버튼을 클릭합니다.

Pycharm은 Java가 설치 되어있어야 실행이 됩니다.

설치되어 있지 않다면 하단의 Jre를 꼭 선택합니다.

시작 메뉴의 등록하는 화면이 나옵니다. 계속 [Next]버튼을 눌러 진행합니다.

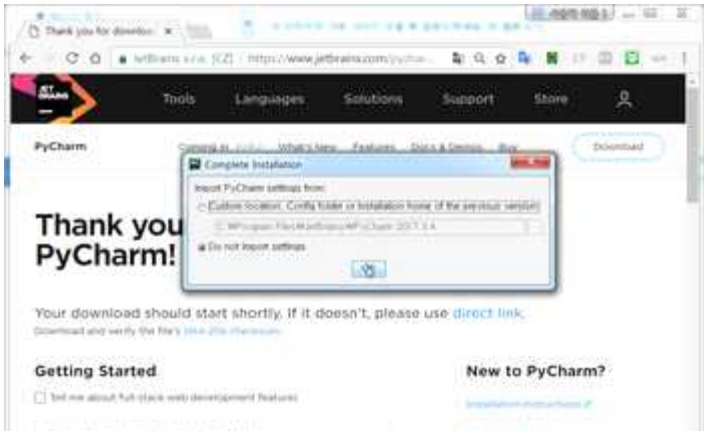
설치가 진행됩니다.



설치가 완료되면 PyCharm을 실행을 선택하고 설치를 종료합니다.

최초 실행할때 설정 정보를 읽어올것인 가를 선택합니다.

없으므로 "Do not ...."을 선택하고 [Ok]버튼을 선택합니다.



라이센스를 읽어보아야 다음으로 진행됩니다.

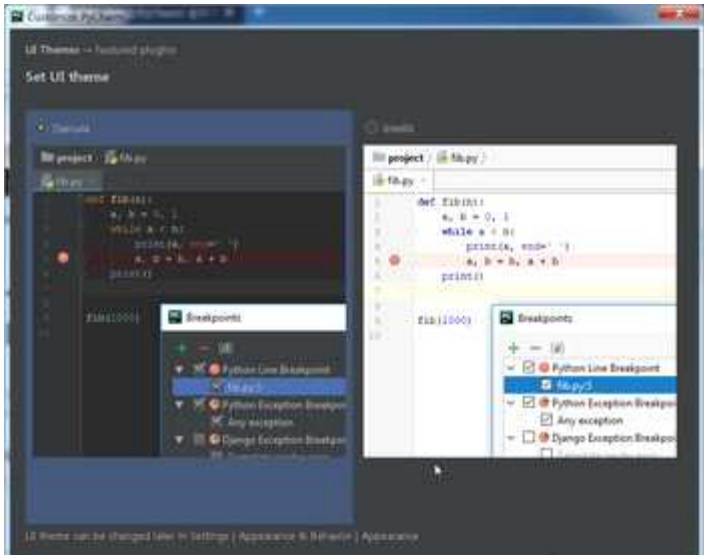
나타난 화면의 내용의 끝까지 스크롤하고 [Accept]버튼을 클릭 합니다.

UI Theme를 선택합니다.

하단의 왼쪽 버튼을 눌러 건너띄기를 하거나 우측 버튼을 눌러 추가적인 플러그인을 설치 합니다.

추가적인 플러그인 설치 화면입니다.

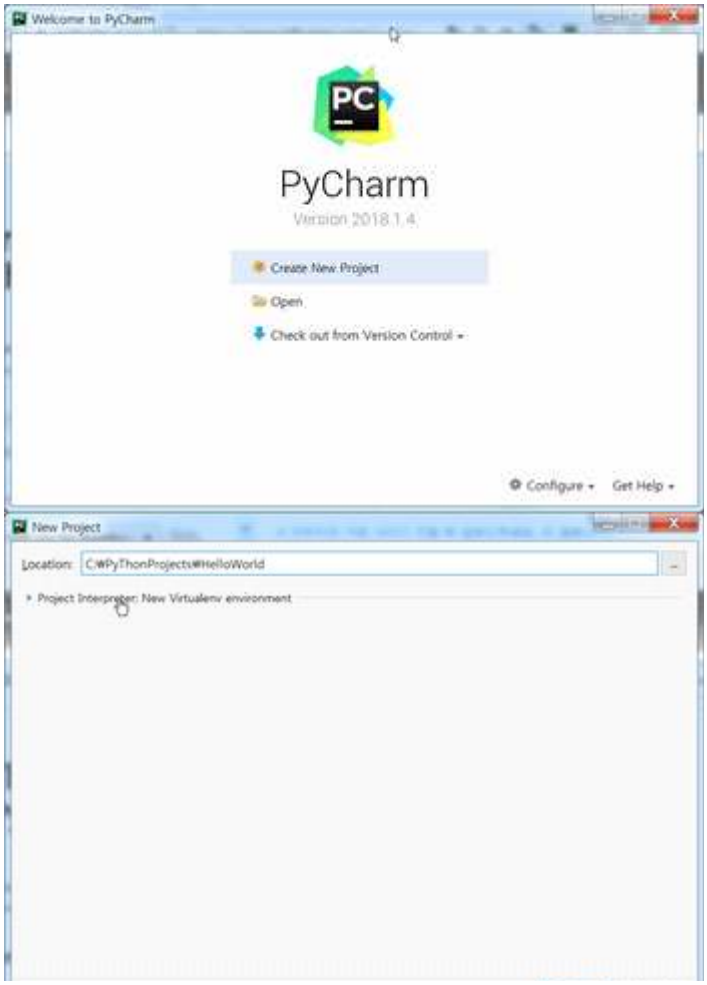
우측 하단의 [Start ...]버튼을 눌러 PyCharm을 시작합니다.



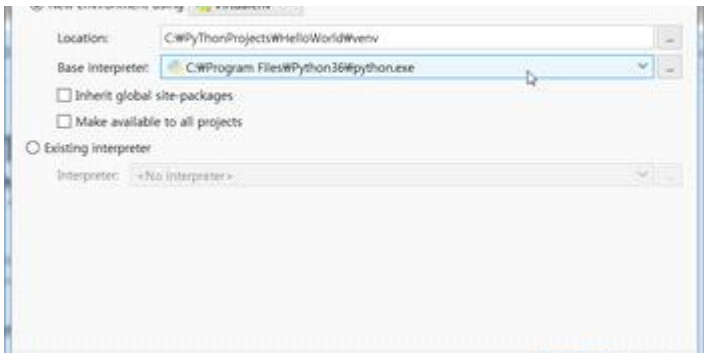
[Create New Project]를 클릭하여 프로젝트를 만듭니다.

나타나는 화면에서 저장할 위치를 지정합니다.

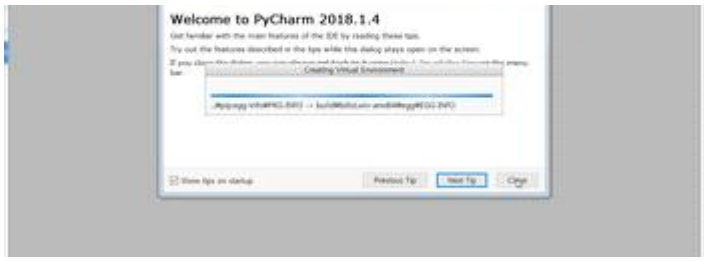
[Location]아래의 [Project Interpreter]를 클릭합니다.



인터프리터로 설치한 Python을 선택하고 하단의 [Create]버튼을 클릭합니다.



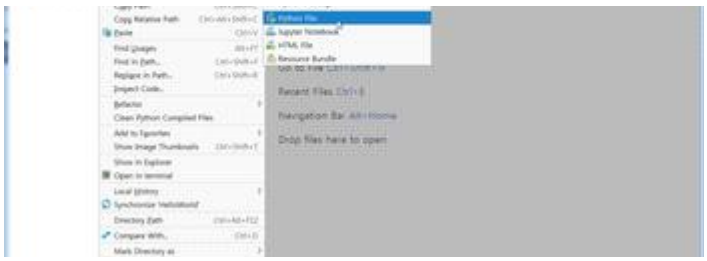
필요한 환경을 구성하면서 프로젝트가 만들어 집니다.



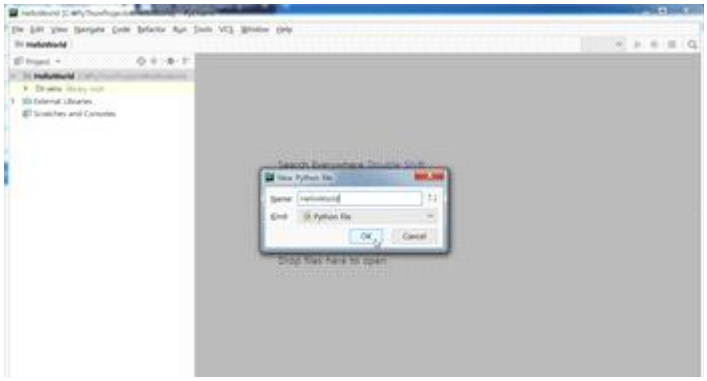
나타난 Tip화면의 좌측 하단의 [Show tips on startup]의 선택을 해제하고 [Close]버튼을 클릭합니다.

만들어진 프로젝트의 우측 프로젝트 창에서 마우스 우측 버튼을 클릭합니다.

[New]-[PythonFile]을 차례대로 선택합니다.

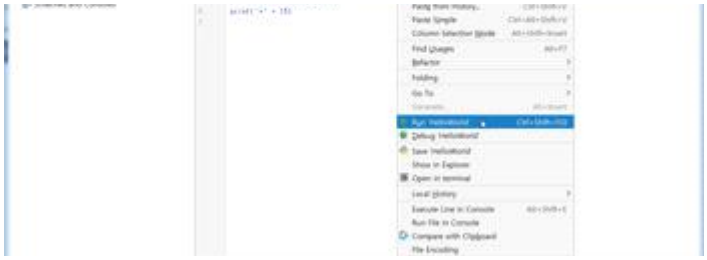


작성할 파일이름을 입력하고 [ok]버튼을 클릭합니다.



다음의 코드를 입력합니다.

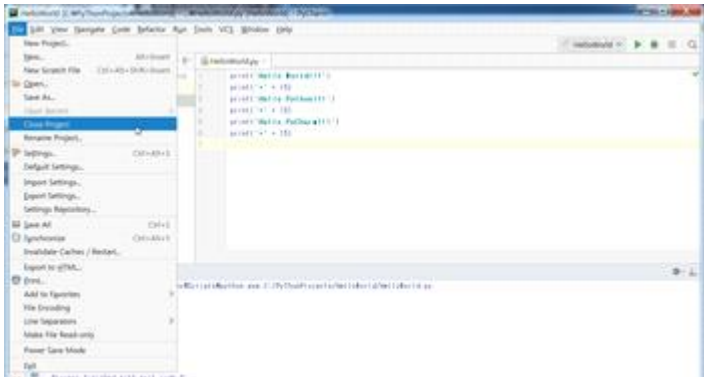
코드 위에서 마우스 우측 버튼을 클릭하여 [Run 'HelloWorld']를 선택합니다.

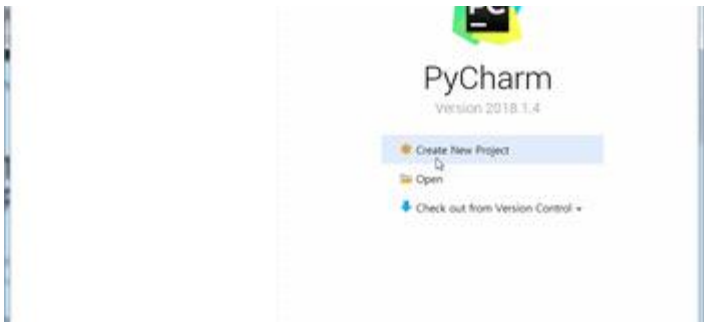


우측 하단에 결과창이 나타납니다.

[File] - [Close Project] 메뉴를 선택합니다. 프로젝트가 닫힙니다.

아래의 화면이 나오면 새로운 프로젝트를 만들거나 기존의 프로젝트를 Open합니다.

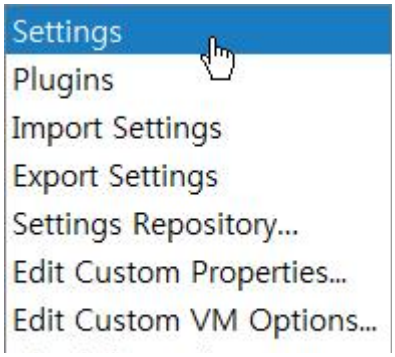




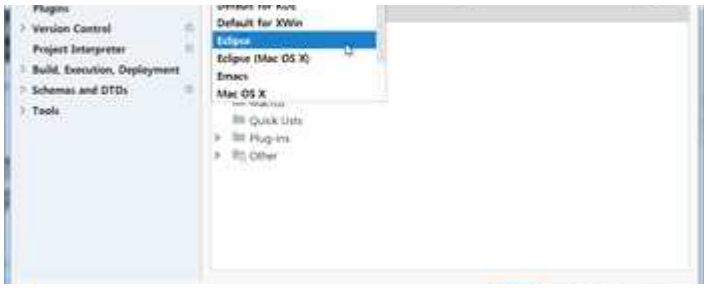
화면 하단의 [onfigure]를 선택하여 기본적인 설정 정보를 변경이 가능합니다.

화면 하단의 [configure]를 선택하고 [Settings]를 선택합니다.

좌측의 [Keymap]을 선택합니다.



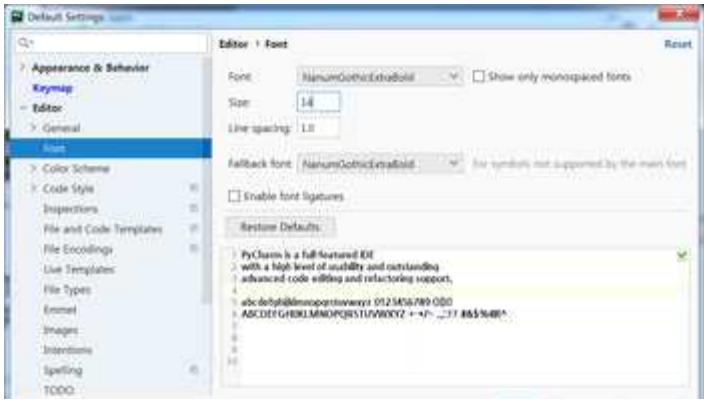
우측에서 사용자가 자주 사용하던 IDE의 단축키 키맵으로 변경이 가능합니다.



좌측의 [Editor]를 선택합니다.

우측의 [Font]를 선택합니다.

사용자가 원하는 폰트를 선택할 수 있습니다.



[File Encodings]를 선택합니다.

[UTF-8]로 변경합니다.

이미 프로젝트를 작성했다면 [File] - [Settings]메뉴에서 변경이 가능합니다.

마지막 편집일시 : 2018년 7월 5일 5:52

---

05. Python이란?

1. 프로그램이란?

사전적 의미로 프로그램을 살펴보면 다음과 같이 나타납니다.

진행 계획이나 순서 또는 그 목록.
순화어로는 계획(표)',차례(표)'.

그렇다면 컴퓨터에서 프로그램이란 무엇일까요?

어떤 문제를 해결하기 위해 컴퓨터에게 주어지는 처리 방법과 순서를 기술한 일련의 명령문의 집합체.

컴퓨터의 특징
신속성 : 입력된 데이터를 보다 빠른 시간 내에 처리하여 원하는 결과를 얻을 수 있음
대량성 : 한꺼번에 많은 양의 자료를 기억하거나 처리할 수 있음
정확성 : (신뢰성) 자료처리 과정에서 발생가능한 오차를 최소화하여 정확한 결과를 얻을 수 있음
호환성 : 다른 컴퓨터나 매체에서 작성한 자료도 공유하여 처리할 수있음
자동성 : 입력된 자료를 프로그램이라고 하는 정해진 처리 순서에 따라 자동으로 처리함
범용성 : 여러 가지 분야에 다양하게 사용됨
수동성 : 컴퓨터는 데이터를 주어진 프로그램에 따라 처리할 뿐 스스로 작업을 수행할 수 없음
위의 특징을 보면 많은 양의 데이터를 정확하고 빠르게 자동으로 처리할 수 있는 기계가 컴퓨터인것입니다. 그런데 마지막에 보이는 특징에 수동성이 있습니다. 그렇습니다. 컴퓨터가 아무리 좋아도 자기 스스로는 아무것도 할 수 없는 기계가 컴퓨터입니다.
그럼 이런 컴퓨터가 어떠한 작업을 수행하려면 누군가가 작업 순서를 차례대로 작성해서 이렇게 처리하라고 지시를 내려야만 컴퓨터가 작업을 처리하는 것 입니다. 이 작업지시서가 바로 컴퓨터 프로그램입니다.

결국 컴퓨터 프로그램이라는 것은 컴퓨터에게 작업을 지시하는 "작업지시서" 입니다.
컴퓨터에게 어떠한 작업을 시키기 위하여 컴퓨터가 알아들을 수 있는 명령어를 이용하여 컴퓨터에게 작업을 지시하는 작업지시서가 컴퓨터 프로그램입니다.

2. 프로그래밍 언어란?
프로그래밍 언어란 주어진 어떤 문제를 해결하기 위해 인간과 컴퓨터 사이에서 의사소통을 가능하게 하는 인공적인 언어입니다. 이 언어를 통하여 사용자는 컴퓨터에게 일련의 일을 시키는 명령어들의 집합체인 프로그램을 작성할 수 있습니다. 수많은 프로그램 언어가 있는데 공통적으로 프로그램 언어에 대해서 살펴보면 다음과 같은 특징을 가지고 있습니다.

프로그래밍 언어의 특징
간결성 : 사람이 프로그램을 쉽게 이해하고 읽을 수 있도록 간결하게 표현할 수 있다.
직교성 : 언어의 각 구성요소가 상호 독립적고 어떤 환경에서도 그 구성요소가 같은 의미로 사용된다.
가독성 : 사람이 이해하기 쉽도록 작성된 문법, 주석 등이 가독성의 향상에 도움이 된다.

정확성 : 잘 정의된 문법은 정확성을 보장한다. 각 언어의 문법은 대부분이 세계 표준이다.
기계 독립성 : 서로 다른 컴퓨터 상 에서 항상 같은 결과를 요구한다.
프로그래밍 언어의 종류
저급언어 : 컴퓨터 개발 초창기에 사용되던 프로그래밍 언어입니다.
기계어와 어셈블리어로 나눕니다.
고급언어 : 사용자 중심의 언어로 사람이 쓰는 자연어에 가까운 프로그래밍 언어입니다.
C/C++, JAVA, Python, Basic 기타...
고급언어로 만들어진 모든 프로그램은 실행되기 전 컴퓨터가 이해할 수 있는 기계어로 변환되어야 하는데, 이를 위해서 컴파일러나 인터프리터를 이용하여 코드를 변환합니다. 이 둘의 차이점은 컴파일러는 한 프로그램을 한꺼번에 번역한 뒤 실행하며, 인터프리터는 명령어 단위로 한 행씩 번역해가며 실행한다는데 있습니다. C/C++, Java등은 컴파일러 언어이고 Python,Basic 등은 인터프리터 언어 입니다.

3. Python언어란?
파이썬(Python)은 1991년 프로그래머인 귀도 반 로섬(Guido van Rossum)이 발표한 고급 프로그래밍 언어로, 플랫폼 독립적이며 인터프리터식, 객체지향적, 동적 타이핑(dynamically typed) 대화형 언어입니다. 파이썬이라는 이름은 귀도가 좋아하는 코미디 "몬티 파이썬의 날아다니는 서커스(Monty Python's Flying Circus)"에서 따온 것이라고 합니다.
파이썬의 사전적 의미는 고대 신화에 나오는 파르나소스 산의 동굴에 살던 큰 뱀을 뜻하며, 아폴로 신이 델파이에서 파이썬을 퇴치했다는 이야기가 전해지고 있습니다. 그래서 대부분의 파이썬 책 표지와 아이콘이 뱀 모양으로 그려져 있는 이유가 여기에 있습니다.
파이썬은 비영리의 파이썬 소프트웨어 재단이 관리하는 개방형, 공동체 기반 개발 모델을 가지고 있습니다. C언어로 구현된 C파이썬 구현이 사실상의 표준입니다.

주요 특징
실행 시간에 자료형을 검사하는 동적 타이핑(dynamic typing)을 지원합니다.
객체의 멤버에 무제한으로 접근할 수 있습니다.
모듈, 클래스, 객체와 같은 언어의 요소가 내부에서 접근할 수 있습니다.
리플렉션을 이용한 기술을 쓸 수 있습니다.

해석 프로그램의 종류
C파이썬 : C로 작성된 인터프리터.
스택리스 파이썬 : C 스택을 사용하지 않는 인터프리터.
자이썬(제이파이썬 : JPython) : 자바 가상 머신 용 인터프리터.
IronPython : .NET 플랫폼 용 인터프리터.
PyPy : 파이썬으로 작성된 파이썬 인터프리터.

기능과 철학
파이썬은 다양한 프로그래밍 패러다임을 지원하는 언어입니다. 객체 지향 프로그래밍과 구조적 프로그래밍을 완벽하게 지원하며 함수형 프로그래밍, 관점 지향 프로그래밍 등도 주요 기능에서 지원 됩니다.

파이썬의 핵심 철학은 PEP 20 문서에 잘 정리되어 있습니다.
https://www.python.org/dev/peps/pep-0020/

The Zen of Python
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
"아름다운게 추한 것보다 낫다." (Beautiful is better than ugly)
"명시적인 것이 암시적인 것 보다 낫다." (Explicit is better than implicit)
"단순함이 복잡함보다 낫다." (Simple is better than complex)
"복잡함이 난해한 것보다 낫다." (Complex is better than complicated)
"가독성은 중요하다." (Readability counts)
파이썬은 언어의 핵심에 모든 기능을 넣는 대신, 사용자가 언제나 필요로 하는 최소한의 기능만을 사용하면서 확장해나갈 수 있도록 디자인되었습니다. 이것은 펄의 TIMTOWTDI(there's more than one way to do it - '문제를 해결하는 방법은 단 한 가지가 아니다') 철학과는 대조적인 것이며, 파이썬에서는 다른 사용자가 썼더라도 동일한 일을 하는 프로그램은 대체로 모두 비슷한 코드로 수렴합니다. 라이브러리는 기본 기능에 없는 많은 기능을 제공합니다.

또, 파이썬에서는 프로그램의 문서화가 매우 중시되고 있어 언어의 기본 기능에 포함되어 있습니다. 파이썬은 원래 교육용으로 설계되었기 때문에 읽기 쉽고, 그래서 효율적인 코드를 되도록 간단하게 쓸 수 있도록 하려는 철학이 구석 구석까지 침투해 있어, 파이썬 커뮤니티에서도 알기 쉬운 코드를 선호하는 경향이 강합니다.

현재 많은 언어들이 사용되고 있지만 TIOBE 프로그래밍 커뮤니티 인덱스(TIOBE Programming Community Index (Monthly))에 방문해 보시면 Python의 위치를 확인하실 수 있을것입니다.

마지막 편집일시 : 2018년 7월 5일 5:52----------
---

06. Python(파이썬) 기본

주석 , 들여쓰기, 행결합/행분리

앞으로의 모든 실습은 PyCharm을 이용하도록 하겠습니다.

4. 01. 주석

1) 1. 주석이란?

주석(註釋)의 사전적의미부터 살펴보면 "낱말이나 문장의 뜻을 쉽게 풀이함 또는 그런글"이라고 나옵니다. 영어로는 annotation, footnote, comment 등으로 불립니다. 프로그램에서 주석이란? 프로그램의 실행에는 전혀 관여하지않으면서 프로그램 코드 중간 중간에 코드에 대한 자세한 설명을 첨부하는것을 말합니다.

2) 2. 주석이 왜 필요한가? 주석의 필요성

주석의 의미는 말그대로 문장의 뜻을 쉽게 풀이한 것입니다. 그럼 이것이 프로그램에서 "왜 필요한가?" 입니다. 주석은 C 언어, Python 이든 데이터베이스든 어느곳에서도 사용됩니다. 사용자가 만든 프로그램을 다른 사람에게 해석시키려면 적은 코드에도 적지 않은 시간과 노력이 소요됩니다. 재미있는 것은 자기자신이 구현한 프로그램인데도 분석하는데 상당한 시간이 걸린다는 것입니다. 실무에서는 혼자 프로그래밍 하는게 아닌 여러 사람이 팀으로 작업을 합니다. 팀원 서로간에 코드에 대한 공유할때나 개발이 완료되어 운영팀으로 넘길때 또는 개발 중간에 자신의 작업을 다른 팀원에게 인수인계를 해야 된다면 어떻게 되겠습니다. 코드에 주석이 없다면 상당한 시간과 노력을 필요로 할것 입니다. 이제 주석의 필요성을 이해 하셨습니까? 주석을 적용하면 해당 부분은 컴퓨터가 읽지 않습니다. 그러므로 신경 쓸 필요가 없으니 안심하고 주석을 달도록 하십시요.

3) 3. 주석의 종류

주석은 블록단위와 한 줄(행) 단위로 주석을 만들 수 있습니다.

- 블록단위 : 작은 따옴표(')를 연달아 세개를 사용합니다.

- 사용법 : ''' 주석처리 적용문장 '''
- 사용예
- '''주석처리 적용 문장 1
- 주석처리 적용 문장 2
- 주석처리 적용 문장 3
- 주석처리 적용 문장 4
- '''

- 행단위 : #을 사용합니다.
  - 사용법 : # 주석처리 적용 문장
  - 사용예
  - #주석처리 적용 문장 1
  - #주석처리 적용 문장 2
  - #주석처리 적용 문장 3

블록단위란 말그대로 많은 분량의 주석, 시작과 끝을 명시하는 주석 방식이라고 보면 됩니다.

행단위는 초심자들을 위해 한줄 한줄마다 무슨 뜻인지 명시하는 주석 방식이라고 보면 됩니다.

※ 주의 : 블록 주석은 중첩 불가합니다.

4) 4. 주석의 사용 예

까끔 큰따옴표(") 3 개를 연속으로 사용하여 주석으로 사용하는데 이는 엄밀히 말하면 주석이아니고 Document String 이라고 합니다. 모듈, 함수, 클래스, 메서드등을 만들때 자체에 설명서를 등록하는 역활을 합니다.

다음은 주석의 사용 예를 보여준 코드입니다. 블록 단위 주석과 행 단위 주석을 적절하게 사용하면 코드 자체가 문서로서의 의미를 가지게 됩니다.

comment.py

```
'''

작은 따옴표 3 개를 연속으로 입력하면 Block comment 라고 합니다.

프로그램명 : comment1.py

작 성 자 : 한 사 람

작 성 일 : 2018 년 8 월 1 일

설  명 : 파이선에서 주석을 어떻게 처리하는지 보여주는 예제

'''



#  random 모듈을 사용할 수 있도록 포함합니다. # 주석 # 주석

import random



#-------------------------------------------------------------
--
```

```
# 함수 정의 시작

def lotto():

    """

    로또 번호를 생성해 줍니다.

    ------------------------------

    호출 방법 : lotto()

    1~45 사이의 정수 6 개를 리스트로 리턴합니다.

    :return : <class 'list'>

    """

    # 1~45 사이의 정수 중에서 6 개를 무작위로 추출합니다.

    lotto =  random.sample(range(1,46),6)

    lotto.sort() #  오름차순으로 정렬합니다.

    return lotto # 결과를  호출항 곳으로 돌려줍니다.

# 함수 정의 끝

#-----------------------------------------------------------------
--

# 함수 호출하여 결과를 바로 출력합니다.
```

```
print(lotto())



# 도큐멘트 스트링 사용방법

print(lotto.__doc__)   # 도큐멘트 스트링 출력

help(lotto) # 도움말 함수 사용
```

실행 결과

```
[13, 16, 29, 33, 35, 45]



    로또 번호를 생성해 줍니다.

    ------------------------------

    호출 방법 : lotto()

    1~45 사이의 정수 6 개를 리스트로 리턴합니다.

    :return : <class 'list'>



Help on function lotto in module __main__:



lotto()
```

```
로또 번호를 생성해 줍니다.

------------------------------

호출 방법 : lotto()

1~45 사이의 정수 6 개를 리스트로 리턴합니다.

:return : <class 'list'>
```

마지막 편집일시 : 2018 년 7 월 6 일 10:23

---

5. 02. 들여쓰기(indent)

1) 1. 들여쓰기(Indent)

코딩을 할 때, 가독성을 위해 들여쓰기를 합니다. 다른 프로그램 언어에서는 {}을 사용하여 영역을 지정합니다. 하지만 파이썬은 들여쓰기를 사용하여 영역을 지정합니다. 그렇기 때문에 파이썬에서 들여쓰기란 문법적인 강제사항입니다.

코드 블럭을 구성하기 위해 if, for, class, def 등등 을 작성하면서 나오는 : 다음 아랫줄은 반드시 들여쓰기를 해야합니다. 들여쓰기의 방법은 한칸, 두칸, 4 칸, 탭 등 여러가지 방식이 있습니다.

중요한 것은 같은 블록 내에서는 들여쓰기 칸 수가 같아야 합니다.
위반시에는 "IndentationError: unexpected indent"라는 에러를 출력합니다.

```
"""

들여쓰기 연습

---------------

2018-08-01

"""
```

```python
def line():
    """
    line 함수 : 30 개의 -를 출력합니다.

    형식) line()
    """
    print("-" * 30)


def show(char="★", count=5):
    """
    show 함수 :    지정문자를 지정한 갯수만큼 출력합니다.

    형식) showshow(char, count)
    첫번째 인수 : 출력한 문자(기본값  : ★)
    두번째 인수 : 출력할 개수(기본값 : 5)
    """
```

```
    print(char * count)

    line() # 이 줄의 들여쓰기 칸수를 변경해보세요!! 에러 확인



show()

show("◆")

# 인수에 기본값을 정해줬을 경우 뒤에서 부터 생략가능 : 30*5 = 150 출력

show(30)

show("○",10)
```

C 언어의 들여쓰기

```
 int factorial(int x) {

     if(x == 0)     {

         return 1;

     }     else     {

         return x * factorial(x - 1);

     }

 }
```

```
int factorial(int x){

    return (x==0) ? 1 :  x * factorial(x - 1);

}
```

Python 의 들여쓰기

```
def factorial(x):

     if x == 0:

         return 1

     else:

         return x * factorial(x - 1)

def factorial(x):

    return 1 if x==0 else x * factorial(x - 1)
```

마지막 편집일시 : 2018 년 7 월 5 일 5:53

---

03. 행결합/행분리

행결합 : 세미콜론(;)

많은 언어들이 구문이 끝날때 문장의 뒤에 구문의 종료를 알리는 세미콜론(;) 을 적습니다. 하지만 파이썬은 구문이 끝나고 다음 줄로 내려갈 때 세미콜론이 필요 없습니다. 줄이 바뀌면 자동으로 구분이 종료되었음을 인식 합니다.

세미콜론을 붙여도 에러는 나지 않습니다. 여러 구문을 이어쓸때는 세미콜론을 쓰기도 합니다.

```
# 세미콜론(semicolon) 사용법
print("Hello Python!!")
print("Hello Python!!");
# print("첫번째 줄!!") print("두번째 줄") # 에러발생
print("첫번째 줄!!"); print("두번째 줄")
```

print("첫번째 줄!!"); print("두번째 줄");
행분리 : (역슬레쉬), 괄호
1줄의 내용이 길어서 여러줄로 타이핑 해야 할경우에는 (역슬레쉬)나 괄호를 이용합니다.

```
# 긴 내용을 한줄인것 처럼 인식 시키기(\ 사용)
a = 1 + 2 + 3 + \
      4 + 5 + 6
# 긴 내용을 한줄인것 처럼 인식 시키기(괄호 사용
b = (1 + 2 + 3 +
        4 + 5 + 6)
#  파라메터를 넘기는 것이라면 콤마(comma) 뒤에서 그냥 엔터를 쓰면 됩니다.
print("Hello",  "Python",
           end="\n", sep="\t")

# 문자열 중간에 줄바꿈을 하면 자동으로 ""가 열고닫히며 1줄로 인식한다.
print("아주 아주 아주 긴 문자열이라고 가정하자 "
          "한줄에 모두 타이핑이 힘들다면")
```

마지막 편집일시 : 2018년 7월 16일 10:50
---

07. Turtle(거북이) 따라해보기

1. Turtle graphics란?

Python 사이트의 라이브러리 설명을 보면 “Turtle graphics는 아이들에게 프로그래밍을 소개하는 인기 있는 방법입니다. 그것은 1966 년 Wally Feurzig와 Seymour Papert가 개발한 오리지널 로고 프로그래밍 언어의 일부입니다.” 라고 설명하고 있습니다.
거북이 그래픽(Turtle Graphic)은 위의 설명처럼 Logo라는 프로그래밍 언어의 일부로 개발된 컴퓨터에 그래픽을 표시하는 하나의 방식입니다. 꼬리에 잉크를 묻인 거북이를 조작하여 그림을 그릴 수 있습니다. 거북이 그래픽(Turtle Graphic)은 이해하기 쉽고 단순합니다. 그리고 직관적입니다. 프로그램을 모르는 초보자도 아주 쉽게 접할 수 있습니다. 따라하다 보면 프로그램을 이해하는데 많은 도움이 되니 하나하나씩 따라하면서 프로그램이 어떻게 작동하는지에 대한 이해를 넓힐 수 있으면 합니다.

2. Turtle graphics 사용

거북이 그래픽(Turtle Graphic)은 Python의 기본 모듈이므로 Python만 설치하면 바로 사용이 가능합니다. 그렇다고 그냥 사용하는 것은 아니고 거북이 그래픽(Turtle Graphic)을 사용하겠다고 먼저 알려주어야 합니다.

Python에서는 어떠한 기능을 사용하기 위해서는 기능을 가지고 있는 모듈을 추가하여야 합니다. 이때 사용하는 명령어가 import 명령어입니다. 다음의 문장은 거북이 그래픽(Turtle Graphic)을 사용하겠다고 알려주는 역할을 담당합니다.

```
import turtle
```

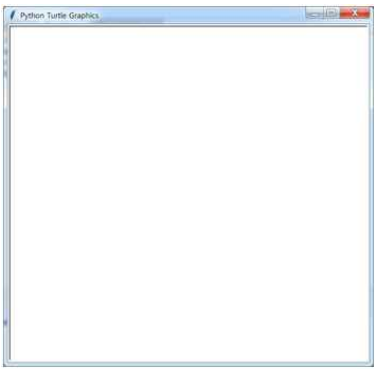
또는
import turtle as trtl
위의 첫 번째 문장은 turtle을 사용하겠다는 것으로 “turtle.shape(‘turtle’)” 형식으로 사용한다는 뜻이며 두 번째 문장은 turtle을 trtl로 줄여서 “trtl.shape(‘turtle’)” 형식으로 사용한다는 뜻입니다.

첫 번째 예제부터 작성해 보도록 하겠습니다. 여기에서는 컴퓨터 프로그램이 어떻게 작성되고 어떻게 실행되는구나 하는 것을 개략적으로 이해하는데 중점을 두도록 하겠습니다. 무작정 따라하다 보면 아~~ 프로그램을 이런 식으로 작성하고 이렇게 작동되는구나 하는 방법을 아시게 될 것입니다.

마지막 편집일시 : 2018년 7월 5일 5:52
---
01. 하나씩 따라해보기
turtle을 사용하기 위하여 라이브러리를 임포트하고 일시정지를 시키면 윈도우가 나타납니다.

Ex01.py

```
# 거북이를 사용하자
import turtle
# 일시정지한다.
turtle.done()
```
실행결과



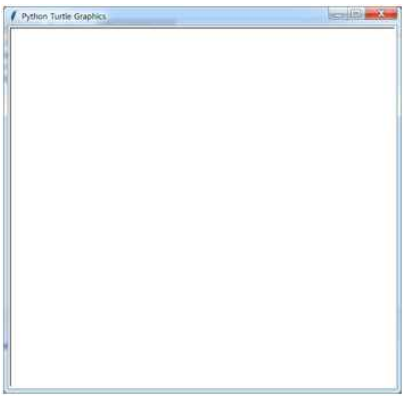
위와 같은 윈도우가 나타납니다. 당연히 우측 상단의 [X]를 누르면 종료합니다. 대단하지 않습니까? 거북이를 사용하겠다고 하고 일시정지 시키는 done()이라는 메서드 하나만을 호출 하였는데 윈도우가 만들어 졌습니다.

프로그램을 조금 변경해보도록 하겠습니다. [X]를 누르지 않고 마우스로 클릭하면 종료되게 하려면 어떻게 할까요? 마지막 줄의 done()을 exitonclick()으로 변경하고 실행해 보겠습니다.

Ex02.py

```
# 거북이를 사용하자
import turtle
# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행결과

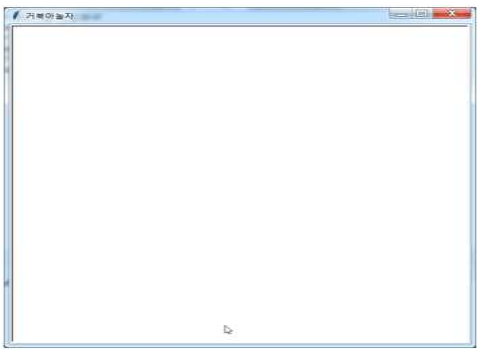


이제는 윈도우의 제목을 변경해보도록 하겠습니다.

Ex03.py

```
# 거북이를 사용하자
import turtle
# 윈도우의 제목을 변경해보자
turtle.title('거북아놀자')
# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행결과



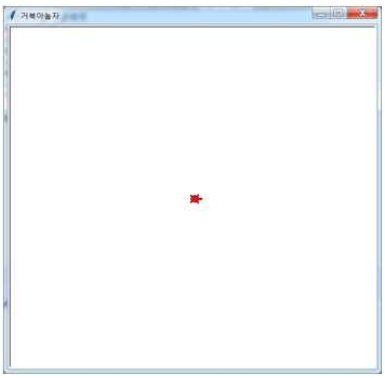
화면에 아무것도 나타나지 않았습니다. 무언가를 나타나게 해보겠습니다. ‘거북아 놀자’로 했

으니 거북이가 나타나도록 해보겠습니다. 아래와 같이 만들고 실행해 보도록 하겠습니다. 색상도 변경해 보도록 하겠습니다.

Ex04.py

```
# 거북이를 사용하자
import turtle
# 윈도우의 제목을 변경해보자
turtle.title('거북아놀자')
# 거북이 색상을 변경해보자
turtle.color('black','red')
# 거북이를 나타나게하자
turtle.shape('turtle')
# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행 결과

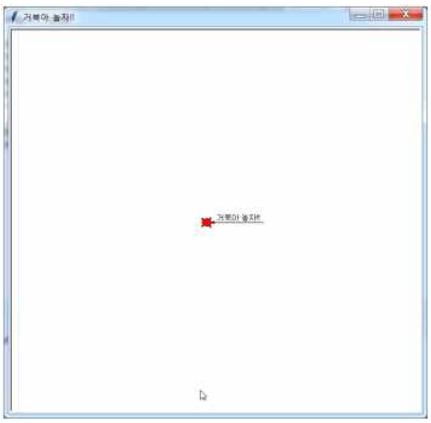


이제는 문자열을 출력해 보도록 하겠습니다. 그리고 거북이를 이동해 보도록 하겠습니다.

Ex05.py

```
import turtle

turtle.title('거북아 놀자!!') # 윈도우 제목
turtle.shape('turtle') #모양을 거북이로
turtle.color('black','red') # 색상변경
turtle.penup(); # 펜 들어
turtle.write("거북아 놀자!!") # 문자열 출력
turtle.forward(80) # 앞으로 80만큼 이동
turtle.pendown(); # 펜 내려
turtle.backward(100) # 뒤로 100만큼 이동
```

turtle.exitonclick() # 마우스로 클릭시 윈도우 종료
실행 결과



소스를 아래처럼 변경해도 같은 결과를 얻습니다.

Ex05_1.py

```
import turtle

turtle.title('거북아 놀자!!') # 윈도우 제목
turtle.shape('turtle') #모양을 거북이로
turtle.color('black','red') # 색상변경
turtle.pu(); # 펜 들어
turtle.write("거북아 놀자!!") # 문자열 출력
turtle.fd(80) # 앞으로 80만큼 이동
turtle.pd(); # 펜 내려
turtle.bk(100) # 뒤로 100만큼 이동
turtle.exitonclick() # 마우스로 클릭시 윈도우 종료
```

거북이를 어떻게 이동을 할까요?

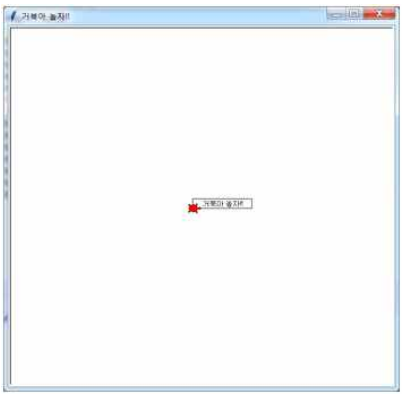
거북이 이동에는 다음의 메서드를 사용합니다.
앞으로 : forward(이동거리) 또는 fd(이동거리)
뒤로 : backward(이동거리) 또는 bk(이동거리) 또는 back(이동거리)
방향전환 : right(각도) 또는 rt(각도), left(각도) 또는 lt(각도)
그리기/이동하기 : penup() 또는 pu(), pendown() 또는 pd()
※ 펜을 들고이동하면 이동하는 선이 그려지지않고 내리고 이동하면 선이 그려집니다.

이제는 조금 응용하여 문자열 둘레에 선을 그려보도록 하겠습니다.

Ex06.py

```
import turtle

turtle.title('거북아 놀자!!') # 윈도우 제목
turtle.shape('turtle') #모양을 거북이로
turtle.color('black','red') # 색상변경
turtle.pu(); # 펜 들어
turtle.write("거북아 놀자!!") # 문자열 출력
turtle.fd(80) # 앞으로 80만큼 이동
turtle.pd(); # 펜 내려
turtle.bk(100) # 뒤로 100만큼 이동
turtle.left(90); turtle.fd(18); # 왼쪽으로 90도 회전 앞으로 18이동
turtle.right(90); turtle.fd(110); # 오른쪽으로 90도 회전 앞으로 110이동
turtle.right(90); turtle.fd(18); # 오른쪽으로 90도 회전 앞으로 18이동
turtle.right(90); turtle.fd(110); # 오른쪽으로 90도 회전 앞으로 110이동
turtle.left(180); #  180도 회전
turtle.exitonclick() # 마우스로 클릭시 윈도우 종료
```

실행 결과



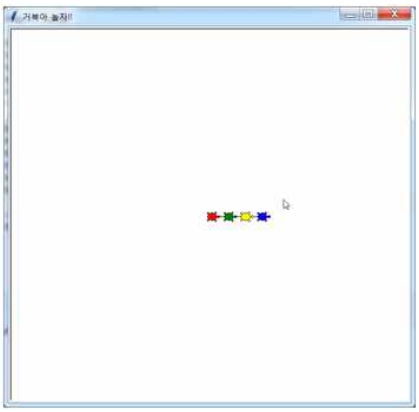
이동하면서 색상을 바꿔보고 도장을 찍어보면 어떻게 될까요?

Ex07.py

```
import turtle

turtle.title('거북아 놀자!!') # 윈도우 제목
turtle.shape('turtle') #모양을 거북이로
turtle.color('black','red') # 거북이 색상을 변경해보자
turtle.stamp() # 도장을 찍어보자
turtle.forward(30)
```

```
turtle.color('black','green') # 거북이 색상을 변경해보자
turtle.stamp() # 도장을 찍어보자
turtle.forward(30)
turtle.color('black','yellow')
turtle.stamp()
turtle.color('black','blue')
turtle.forward(30)
turtle.stamp()

# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행결과



네 마리의 거북이가 색상이 변경되어 나타났습니다. 몇 가지 예를 들어서 프로그램을 작성해 보았습니다. 어떻습니까? 프로그램 별거 아니죠? 그렇습니다. 프로그램이란 “컴퓨터에게 시키고 싶은 일을 차례대로 컴퓨터가 알아들을 수 있는 언어로 기술한 작업지시서”입니다.

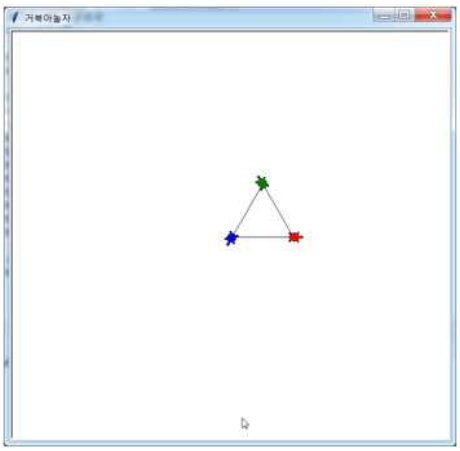
이번에는 삼각형을 그려보도록 하겠습니다.

Ex08.py

```
# 거북이를 사용하자
import turtle

turtle.title('거북아놀자')    # 윈도우의 제목을 변경해보자
turtle.shape('turtle')        # 거북이를 나타나게하자

turtle.color('black','red') # 거북이 색상을 변경해보자
turtle.forward(100)
turtle.stamp()                # 도장을 찍어보자
```

```
turtle.left(120)
turtle.color('black','green') # 거북이 색상을 변경해보자
turtle.forward(100)
turtle.stamp()
turtle.left(120)
turtle.color('black','blue')
turtle.forward(100)
turtle.stamp()

# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행 결과



프로그램 언어를 배우는 학생들을 보면 프로그램을 굉장히 어려운 것으로 생각하는데 생각을 다시해 보시기 바랍니다. 컴퓨터는 아무것도 스스로 할 수 없는 전자제품에 불과합니다. 컴퓨터가 어떠한 일을 하기 위해서는 사용자가 컴퓨터에게 일을 하도록 작업을 지시해야만 합니다. 이 작업을 지시하는 언어가 “프로그램언어”이고 그 언어로 작성한 작업 지시서가 바로 “프로그램”입니다. 여러분 쉽게 생각해야 배우는 것도 쉬운 법입니다.

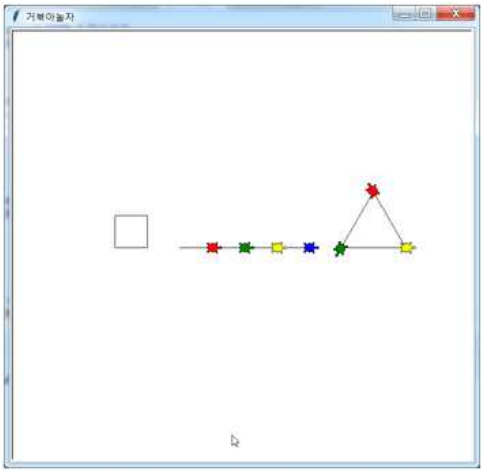
그런데 위의 예제들을 따라하면서 느끼는 의문점은 없었나요? 위의 예제를 보면 무언가 냄새가 난다는 생각이 들지 않나요? 프로그램계에서는 “코드 스멜(code smell)”이라는 용어가 있습니다. 코드 스멜(code smell←코드 냄새)은 컴퓨터 프로그래밍 코드에서 더 심오한 문제를 일으킬 가능성이 있는 프로그램 소스 코드의 증상을 나타내는 용어입니다. 무슨 냄새가 날까요? 다음의 코드로 고쳐보면 어떻게 될까요?

Ex09.py

```
# 거북이를 사용하자
import turtle
```

```
turtle.title('거북아놀자')    # 윈도우의 제목을 변경해보자
turtle.color('black','red') # 거북이 색상을 변경해보자
turtle.shape('turtle')        # 거북이를 나타나게하자

# 함수 정의
def move(length):
    turtle.penup()
    turtle.forward(length)
    turtle.pendown()

move(-200) # 함수호출
# 사각형 그리기
for _ in range(4): # 반복문
    turtle.forward(50)      # 앞으로 100 이동
    turtle.left(90) # 왼쪽(시계방향)으로 방향전환

move(100)
# 도장 찍기
colors = ['red','green','yellow', 'blue'] # 리스트
for color in colors:
    turtle.color('black',color) # 거북이 색상을 변경해보자
    turtle.forward(50)
    turtle.stamp()

move(150)
# 삼각형그리기
for i in range(3):
    turtle.left(120)
    turtle.color('black', colors[i])  # 거북이 색상을 변경해보자
    turtle.forward(100)
    turtle.stamp()  #

# 클릭시 종료되게 하자
turtle.exitonclick()
```

실행 결과



코드가 많이 짧아졌습니다. 그리고 많은 기능이 부여되었습니다. 왜 이렇게 되었을까요? 자주 사용되는 코드를 별도의 함수로 작성하였으며 복사해서 붙여 넣었던 코드를 반복문을 이용하여 간단하게 줄였습니다. 또 하나는 색상 값을 리스트라는 자료구조를 이용하여 저장한 후 재사용을 하였습니다. 여기에서 변수, 함수, 반복문 이러한 것들이 나타나는데 이것을 정확하게 배워야 짧으면서 많은 일을 하는 프로그램을 작성할 수 있겠구나 하는 생각이 들지 않나요?

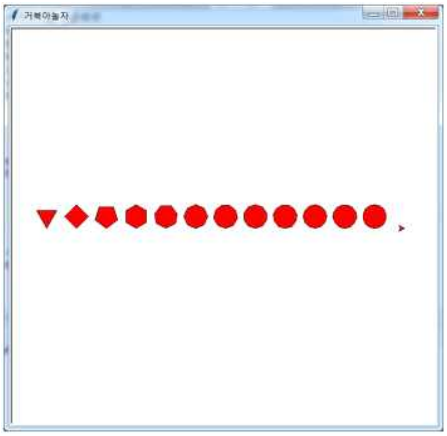
이제 Python을 제대로 배워보겠습니다!! 문법과 사용법을 익혀서 좀 더 좋은 프로그램을 만들자!!!

보너스 하나
Ex10.py

```
# 거북이를 사용하자
import turtle
turtle.title('거북아놀자')    # 윈도우의 제목을 변경해보자
turtle.color('black','red') # 거북이 색상을 변경해보자
# turtle.shape('turtle')        # 거북이를 나타나게하자
# 함수 정의
def move(length):
    turtle.penup()
    turtle.forward(length)
    turtle.pendown()

move(-300)
for i in range(3,15):
    turtle.begin_fill()
    turtle.circle(20,steps=i)
    move(50)
```

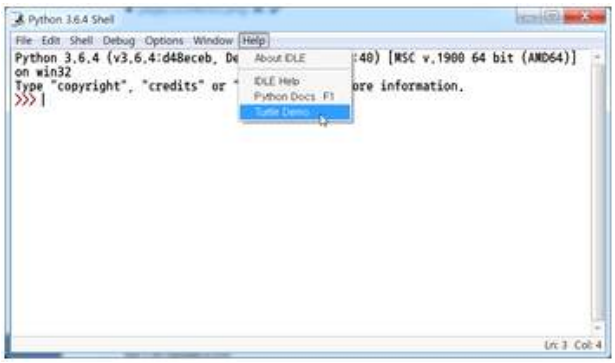
```
    turtle.end_fill()

# 클릭시 종료되게 하자
turtle.exitonclick()
```

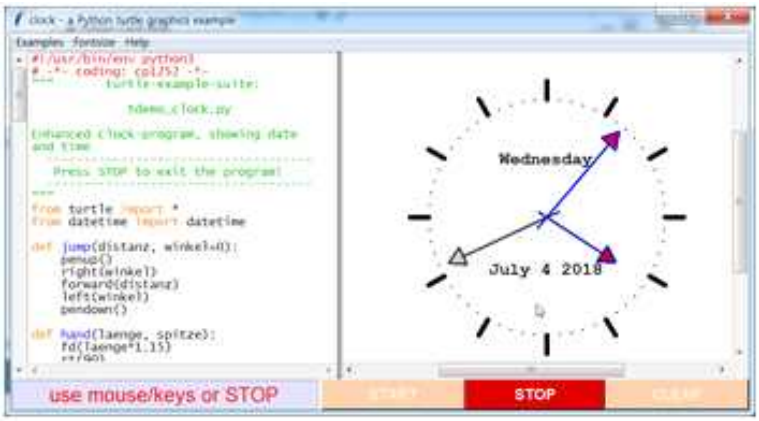
실행 결과



보너스 둘

파이선에서 제공되는 IDLE의 [Help]메뉴의 [Turtle Demo]를 보시면 많은 예제가 있습니다.
하지만 아직까지 소스를 이해하는데는 문제가 있겠죠?
전체적인 프로그램의 구조를 참조하는 정도에서 한번 정도 실행해 보세요





마지막 편집일시 : 2018년 7월 5일 5:53

제 02장 두번째 계단밟기
Python(파이썬) 기초 다지기

6. 01. Python(파이썬) 표준 입출력

왜! 모든 프로그램은 배울 때 처음에 출력문을 배우는것일까요?

프로그램이라는 것이 **작업지시서**라고 했습니다. 그렇다면 컴퓨터로 작업을 지시하고 컴퓨터가 처리한 결과를 확인할 방법이 있어야 겠지요. 그래서 처음에 출력문을 배우는것 입니다. 여기에서 배우는 출력문은 사실 그렇게 중요하지 않습니다. 많이 배우고나면 출력을 어디로할것인가에 따라 프로그램이 변경되며 그때 출력 대상에따라 다양한 출력이 생성되기 때문입니다.

- 콘솔(Console)에 출력하면 콘솔 어플리케이션(Console Application)이라고 합니다.
- 웹브라우저(Web Browser)에 출력하면 웹 어플리케이션(Web Application)이라고 합니다.
- 스마트기기에 출력하면 App(앱)이라고 합니다.
- 윈도우(Window)에 출력하면 윈도우 어플리케이션(Window Application)이라고 합니다.

콘솔이란 대개 디스플레이 모니터와 키보드(또는 입력이 가능한 다른 장치)가 조합된 장치를 말한다. 콘솔을 다른 말로는 터미널이라고 부른다. 콘솔이라는 용어는 마이크로컴퓨터나 메인프레임에 부착되어 있어, 시스템의 상황을 모니터하는데 사용되는 터미널을 의미한다.

마지막 편집일시 : 2018 년 7 월 5 일 5:53
---

01. 들어가기 전에

Hello World의 유래

대부분의 프로그래밍 언어 책을 보면 첫번째 예제가 "HelloWorld"입니다. 왜! 그렇까요? 궁금

해서 한번 찾아 보았습니다.

1978년에 브라이언 커니핸과 데니스 리치가 쓴 "The C Programming Language" 교재의 첫 번째 예제가 화면에 "Hello, world!"를 출력하는 것이었고, 이것이 유명해지면서 모든 프로그래밍 언어의 첫번째 예제로 굳어지게 되었습니다.

참고로 프로그래밍 언어를 공부할 때 대부분 등장하기 때문에 The Hello World Collection 이라는 사이트도 있습니다.

그렇다면 Python의 "Hello World"는 어떨까요?

```
# Hello world in Python 2

print "Hello World"
```

```
# Hello world in Python 3

print("Hello World")
```

파이썬 2 또는 파이썬 3

https://wiki.python.org/moin/Python2orPython3 에 방문해보면 자세한 설명이 나와 있습니다. 간단하게 말하면 다음과 같습니다.

Python 2.x는 유산이며, Python 3.x는 현재와 미래의 언어입니다.

학업에 있고 파이썬을 처음 접한다면, 파이썬3로 공부하는 것이 편할 것입니다. 대부분의 파이썬 라이브러리들이 3를 완전히 지원하고 있고 많은 사람들이 파이썬2와 파이썬3가 호환될

수 있는 코드로 작성을 하기 때문입니다. 또한, 큰 프로젝트에서 프로그래밍을 하기에 보다 구조적이고 효율적입니다.

하지만, 좀 더 실무적으로 깊숙이 들어간다면 아직 파이썬2만 지원하는 패키지도 있고, 유지보수해야하는 구축된 파이썬 코드들 대부분이 파이썬2로 만들어졌기 때문에, 아직은 파이썬2를 사용하는 일이 대다수라고 할 수 있습니다.

그러므로 이러한 점을 참고하여 파이썬2나 3를 선택하여 사용하면 될 것입니다.

파이썬 버전 2와 3의 차이

파이썬 3에서는 모든 변수가 객체(object)로 처리됩니다.

이외에 여러가지 내부적 변화가 있지만, 다음 4가지 정도를 알아두면 됩니다.

1. int 나누기 결과가 float

```
# Python 2

print( 1/2 )

print( type(1/2) )

# 0

# <type 'int'>

# Python 3

print( 1/2 )

print( type(1/2) )

# 0.5

# <class 'float'>
```

2. print문 괄호 필수

```
# Python 2
print( 'hello' )
# hello
print 'hello'
# hello
# Python 3
print( 'hello' )
# hello
print 'hello'
# Error! invalid syntax
```

3. str과 unicode 통일

모든 문자열은 유니코드인 str

```
# Python 2
print( type('hello') )
print( type(u'hello') )
# <type 'str'>
# <type 'unicode'>
# Python 3
print( type('hello') )
```

```
print( type(u'hello') )

# <class 'str'>

# <class 'str'>
```

4. long → int로 통일

```
# Python 2

print( 2**30 )

print( type(2**30) )

print( 2**100 )

print( type(2**100) )

# 1073741824

# <type 'int'>

# 1267650600228229401496703205376

# <type 'long'>

# Python 3

print( 2**30 )

print( type(2**30) )

print( 2**100 )

print( type(2**100) )

# 1073741824

# <class 'int'>

# 1267650600228229401496703205376
```

<class 'int'>

마지막 편집일시 : 2018년 7월 5일 5:53

---

02. Python(파이썬) 표준 출력

1. print함수

Python에서는 표준 출력을 하는 함수로 print함수를 지원합니다.

일단 도움말을 통해서 print 함수의 사용법을 알아보도록 하겠습니다.

Ex00_print0.py

```
#  출력에 대하여 알아보자

# print함수의 도움말을 확인해 보자

help(print)
```

실행 결과

```
Help on built-in function print in module builtins:


print(...)

    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)


    Prints the values to a stream, or to sys.stdout by default.
```

Optional keyword arguments:

file: a file-like object (stream); defaults to the current sys.stdout.

sep: string inserted between values, default a space.

end: string appended after the last value, default a newline.

flush: whether to forcibly flush the stream.

위의 도움말을 보면 print함수의 사용법은 다음과 같습니다.

print(출력대상1, 출력 대상2 ....)

출력시 출력대상들의 사이에 구분자를 넣을때는 sep를 기술합니다.(기본값 : 공백)

마지막 문자열을 출력하고 출력할문자는 end를 기술합니다.(기본값 : 줄바꿈문자)

출력 방향을 변경 할 때는 file을 기술합니다.(기본값 : sys.stdout(표준출력장치,모니터))

스트림을 강제적으로 flush할지를 지정할때는 flush를 기술합니다.(기본값 : False)

Ex01_print1.py

```
#  출력에 대하여 알아보자

print("하나","둘","셋",1,2,3)

print("하나","둘","셋",1,2,3,sep='-')

print("첫번째 값")

print("두번째 값") # 다른 줄에 출력

print("첫번째 값", end=" ---> ")

print("두번째 값") # 같은 줄에 출력
```

```
# 출력 방향 변경

file = open("test.txt","w")

print("Hello Python!!", file=file) # 파일로 출력

file.close()
```

실행 결과

```
하나 둘 셋 1 2 3

하나-둘-셋-1-2-3

첫번째 값

두번째 값

첫번째 값 ---> 두번째 값
```

마지막에 출력한 "Hello Python!!"이란 문자열은 보이지 않습니다. 하지만 "test.txt" 파일이 생성되었으며 파일의 내용으로 "Hello Python!!"이란 문자열이 저장되었음을 확인됩니다.

2. 문자열 출력

작은따옴표이든 큰따옴표이든 문자열 출력시 같이 사용됩니다.

짝은 따옴표안에 큰따옴표가 가능합니다.

큰 따옴표안에 작은따옴표가 가능합니다.

큰따옴표안에서 큰따옴표를 출력하려면 \"로 해야 합니다.

작은따옴표안에서 작은따옴표를 출력하려면 \'로 해야 합니다.

동일한 문자열을 여러번 출력시 " * 횟수"를 사용합니다.

Ex02_print2.py

```
print('Hello World!!')

print("Hello World!!")

print("나의 이름은 '한사람'입니다.")

print('나의 이름은 "한사람"입니다.')

print("나의 이름은 \"한사람\"입니다.")

print('나의 이름은 \'한사람\'입니다.')

print('-' * 40)
```

실행 결과

```
Hello World!!

Hello World!!

나의 이름은 '한사람'입니다.

나의 이름은 "한사람"입니다.

나의 이름은 "한사람"입니다.

나의 이름은 '한사람'입니다.

----------------------------------------
```

3. 확장 문자 출력

기본 확장 문자(escape sequence)를 지원합니다.

\' : 따옴표 문자

\" : 쌍따옴표 문자

\ : backslash 문자

\a : bell 문자

\b : backslash 문자

\f : Formfeed 문자

\n : newline 문 \r : carriage return 문자(\n와 동일하지 않다.)

\t : tab 문자

\v : vertical tab 문자

숫자 형식 (8진법과 16진법) 확장 문자(escape sequence)들과 Unicode 확장 문자(escape sequence)들 8진수 또는 16진수 확장 문자(escape sequence)를 사용하여 ASCII 문자를 포함시킬 수 있습니다.

8진수 확장 문자(escape sequence)는 backslash()와 세 자릿수 8진수 숫자를 사용합니다.

결과로는 8진수 확장 문자(escape sequence)에 해당하는 ASCII 문자가 출력됩니다.

16진수 확장 문자(escape sequence)도 이와 비슷하지만 “\”가 아니라 “\x”로 시작되며 16진수 숫자로 구성되는 점이 다릅니다.

이 확장 문자(escape sequence)는 문자가 더 이상 16진수가 아닐 때 종료됩니다. 예를 들어 문자 n은 ASCII 문자표에서 십진수 값 109이다. 이는 8진수로 하면 156이며 16진수로 하면 6E 입니다.

Ex03_print3.py

```
#  출력에 대하여 알아보자

print('Hello \tWorld!!')

print("Hello \nWorld!!")
```

```
print("나의 이름은 \"한사람\"입니다.")

print('나의 이름은 \'한사람\'입니다.')

print('-' * 40)

print('1 \n')

print('2 \012')

print('3 \x0A')

print('-' * 40)

print('n')

print('\156')

print('\x6E')

print('-' * 40)
```

실행 결과

```
Hello     World!!

Hello

World!!

나의 이름은 "한사람"입니다.

나의 이름은 '한사람'입니다.

----------------------------------------

1
```

2

3

---

n

n

n

Python 3 내의 모든 문자열은 Unicode 문자열(string)이므로 모든 language에서 사용 가능한 대부분의 문자를 포함할 수 있습니다. 여기에서 설명하지는 않지만 다음 예제는 위에서 설명한 숫자 형식이나 Unicode 명으로 Unicode 문자를 escape할 수도 있다는 것을 보여줍니다.

Ex04_print4.py

```
#  출력에 대하여 알아보자


# Unicode 명으로 escape

unicode_a = '\N{LATIN SMALL LETTER A}'

print(unicode_a)

# Unicode 명으로 escape

unicode_a_with_acute = '\N{LATIN SMALL LETTER A WITH ACUTE}'
```

print(unicode_a_with_acute)

실행 결과

a

á

4. 형식에 맞추어 출력

데이터를 형식에 맞추어 출력할 수 있습니다.

이것은 print함수의 설명이라기보다 문자열의 형식을 지정하는 방법입니다. 형식화된 문자열로 만든 다음 문자열을 출력하는것 입니다. 문자열에 대해서는 자료형을 설명할 때 자세하게 설명하겠습니다.

이전 방식 : "출력형식"%(데이터....)

출력형식에 형식화된 문자열을 넣어 출력의 모양을 변경하는것입니다.

"%형식문자"로 지정하면 % 뒤의 출력 대상들이 1:1로 대응되어 출력됩니다.

%s : 문자열, %d : 정수, %f : 실수 등이 있습니다.

좀더 자세한 설명은 문자열 설명에서 하도록 하겠습니다.

예 1) %03d : 정수를 3칸에 맞추어 출력하는데 앞의 빈칸은 0으로 채워라 입니다.

예 2) %6.2f : 실수를 전체 6칸 소수이하 2칸에 맞추어 출력하라 입니다.

파이썬(Python) 3 포맷팅 방식 : "출력형식".format(데이터...)

출력형식에 형식화된 문자열을 넣어 출력의 모양을 변경하는것입니다.

"{}" 지정하면 format에 기술한 출력 대상들이 대응되어 출력됩니다.

"{n}"안에 숫자를 지정하여 출력 대상의 위치를 지정할 수 있습니다.

함수의 인자처럼 키워드를 사용해서 나타낼 수 있습니다.

동일한 데이터를 여러번 출력할 수 있습니다.

Ex05_print5.py

```
# 형식에 맞추어 출력하기


# 이전 방식
print('나의 이름은 %s입니다'%('한사람'))
print('나의 이름은 "%s"입니다. 나이는 %d세이고 성별은 %s입니다.'%('한사람',33,'남성'))
print('나이는 %d세이고 성별은 %s입니다. 나의 이름은 "%s"입니다. '%(33,'남성','한사람'))
print('나이는 %03d세이고 신장은 %6.2f입니다. 나의 이름은 "%s"입니다. '%(33,56.789,'한사람'))
print('-' * 40)


# 파이썬(Python) 3 포맷팅 방식
print('나의 이름은 {}입니다'.format('한사람'))
print('나의 이름은 "{0}"입니다. 나이는 {1}세이고 성별은 {2}입니다.'.format('한사람',33,'남성'))
print('나이는 {1}세이고 성별은 {2}입니다. 나의 이름은 "{0}"입니다. '.format('한사람',33,'남성'))
print('나이는 {age}세이고 성별은 {gender}입니다. 나의 이름은 "{name}"입니다. '
```

```
            .format(name='한사람',age=33,gender='남성'))
print('만세삼창 :  {0}!!! {0}!!! {0}!!! '.format('만세'))
print('삼삼칠 박수 :  {0}!!! {0}!!! {1}!!! '.format('짝'*3,'짝'*7))
print('-' * 40)
```

실행 결과

```
나의 이름은 한사람입니다
나의 이름은 "한사람"입니다. 나이는 33세이고 성별은 남성입니다.
나이는 33세이고 성별은 남성입니다. 나의 이름은 "한사람"입니다.
나이는 033세이고 신장은  56.79입니다. 나의 이름은 "한사람"입니다.
----------------------------------------
나의 이름은 한사람입니다
나의 이름은 "한사람"입니다. 나이는 33세이고 성별은 남성입니다.
나이는 33세이고 성별은 남성입니다. 나의 이름은 "한사람"입니다.
나이는 33세이고 성별은 남성입니다. 나의 이름은 "한사람"입니다.
만세삼창 :  만세!!! 만세!!! 만세!!!
삼삼칠 박수 :  짝짝짝!!! 짝짝짝!!! 짝짝짝짝짝짝짝!!!
----------------------------------------
```

5. 문자열 앞 0으로 채우기 zfill()

지정한 길이만큼 공간을 확보하고 남는 앞부분을 "0"으로 채워줍니다.

길이가 문자열보다 적을 경우에는 모든 문자열을 출력합니다.

Ex06_print6.py

```
# 형식에 맞추어 출력하기
#  앞의 빈칸 0으로 채우기
print('12'.zfill(5))
print('-12'.zfill(5))
print('3.141'.zfill(7))
print('-3.141'.zfill(7))
print('3.14159265359'.zfill(5))
print('-3.14159265359'.zfill(5))
print('문자열'.zfill(10))
print('문자열'.zfill(1))
print('-' * 40)
```

실행 결과

```
00012
-0012
003.141
-03.141
3.14159265359
```

-3.14159265359

0000000문자열

문자열

---

6. 길이와 정렬

{:길이} : 출력할 데이터의 길이를 지정합니다. 문자열(왼쪽 정렬), 숫자(오른쪽 정렬)

{:<길이} : 왼쪽 정렬

{:>길이} : 오른쪽 정렬

{:^길이} : 가운데 정렬

Ex07_print7.py

```
print('Python is [{:15}]'.format('good'))

print('Python is [{:<15}]'.format('good'))

print('Python is [{:>15}]'.format('good'))

print('Python is [{:^15}]'.format('good'))

print('당신의 나이는 [{:15}]세'.format(22))

print('당신의 나이는 [{:<15}]세'.format(22))

print('당신의 나이는 [{:>15}]세'.format(22))

print('당신의 나이는 [{:<15}]세'.format(22))

print('-' * 40)
```

실행 결과

```
Python is [good              ]

Python is [good              ]

Python is [              good]

Python is [       good       ]

당신의 나이는 [               22]세

당신의 나이는 [22               ]세

당신의 나이는 [               22]세

당신의 나이는 [22               ]세

----------------------------------------
```

Ex08_print8.py

```
# 왼쪽 정렬

for x in range(1, 4):

    print('[{0:2d}] [{1:3d}] [{2:4d}]'.format(x, x*x, x*x*x))

print()

# 가운데 정렬

for x in range(1, 4):

    print('[{0:^2d}] [{1:^3d}] [{2:^4d}]'.format(x, x*x, x*x*x))

print()

# 오른쪽 정렬
```

```
for x in range(1, 4):
    print('[{0:<2d}] [{1:<3d}] [{2:<4d}]'.format(x, x*x, x*x*x))
print()
# 0으로 채우기
for x in range(1, 4):
    print('[{0:02d}] [{1:03d}] [{2:04d}]'.format(x, x*x, x*x*x))
print()
# 부호 출력
print('[{0:05d}] [{1:05d}] [{2:05d}]'.format(1,-2,3))     # 음수만 부호
print('[{0:+05d}] [{1:+05d}] [{2:+05d}]'.format(1,-2,3)) # 부호
print('[{0:<5d}] [{1:<5d}] [{2:<5d}]'.format(1,-2,3))      # 정렬
print('-' * 40)
```

실행 결과

```
[ 1] [   1] [    1]
[ 2] [   4] [    8]
[ 3] [   9] [   27]

[1 ] [ 1 ] [ 1   ]
[2 ] [ 4 ] [ 8   ]
[3 ] [ 9 ] [ 27 ]
```

[1] [1   ] [1   ]

[2] [4   ] [8   ]

[3] [9   ] [27   ]

[01] [001] [0001]

[02] [004] [0008]

[03] [009] [0027]

[00001] [-0002] [00003]

[+0001] [-0002] [+0003]

[1   ] [-2   ] [3   ]

---

7. 채움문자와 숫자 표시형식

{[인덱스]:[채움문자][정렬][길이][,|_][형식문자]

공백을 채움문자로 채워줍니다.

, : 천단위 마다 콤마를 붙여줍니다.

_ : 천단위 마다 밑줄을 붙여줍니다.

e : 숫자를 지수 형식으로 만들어 줍니다.

= : 숫자 형식에만 사용합니다. 부호를 항상 제일 앞에 출력합니다.

Ex09_print9.py

```
# 채움문자
print('[{0:05d}] [{1:05d}] [{2:05d}]'.format(1,-2,3))
print('[{0:★<5d}] [{1:★<5d}] [{2:★<5d}]'.format(1,-2,3))
print('[{0:★>5d}] [{1:★>5d}] [{2:★>5d}]'.format(1,-2,3))
print('[{0:★^5d}] [{1:★^5d}] [{2:★^5d}]'.format(1,-2,3))
print('[{0:5d}] [{1:5d}] [{2:5d}]'.format(1,-2,3))
print('[{0:=5d}] [{1:=5d}] [{2:=5d}]'.format(1,-2,3))
print('[{0:=05d}] [{1:=05d}] [{2:=05d}]'.format(1,-2,3))
# 숫자 표시 형식( , _)
print('[{0:>5,}] [{1:>5,}] [{2:>5,}]'.format(11544435,-2544254,35454343))
print('[{0:>5_}] [{1:>5_}] [{2:>5_}]'.format(11544435,-2544254,35454343))
print('[{0:>5e}] [{1:>5e}] [{2:>5e}]'.format(11544435,-2544254,35454343))
print('-' * 40)
```

실행 결과

```
[00001] [-0002] [00003]
[1★★★★] [-2★★★] [3★★★★]
[★★★★1] [★★★-2] [★★★★3]
[★★1★★] [★-2★★] [★★3★★]
[    1] [   -2] [    3]
```

```
[      1] [-     2] [      3]

[00001] [-0002] [00003]

[11,544,435] [-2,544,254] [35,454,343]

[11_544_435] [-2_544_254] [35_454_343]

[1.154444e+07] [-2.544254e+06] [3.545434e+07]

----------------------------------------
```

8. 진법과 실수 출력 형식지정

b(2진수), o(8진수), x(16진수 소문자), X(16진수 대문자)

{[인덱스]:[전체자릿수][.소수이하자릿수]f

Ex10_print10.py

```
print('[{0:5b}] [{1:5b}] [{2:5b}]'.format(11,-22,33))      # 2진수

print('[{0:5o}] [{1:5o}] [{2:5o}]'.format(11,-22,33))      # 8진수

print('[{0:5x}] [{1:5x}] [{2:5x}]'.format(11,-22,33))      # 16진수 소문자

print('[{0:5X}] [{1:5X}] [{2:5X}]'.format(11,-22,33))      # 16진수 대문자

print('-' * 40)


import math

print('원주율 [{0:.3f}]'.format(math.pi))

print('원주율 [{0:.7f}]'.format(math.pi))

print('원주율 [{0:10.3f}]'.format(math.pi))
```

```
print('원주율 [{0:20.7f}]'.format(math.pi))

print('원주율 [{0:0.5f}]'.format(math.pi))

print('-' * 40)
```

실행 결과

```
[  1011] [-10110] [100001]

[     13] [   -26] [     41]

[      b] [   -16] [     21]

[      B] [   -16] [     21]

----------------------------------------

원주율 [3.142]

원주율 [3.1415927]

원주율 [       3.142]

원주율 [              3.1415927]

원주율 [3.14159]

----------------------------------------
```

9. {}와 % 출력

{}를 출력하려면 "{"과 "}"을 세번 연달아 입력해야 합니다.

% 자체를 출력하려면 %%로 입력해야 합니다.

Ex11_print11.py

형식에 맞추어 출력하기

print('{0}씨는 상위 {1}%안에 있는 사람입니다.'.format('한사람',10))

print('{{0}}씨는 상위 {{1}}%안에 있는 사람입니다.'.format('한사람',10))

print('{{{0}}}씨는 상위 {{{1}}}%안에 있는 사람입니다.'.format('한사람',10))

print('%s씨는 상위 %d%%안에 있는 사람입니다.'%('한사람',10))

print('-' * 40)

실행 결과

한사람씨는 상위 10%안에 있는 사람입니다.

{0}씨는 상위 {1}%안에 있는 사람입니다.

{한사람}씨는 상위 {10}%안에 있는 사람입니다.

한사람씨는 상위 10%안에 있는 사람입니다.

---

10. 출력형식 정리

표준 형식 지정자의 일반적인 형식 은 다음과 같습니다.

format_spec :: = [[fill] align] [sign] [#] [0] [width] [grouping_option] [. precision] [type]

fill :: = <모든 문자>

align :: = "<"| ">"| "="| "^"

sign :: = "+"| "-"| ""

width :: = digit+ grouping_option :: = "_"| ","

precision :: = digit+

type :: = "b"| "c"| "d"| "e"| "E"| "f"| "F"| "g"| "G"| "n"| "o"| "s"| "x"| "X"

fill(채움문자) : 어떠한 문자도 가능합니다.

align(정렬) : <(왼쪽정렬, 문자열 기본값), >(오른쪽 정렬, 숫자 기본값), ^(가운데정렬)

=(숫자에만 사용, '+000000120'형식으로 필드를 인쇄하는 데 사용)

sign(부호) : +(양수도 부호 표시), -(음수에 대해서만 부호를 사용,기본값)

width(폭) : 전체 폭을 양의 정수로 지정합니다.

, | _ : 천단위마다 콤마(,) 또는 밑줄(_)을 붙여줍니다.

precision(소수이하 자리수) : 소수이하 자리수를 정수로 지정합니다.

type(형식) : 문자열(s), 정수(b, c, d, o, x, X, n), 실수(e, E, f, F, g, G, n)

마지막 편집일시 : 2018년 7월 6일 9:57

---

03. Python(파이썬) 표준 입력

1. input 함수

Python에서는 표준 입력을 하는 함수로 input 함수를 지원합니다.

일단 도움말을 통해서 input 함수의 사용법을 알아보도록 하겠습니다.

Ex12_input0.py

표준 입력

input함수의 도움말을 확인해 보자

help(input)

실행 결과

Help on built-in function input in module builtins:

input(prompt=None, /)

Read a string from standard input. The trailing newline is stripped.

The prompt string, if given, is printed to standard output without a

trailing newline before reading input.

If the user hits EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return), raise EOFError.

On *nix systems, readline is used if available.

위의 도움말을 보면 input함수의 사용법은 다음과 같습니다.

input() : 표준 입력장치(키보드)로 부터 문자열을 입력 받습니다.

input('문자열') : 문자열을 출력하고 표준 입력장치(키보드)로 부터 문자열을 입력 받습니다.

입력된 값은 문자열 입니다.

사용자가 EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return)을 입력하면 EOFError를 발생시킵니다.

Ex13_input1.py

```
# 표준 입력

print('이름을 입력하세요', end="")

name = input();

print("이름 : {0}, type : {1}".format(name,type(name)))

name = input('이름을 입력하세요 ');

print("이름 : {0}, type : {1}".format(name,type(name)))

name = input('아무것도 입력하지 말고 EOF(Ctrl+D 또는 Ctrl+Z+Enter)를 입력해보세요');
```

실행 결과

```
이름을 입력하세요 한사람

이름 : 한사람, type : <class 'str'>

이름을 입력하세요 두사람

이름 : 두사람, type : <class 'str'>

아무것도 입력하지 말고 EOF(Ctrl+D 또는 Ctrl+Z+Enter)를 입력해보세요^D

Traceback (most recent call last):

  File "C:/PyThonProjects/Ex01/basic03/Ex13_input1.py", line 7, in <module>

    name = input('아무것도 입력하지 말고 EOF(Ctrl+D 또는 Ctrl+Z+Enter)를 입력해보세요');

EOFError: EOF when reading a line

"이름을 입력하세요"라고 나오면 "한사람"을 입력하세요

"이름을 입력하세요"라고 나오면 "두사람"을 입력하세요
```

"아무것도 입력하지 말고 EOF(Ctrl+D 또는 Ctrl+Z+Enter)를 입력해보세요"라고 나오면 Ctrl 키와 D키를 같이 눌러 보세요

2. 정수 입력

기본적으로 input함수는 문자열로 입력됩니다.

그래서 입력받은 값을 정수형으로 변환해서 사용해야 합니다.

eval() 함수 : 인수를 유효한 파이썬 표현식으로 리턴 합니다.

int() 클래스 : class int(x=0), class int(x,base=10)

숫자나 문자열 x 로 부터 만들어진 정수 객체를 돌려줍니다.

인자가 주어지지 않으면 0 을 돌려줍니다.

base는 진법을 나타내며 주로 2, 8, 10, 16을 사용합니다. 10이 기본값입니다.

Ex14_input2.py

```
# 표준 입력

data = input("정수를 입력하시오 : ")

print(data, type(data))

# print(data, type(data), data + 1) 에러 문자열과 정수를 +(더하기)할 수 없습니다.


data = eval(input("정수를 입력하시오 : "))

print(data, type(data), data + 1)
```

```
data = int(input("정수를 입력하시오 : "))
print(data, type(data), data + 1)

data = int(input("2진수를 입력하시오 : "), 2)
print(data, type(data), data + 1)

data = int(input("8진수를 입력하시오 : "), 8)
print(data, type(data), data + 1)

data = int(input("10진수를 입력하시오 : "), 10)
print(data, type(data), data + 1)

data = int(input("16진수를 입력하시오 : "), 16)
print(data, type(data), data + 1)
```

실행 결과

```
정수를 입력하시오 : 1
1 <class 'str'>
정수를 입력하시오 : 2
2 <class 'int'> 3
정수를 입력하시오 : 3
```

3 <class 'int'> 4

2진수를 입력하시오 : 1010

10 <class 'int'> 11

8진수를 입력하시오 : 10

8 <class 'int'> 9

10진수를 입력하시오 : 10

10 <class 'int'> 11

16진수를 입력하시오 : 1a

26 <class 'int'> 27

실행할 때 차례대로 1, 2, 3, 1010, 10, 10, 1a를 입력해보세요

3. 실수 입력

기본적으로 input함수는 문자열로 입력됩니다.

그래서 입력받은 값을 실수형으로 변환해서 사용해야 합니다.

eval() 함수 : 인수를 유효한 파이썬 표현식으로 리턴 합니다.

float() 클래스 : class float(x)

숫자나 문자열 x 로 부터 만들어진 실수 객체를 돌려줍니다.

인자가 주어지지 않으면 0 을 돌려줍니다.

Ex15_input3.py

```
# 표준 입력

data = input("실수를 입력하시오 : ")

print(data, type(data))

# 에러 문자열과 실수를 +(더하기)할 수 없습니다.

# print(data, type(data), data + 1.2)


data = eval(input("실수를 입력하시오 : "))

print(data, type(data), data + 1.2)


data = float(input("정수를 입력하시오 : "))

print(data, type(data), data + 1.2)
```

실행 결과

```
실수를 입력하시오 : 3.14

3.14 <class 'str'>

실수를 입력하시오 : 3.14

3.14 <class 'float'> 4.34

정수를 입력하시오 : 3.14

3.14 <class 'float'> 4.34
```

4. 튜플(tuple), 리스트(list)로 입력받기

기본적으로 input함수는 문자열로 입력됩니다.

eval() 함수 : 인수를 유효한 파이썬 표현식으로 리턴 합니다.

Ex16_input4.py

```
# 표준 입력
string = input("(1,2) 처럼입력하시오 ")
print(string, type(string))

string = eval( input("(1,2) 처럼입력하시오 "))
print(string, type(string))

string = input("[1,2,3,4,5,6] 처럼입력하시오 ")
print(string, type(string))

string = eval( input("[1,2,3,4,5,6] 처럼입력하시오 "))
print(string, type(string))
```

실행 결과

```
(1,2) 처럼입력하시오 (1,2)
(1,2) <class 'str'>
(1,2) 처럼입력하시오 (1,2)
```

(1, 2) <class 'tuple'>

[1,2,3,4,5,6] 처럼입력하시오 [1,2,3]

[1,2,3] <class 'str'>

[1,2,3,4,5,6] 처럼입력하시오 [1,2,3]

[1, 2, 3] <class 'list'>

마지막 편집일시 : 2018년 7월 5일 5:53

---

04. 모듈의 사용(import)

1. 모듈이란?

파이썬(Python) 모듈(Module)이란? 자주 사용되는 코드나 유용한 코드를 논리적으로 묶어서 관리하고 사용할 수 있도록 하는 것입니다. 보통 하나의 파이썬 .py 파일이 하나의 모듈이 됩니다. 모듈 안에는 함수, 클래스, 혹은 변수들이 정의될 수 있으며, 실행 코드를 포함할 수도 있습니다.

여기서는 모듈의 사용법에 대하여만 알아보고 뒤에 모듈의 작성법부터 차례대로 알아보도록 하겠습니다.

2. 내장 모듈

파이썬은 기본적으로 상당히 많은 표준 라이브러리 모듈들을 제공하고 있으며, 3rd Party에서도 많은 파이썬 모듈들을 제공하고 있습니다. IDLE에서 다음과 같이 help("modules") 을 입력하면 설치된 모듈의 목록을 확인 할 수 있습니다.

>>> help("modules") [Enter]

Please wait a moment while I gather a list of all available modules...

```
__future__          autocomplete        idle                sched
__main__            autocomplete_w      idle_test           scrolledlist
_ast                autoexpand          idlelib             search
_asyncio            base64              idna                searchbase
_bisect             bdb                 imaplib             searchengine
```

중간 생략.....

```
asyncio             hmac                rstrip              zoomheight
asyncore            html                run                 zzdummy
atexit              http                runpy
audioop             hyperparser         runscript

Enter any module name to get more help.  Or, type "modules spam" to search
for modules whose name or summary contain the string "spam".

>>>
```

3. 모듈의 사용(import)

이러한 모듈들을 사용하기 위해서는 모듈을 import하여 사용하면 되는데, import 문은 다음과 같이 하나 혹은 복수의 모듈을 불러들일 수 있습니다.

import 모듈

import 모듈1, 모듈2, 모듈3 ...

import 모듈명 as 별명

모듈에 있는 모든 내용을 포함시킵니다.

'모듈명.함수명'처럼 모듈명을 반드시 입력해야 합니다.

as를 사용하여 모듈명에 별명을 붙일 수 있습니다.

from 모듈 import 함수

from 모듈 import 함수1, 함수2, 함수3 ...

from 모듈 import *

from 모듈 import 함수 as 별명

form import 방식을 사용하면 모듈이름을 생략할 수 있습니다.

*를 사용하면 모듈에 있는 모든 내용이 포함됩니다.

as를 사용하여 함수명에 별명을 붙일 수 있습니다.

Ex18_module2.py

```
# 모듈(Module) 사용하기

import datetime
```

```
import math

import math as m

from math import pi

from math import *


# import datetime

now = datetime.datetime.now()

print(now)


# import math

print("The value of pi is", math.pi)


# import math as m

print("The value of pi is", m.pi)


# from math import pi

print("The value of pi is", pi)


# from math import *

print("The value of e is", e)
```

실행 결과

2018-07-06 17:27:36.859288

The value of pi is 3.141592653589793

The value of pi is 3.141592653589793

The value of pi is 3.141592653589793

The value of e is 2.718281828459045

마지막 편집일시 : 2018년 7월 6일 5:28

---

02. 키워드, 식별자, 변수, 상수, 리터럴

키워드(Keyword)는 파이썬에서 이미 예약되어있는 문자열로서 다른 용도로 사용이 불가능한 문자열 입니다.
식별자(identifier)는 프로그래밍 언어의 스펙(specification)에서나 사용하는 용어로 변수, 상수, 함수, 사용자 정의 타입 등에서 다른 것들과 구분하기 위해서 사용되는 변수의 이름, 상수의 이름, 함수의 이름, 사용자 정의 타입의 이름 등 '이름'을 일반화 해서 지칭하는 용어입니다.
변수(Variable)는 문자나 숫자 같은 값을 담는 컨테이너입니다. 여기에 담겨진 값은 다른 값으로 바꿀 수 있습니다.
상수(constant)는 항상 똑같은 값을 저장하고 있는 곳이라 할 수 있습니다. 프로그래머나 시스템에 의해 미리 정해져있는 것으로, 복잡한 숫자의 값을 인지하기 쉬운 문자로 변경하여 사용하고자 할 때 주로 사용합니다.
리터럴(literal)은 "값" 그 자체로, 고정된 값을 표현하는 것을 의미합니다.
23, 200u, 0x34, 3.23, 309E, "hello", "안녕~" 은 리터럴입니다.
"a = 7"에서 a는 변수이고, 7은 리터럴입니다. "MAX = 8"에서 MAX는 상수이고, 8은 리터럴입니다.

마지막 편집일시 : 2018년 7월 6일 11:07

01. 키워드(Keyword)

키워드(Keyword)는 파이썬에서 이미 예약되어있는 문자열로서 다른 용도로 사용이 불가능한 문자열 입니다. 다음의 코드를 키워드를 확인할 수 있습니다.

Ex01_keyowrd1.py

```
import keyword
# print(keyword.kwlist)
kwlist = keyword.kwlist
for i in range(0,len(kwlist)):
    print("[{:10}]".format(kwlist[i]), end=" ")
    if (i+1)%5==0: print()
```

실행 결과

```
[False     ] [None    ] [True    ] [and     ] [as      ]
[assert    ] [break   ] [class   ] [continue] [def     ]
[del       ] [elif    ] [else    ] [except  ] [finally ]
[for       ] [from    ] [global  ] [if      ] [import  ]
[in        ] [is      ] [lambda  ] [nonlocal] [not     ]
[or        ] [pass    ] [raise   ] [return  ] [try     ]
[while     ] [with    ] [yield   ]
```

앞으로 배워나가면서 알게되겠지만 몇 가지만 먼저 알아보도록 하겠습니다.

1. True/False

True/False는 파이썬에서 진리 값입니다. 그것들은 파이썬에서의 비교 연산 또는 논리적 (Boolean) 연산의 결과입니다.

Ex02_keyword2.py

```
# 진리값

print("참 : ", True)

print("거짓 : ", False)

print(1==1)

print(3>5)

print(3<5)

print(True or False)

print(True and False)

print(not False)

print(True==1)

print(False==0)

print(True + True)
```

실행 결과

```
참 :  True

거짓 :  False

True
```

False

True

True

False

True

True

True

2

2. None

값이 없음을 나타내는 Python의 특수 상수입니다.

Ex03_keyword3.py

```
# 진리값
print("None : ", None)
print(None == 0)
print(None == [])
print(None == False)
print(None == "")
x = None
y = None
```

```
print(x == y)


# 아무값도 리턴하지 않는 함수
def void_function():
    a = 10
    b = 20
    c = a + b


result = void_function()
print(result)
print("값이없음" if result==None else result)
```

실행 결과

```
None :  None
False
False
False
False
True
None
값이없음
```

※ 3항 연산자 : 연산 대상이 3개인 연산자

참일때 값 if 조건 else 거짓일때값

3. assert

assert 디버깅 목적으로 사용됩니다.

디버그(debug), 디버깅(debugging)은 컴퓨터 프로그램의 정확성이나 논리적인 오류(버그)를 찾아내는 테스트 과정을 뜻합니다. 일반적으로 디버깅을 하는 방법으로 테스트 상의 체크, 기계를 사용하는 테스트, 실제 데이터를 사용해 테스트하는 방법이 있습니다.

사용형식 : assert condition, message

프로그래밍하는 동안 때때로 우리는 프로그램의 내부 상태를 알고 싶거나 우리의 가정이 사실인지 확인하기를 원합니다. assert이 작업을 수행하고 버그를보다 편리하게 찾을 수 있습니다. assert뒤에 조건이옵니다.

조건이 true이면 아무 일도 발생하지 않습니다. 그러나 조건이 거짓이면 AssertionError가 발생합니다.

Ex04_keyword4.py

```
# assert 디버깅 목적으로 사용됩니다.

a = 10

assert a > 5

# assert a < 5
```

```
assert a < 5, "a의 값이 너무 큽니다."
```

실행 결과

```
# 메시지를 지정하지 않은 경우
Traceback (most recent call last):
  File "C:/PyThonProjects/Ex01/basic04/Ex04_keyword4.py", line 4, in <module>
    assert a < 5
AssertionError

# 메시지를 지정한 경우
Traceback (most recent call last):
  File "C:/PyThonProjects/Ex01/basic04/Ex04_keyword4.py", line 5, in <module>
    assert a < 5, "a의 값이 너무 큽니다."
AssertionError: a의 값이 너무 큽니다.
```

"assert a > 5"는 아무일도 일어나지 않습니다.

"assert a < 5"에서 문제가 발생합니다.

뒤에 메시지를 쓰면 지정한 메세지로 AssertionError를 발생합니다.

다음의 식과 같은 역할을 합니다. 예외 처리하는 방법은 뒤에 자세하게 배우게 됩니다.

```
if not condition:
    raise AssertionError(message)
```

마지막 편집일시 : 2018년 7월 6일 12:24

---

02. 식별자(identifier)

1. 식별자(identifier)란?

변수, 상수, 함수, 사용자 정의 타입 등에서 다른 것들과 구분하기 위해서 사용되는 변수의 이름, 상수의 이름, 함수의 이름, 사용자 정의 타입의 이름 등 '이름'을 일반화 해서 지칭하는 용어입니다.

사용자가 임의로 지정합니다. 이름만 보아도 뜻을 알 수 있도록 지정하는 것이 좋습니다.

2. 식별자(identifier) 명칭의 작성 스타일

변수나 함수명 같은 식별자(identifier)의 명칭을 작성할 때 이름을 정의하는 것도 코딩 스타일 중 일부입니다. 대표적인 표기법은 다음 네 가지가 있습니다.

카멜 표기법(Camel Case)

여러 단어를 연달아 사용할 때 각 단어의 첫 글자를 대문자로 적되, 맨 앞에 오는 글자는 소문자로 표기하는 것입니다. 낙타의 등에 있는 혹과 같다고 하여 카멜(Camel) 표기법이라고 부릅니다. Java의 권장 표기법입니다.

예) camelVariable, firstName, lastName ...

파스칼 표기법(Pascal Case)

단어의 모든 앞글자를 대문자로 표기합니다. 카멜 표기법이 단봉낙타라면 파스칼은 쌍봉낙타라고 할 수 있습니다. 예) PascalVariable, FirstName, LastName ...

헝가리안 표기법(Hungarian Notation)

접두어에 자료형을 붙이는 표기법입니다. 요즈음은 잘 사용하지 않는 스타일입니다. 언어의 종류가 다양한 만큼 자료형도, 문서 데이터도 다양해졌기 때문에 접두어가 의미가 없어졌기 때문 입니다. 예) strName, bBusy, szName ...

스네이크 표기법(Snake Case)

단어 사이에 언더바를 넣어서 표기하는 방법입니다. 하지만 "_" 이나 "__"로 시작되는 식별자들은 많은 언어에서 확장으로 예약되어 있는 경우가 많으므로 사용하지 않는 것이 관례입니다.

대부분의 경우 카멜 기법과 파스칼 표기법을 적절하게 조합하여, 변수명이나 함수명은 카멜 표기법을 따르고 클래스명은 파스칼 표기법을 따르는 작성 스타일, 혹은 변수명은 카멜로 표기하고 함수와 클래스명은 파스칼로 표기하는 작성 스타일이 대세입니다. 전자는 Java, 후자는 C++에서 주로 볼 수 있는 스타일이다. 언어에 따라 전부 카멜, 전부 파스칼로 표기할 것을 권장하기도 합니다.

3. Style Guide for Python Code

Python에서는 https://www.python.org/dev/peps/pep-0008/ 에 "Style Guide for Python Code"를 기술하고 있습니다. 위의 내용을 간단하게 설명하면 다음과 같습니다.

코드 배치

들여쓰기는 공백 4칸을 권장합니다.

한 줄은 최대 79자까지

최상위(top-level) 함수와 클래스 정의는 2줄씩 띄어 씁니다.

클래스 내의 메소드 정의는 1줄씩 띄어 씁니다.

표현식과 구문의 공백

다음과 같은 곳의 불필요한 공백은 피합니다.

대괄호([])와 소괄호(())안의 공백은 피합니다.

쉼표(,), 쌍점(:)과 쌍반점(;) 앞의 공백은 피합니다.

키워드 인자(keyword argument)와 인자의 기본값(default parameter value)의 = 는 붙여 씁니다.

주석

코드와 모순되는 주석은 없느니만 못합니다. 항상 코드에 따라 갱신해야 합니다.

불필요한 주석은 달지 마세요.

한 줄 주석은 신중히 다세요.

문서화 문자열(Docstring)에 대한 컨벤션은 PEP 257을 참고하세요.

이름 지정 규칙

변수명에서 _(밑줄)은 위치에 따라 다음과 같은 의미가 있습니다.

1개의 밑줄로 시작 : 내부적으로 사용되는 변수

1개의 밑줄로 종료 : 파이썬 기본 키워드와 충돌을 피하려고 사용

2개의 밑줄로 시작 : 클래스 속성으로 사용

2개의 밑줄로 종료 : 사용자가 조정할 수 있는 네임스페이스 안의 속성

소문자 L, 대문자 O, 대문자 I는 변수명으로 사용하지 마세요

모듈(Module) 명은 짧은 소문자로 구성되며 필요하다면 밑줄로 나눕니다.

모듈은 파이썬 파일(.py)에 대응하기 때문에 파일 시스템의 영향을 받으니 주의하세요.

C/C++ 확장 모듈은 밑줄로 시작합니다.

클래스명은 카멜케이스(CamelCase)로 작성합니다.

내부적으로 쓰이면 밑줄을 앞에 붙입니다.

예외(Exception)는 실제로 에러인 경우엔 “Error”를 뒤에 붙입니다.

함수명은 소문자로 구성하되 필요하면 밑줄로 나눕니다.

대소문자 혼용은 이미 흔하게 사용되는 부분에 대해서만 하위호환을 위해 허용합니다.

메소드명은 함수명과 같으나 비공개(non-public) 메소드, 혹은 변수면 밑줄을 앞에 붙입니다.

서브 클래스(sub-class)의 이름 충돌을 막기 위해서는 밑줄 2개를 앞에 붙입니다.

상수(Constant)는 모듈 단위에서만 정의하며 모두 대문자에 필요하다면 밑줄로 나눕니다.

프로그래밍 권장 사항

코드는 될 수 있으면 PyPy, Jython, IronPython등에서도 불이익이 없게끔 작성되어야 합니다.

None을 비교할때는 is나 is not만 사용합니다.

클래스 기반의 예외를 사용하세요.

모듈이나 패키지에 자기 도메인에 특화된(domain-specific)한 기반 예외 클래스(base exception class)를 빌트인(built-in)된 예외를 서브클래싱해 정의하는게 좋습니다. 이 때 클래스는 항상 문서화 문자열을 포함해야 합니다.

raise ValueError('message')가 (예전에 쓰이던) raise ValueError, 'message' 보다 좋습니다.

예외를 except:로 잡기보단 명확히 예외를 명시합니다.(ex. except ImportError:

try: 블록의 코드는 필요한 것만 최소한으로 작성합니다.

string 모듈보다는 string 메소드를 사용합니다. 메소드는 모듈보다 더 빠르고, 유니코드 문자열에 대해 같은 API를 공유합니다.

접두사나 접미사를 검사할 때는 startswith()와 endwith()를 사용합니다.

객체의 타입을 비교할 때는 isinstance()를 사용합니다.

빈 시퀀스(문자열, 리스트(list), 튜플(tuple))는 조건문에서 거짓(false)입니다.

불린형(boolean)의 값을 조건문에서 ==를 통해 비교하지 마세요.

4. 식별자 작성 규칙

식별자는 소문자 (a ~ z) 또는 대문자 (A ~ Z) 또는 숫자 (0 ~ 9) 또는 밑줄 (_)의 조합입니다.

myClass, var_1및 print_this_to_screen 모두 유효한 예입니다.

식별자는 숫자로 시작될 수 없습니다. 1variable무효이지만 variable1완벽하게 괜찮습니다.

키워드는 식별자로 사용할 수 없습니다.

!, @, #, $, % 등 특수 기호는 우리 식별자에서 사용할 수 없습니다.

식별자의 길이는 제한이 없습니다.

파이썬은 대소 문자를 구별하는 언어입니다. 이는, Variable과 variable동일하지 않습니다.

의미있는 식별자의 이름을 항상 지정하십시오.

마지막 편집일시 : 2018년 7월 6일 2:16

---

03. 변수, 상수 그리고 리터럴

1. 변수(Variable)

컴퓨터가 어떠한 일을 하기위해서는 모든 데이터는 보조기억장치(SSD,HDD,USB 등)에서 주기억 장치(RAM)로 읽혀져야만 중앙처리장치(CPU)가 주기억 장치의 내용을 읽어서 데이터를 가공처리합니다.

그런데 이 주기억 장치는 Byte마다 번지가 지정되어 있습니다. 번지는 숫자이므로 프로그램을 작성하려면 "10번지의 값을 읽어서 20번지의 값과 더하여 30번지의 값에 저장하여라" 등의 명령어를 내리게 됩니다. 프로그래머가 모든 번지를 제어하려면 힘들것입니다. 그래서 번지에 별명을 붙여 "이곳은 name이라고 하고 저곳은 score라고 하자"는 것이 변수입니다.

메모리 관리는 운영체제가 맡아서 합니다. 프로그래머가 메모리가 필요하면 변수를 선언하고 값을 할당하면 됩니다. 그러면 운영체제가 알아서 현재 사용하지 않는 메모리를 확보해서 그곳에 변수명을 할당합니다. 그 후로 프로그래머는 그 변수명을 이용하여 메모리에 접근하여 프로그램을 작성하면 됩니다.

메모리 관리원칙

메모리 관리의 주체는 운영체제입니다.

운영체제는 메모리가 있는 한은 할당 요청을 거절하지 않습니다.

한 번 할당된 메모리 공간은 절대로 다른 목적을 위해 재할당되지 않습니다.

응용 프로그램이 할당된 메모리를 해제하면 운영체제는 이 공간을 빈 영역으로 인식하고 다른 목적을 위해 사용할 수 있도록 합니다.

결론적으로 변수(Variable)는 문자나 숫자 같은 값을 담는 컨테이너입니다. 컴퓨터로 어떤한 일을 한다는 것은 필요한 데이터를 담아서 가공 처리하여 사용자가 원하는 결과를 만들어내는 것 입니다. 가공처리하는 과정에 발생하는 데이터들을 어딘가에 보관해서 계속 처리를 해야 된다면 어디에 담아놓을 까요? 그래서 저장해놓는 기억 공간이 필요하게 되는데 이때 변수를 사용하는거서 입니다. 변수는 프로그램 실행 중에 변경되는 값을 일시적으로 저장하는 임시 기억장소의 별명이라고 할 수 있습니다. 변수는 항상 맨 마지막에 저장한 값만을 저장합니다. 다시말하면 변수는 값을 저장하는 그릇과 같습니다.

파이썬에서 변수 선언하기

파이썬에서 변수는 메모리 공간을 예약하기 위해 선언 할 필요가 없습니다. "변수 선언"또는 "변수 초기화"는 변수에 값을 할당 할 때 자동으로 발생합니다.

파이썬에서 변수에 값 할당하기

대입 연산자 =를 사용하여 변수에 값을 할당 할 수 있습니다 .

=(대입연산자)는 우변의 값을 좌변에 넣으라는 뜻 입니다.

Ex05_variable1.py

```
website = "some.com"

print(website)

website = "another.com"

print(website)
```

실행 결과

```
some.com

another.com
```

파이썬에서 여러개 변수에 여러개 값 할당하기

Ex06_variable2.py

```
a=b=c="모두같은 값"

print("a의 값 : {}".format(a))

print("b의 값 : {}".format(b))

print("c의 값 : {}".format(c))


d,e,f = "한사람",23,178.9

print("d의 값 : {}".format(d))

print("e의 값 : {}".format(e))

print("f의 값 : {}".format(f))
```

```
# 일반 언어의 경우 두 변수값 교환

g,h = 100,200

print("g의 값 : {}, h의 값 :{}".format(g,h))

t = g;

g = h;

h = t;

print("g의 값 : {}, h의 값 :{}".format(g,h))


# 파이선의 경우 두 변수값 교환

g,h = 100,200

print("g의 값 : {}, h의 값 :{}".format(g,h))

g,h = h,g

print("g의 값 : {}, h의 값 :{}".format(g,h))
```

실행 결과

```
a의 값 : 모두같은 값

b의 값 : 모두같은 값

c의 값 : 모두같은 값

d의 값 : 한사람

e의 값 : 23

f의 값 : 178.9
```

g의 값 : 100, h의 값 :200

g의 값 : 200, h의 값 :100

g의 값 : 100, h의 값 :200

g의 값 : 200, h의 값 :100

2. 상수(constant)

상수(constant)는 항상 똑같은 값을 저장하고 있는 곳이라 할 수 있습니다. 프로그래머나 시스템에 의해 미리 정해져있는 것으로, 복잡한 숫자의 값을 인지하기 쉬운 문자로 변경하여 사용하고자 할 때 주로 사용합니다.

constant1.py

```
PI = 3.14

GRAVITY = 9.8
```

Ex06_constant1.py

```
import constant1


print(constant1.PI)

print(constant1.GRAVITY)

constant1.PI = 3.141592 # 변경된다. 상수가 아니다.

print(constant1.PI)
```

실행 결과

3.14

9.8

3.141592

위의 프로그램에서는 constant1.py 모듈 파일을 만듭니다. 그런 다음 PI 및 GRAVITY에 상수 값을 할당합니다. 그런 다음 Ex06_constant1.py 파일을 만들고 상수 모듈을 가져옵니다. 마지막으로 상수 값을 출력합니다.

실제로는 파이썬에서 상수를 사용하지 않습니다. 하지만 다음과 같이하여 클래스를 만들어 상수를 만들 수는 있습니다. 클래스는 다음에 배우게 됩니다. 상수가 무엇인지만 알아 두시기 바랍니다.

constant2.py

```
class _constant2:

    def __setattr__(self, name, value):

        if name in self.__dict__:

            raise Exception('변수에 값을 할당할 수 없습니다.')

        self.__dict__[name] = value


    def __delattr__(self, name):

        if name in self.__dict__:

            raise Exception('변수를 삭제할 수 없습니다.')
```

```
import sys

sys.modules[__name__] = _constant2()
```

Ex08_constant2.py

```
import constant2


constant2.PI = 3.14

constant2.MAX = 1000

print(constant2.PI)

print(constant2.MAX)


constant2.PI = 3.141592

constant2.MAX = 2000

print(constant2.PI)

print(constant2.MAX)
```

실행 결과

```
3.14

Traceback (most recent call last):

1000
```

```
  File "C:/PyThonProjects/Ex01/basic04/Ex08_constant2.py", line 8, in <module>
    constant2.PI = 3.141592
  File "C:\PyThonProjects\Ex01\basic04\constant2.py", line 4, in __setattr__
    raise Exception('변수에 값을 할당할 수 없습니다.')
Exception: 변수에 값을 할당할 수 없습니다.
```

3. 리터럴(liternal)

리터럴(liternal)은 "값" 그 자체로, 고정된 값을 표현하는 것을 의미합니다.

파이썬에는 다음과 같은 다양한 유형의 리터럴이 있습니다.

숫자 리터럴

정수 리터럴, 실수 리터럴, 복소수 리터럴 3가지가 있습니다.

정수 리터럴 : 0b로 시작하면 2진수, 0o로 시작하면 8진수 ,0~9로 시작하면 10진수, 0x로 시작하면 16진수

실수 리터럴 : 소숫점을 포함하거나 e를 포함합니다.

복소수 리터럴 : j로 끝나면 복소수의 허수를 나타냅니다.

Ex09_literals1.py

```
# Integer Literals
a = 0b1010 # Binary Literals
b = 100 # Decimal Literal
c = 0o310 # Octal Literal
d = 0x12c # Hexadecimal Literal
```

```
# Float Literal

float_1 = 10.5

float_2 = 1.512e2


# Complex Literal

x = 1.1 + 3.14j


print(a, b, c, d)

print(float_1, float_2)

print(x, x.real, x.imag)
```

실행 결과

```
10 100 200 300

10.5 151.2

(1.1+3.14j) 1.1 3.14
```

문자 리터럴

문자열 리터럴은 따옴표로 묶인 일련의 문자입니다. 문자열에 대해 단일, 이중 또는 삼중 따옴표를 모두 사용할 수 있습니다. 그리고 문자 리터럴은 작은 따옴표 또는 큰 따옴표로 묶인 단일 문자입니다.

Ex10_literals2.py

```
# String Literals

strings = "This is Python"

char = "C"

multiline_str = """This is a multiline string with more than one line code."""

unicode = u"\u00dcnic\u00f6de"

raw_str = r"raw \n string"


print(strings)

print(char)

print(multiline_str)

print(unicode)

print(raw_str)
```

실행 결과

```
This is Python

C

This is a multiline string with more than one line code.

Ünicöde

raw \n string
```

※ "r"문자는 raw string으로 백슬래시 문자를 해석하지 않고 남겨두기 때문에 정규표현식과 같은 곳에 유용합니다.

논리값 리터럴

논리값 리터럴은 다음 두 값 중 하나를 가질 수 있습니다. True 또는 False.

Ex11_literals3.py

```
# Boolean Literals
x = (1 == True)
y = (1 == False)
a = True + 4
b = False + 10

print("x is", x)
print("y is", y)
print("a:", a)
print("b:", b)
```

실행 결과

```
x is True
y is False
a: 5
b: 10
```

특수 리터럴

파이썬에는 특별한 리터럴이 하나가 들어 있습니다.

None. 우리는이 필드를 사용하여 생성되지 않은 필드를 지정합니다.

Ex12_literals4.py

```
# Special Literals
drink = "Available"
food = None



def menu(x):
    if x == drink:
        print(drink)
    else:
        print(food)



menu(drink)
menu(food)
```

실행 결과

Available

None

컬렉션 리터럴

자료형은 다음장에서 자세하게 배우게 됩니다.

[...]로 감싸져 있으면 list자료형

(...)로 감싸져 있으면 tuple자료형

{ 키:값, ... }로 감싸져 있으면 dictionary자료형

{ ... }로 감싸져 있으면 set자료형

Ex13_literals5.py

```
# Literal Collections
fruits = ["apple", "mango", "orange"]  # list
numbers = (1, 2, 3)  # tuple
alphabets = {'a': 'apple', 'b': 'ball', 'c': 'cat'}  # dictionary
vowels = {'a', 'e', 'i', 'o', 'u'}  # set

print(fruits)
print(numbers)
print(alphabets)
print(vowels)
```

실행 결과

['apple', 'mango', 'orange']

(1, 2, 3)

{'a': 'apple', 'b': 'ball', 'c': 'cat'}

{'a', 'e', 'u', 'o', 'i'}

마지막 편집일시 : 2018년 7월 6일 4:47

---

03. Python(파이썬) 자료형 기본

- 파이썬의 모든 값에는 데이터 유형이 있습니다.
- 파이썬 프로그래밍에서는 모든 것이 객체이기 때문에 실제로 데이터 유형은 클래스이고 변수는 이러한 클래스의 인스턴스 (객체)입니다.
- 여기서는 간단하게 알아보고 자세한것은 다음 계단에서 알아보도록 하겠습니다.

목차

- Python Numbers
- Python Strings
- Python Tuple
- Python Set
- Python List
- Python Dictionary

마지막 편집일시 : 2018년 7월 12일 1:46

01. Python Numbers

숫자로 이루어진 자료형을 말합니다.
정수형, 실수형, 복소수형이 있습니다.

1. 정수형

- 양의 정수, 0, 음의 정수를 말합니다. 소수이하 값이 없는 자료형입니다.
- 파이썬에서 정수의 값은 비트 수에 의해 제한되지 않고 사용 가능한 메모리의 한계까지 확장 될 수 있습니다 . 따라서 큰 수를 저장하기위한 특별한 배열을 필요로하지 않습니다.

Ex01_number1.py

```
# 정수형 :

실행 결과
min = -max-1
print(max, type(max))
print(min, type(min))

print(max+1, type(max+1))
print(10**40, type(10**40))
# print(10**100, type(10**100))
x = 1000000000000000000000000000000000000000000000;
x = x + 1
print(x)
127 <
0 <
-
-128 <
2 <
8 <
10 <
16 <
9223372036854775807 <
-
-9223372036854775808 <
9223372036854775808 <
1000000000000000000000000000000000000000000 <
```

2. 실수형

- 실수형(Floating)은 소수점이 포함된 숫자를 말합니다.
- 부동 소수점 숫자는 소수점 이하 15 자리까지 정확합니다.
- 정수 및 부동 소수점은 소수점으로 구분됩니다. 1정수, 1.0부동 소수점 숫자입니다.

Ex02_number2.py

실수형 :

실행 결과

```
float_number4 = -123.456e-3
print(float_number3, type(float_number3))
print(float_number4, type(float_number4))

float_number5 = 10/3
float_number6 = 1.12345678901234567890123456789 0
print(float_number5, type(float_number5))
print(float_number6, type(float_number6))
```

```
123.456 <
-
-123.456 <
12300.0 <
-
-0.
-0.123456 <
3.3333333333333335 <
1.1234567890123457 <
```

3. 복소수형

- x + yj 처럼 복소수 형태로 쓰여집니다. i가 아니라 j입니다.
- x 는 실수 부분이고 y 는 허수 부분입니다.
- 실수부와 허수부를 따로 분리가 가능합니다.
- 복소수끼리의 연산도 지원 합니다.

Ex03_number3.py

복소수형 :

실행 결과

```
print(complex_number3, type(complex_number3))
print(complex_number4, type(complex_number4))
print(complex_number5, type(complex_number5))
print(complex_number6, type(complex_number6))

print(complex_number1.real, complex_number1.imag)
print(complex_number2.real, complex_number2.imag)
```

(

print(complex_number3, type(complex_number3))
print(complex_number4, type(complex_number4))
print(complex_number5, type(complex_number5))
print(complex_number6, type(complex_number6))

print(complex_number1.real, complex_number1.imag)
print(complex_number2.real, complex_number2.imag)
(1.2+

print(complex_number3, type(complex_number3))
print(complex_number4, type(complex_number4))
print(complex_number5, type(complex_number5))
print(complex_number6, type(complex_number6))

print(complex_number1.real, complex_number1.imag)
print(complex_number2.real, complex_number2.imag)
(1.2+3.4j) <

4. 출력형식 지정

Ex04_number4.py
number =
print(
print("{0:,}".
print("{0:,}".format(number))

numberString =
numberString = format(number,
numberString = format(number, ',')
numberString = format(number, ',')
print(numberString)

print(
print(format(
print(format(1234567890.56789,
print(format(1234567890.56789, ","))
print(
print(format(
print(format(1234567890.56789,
print(format(1234567890.56789, "E"))

```
print(
print(format(
print(format(123,
print(format(123, "x"))
print(
print(format(
print(format(123,
print(format(123, "o"))
print(
print(format(
print(format(123,
print(format(123, "b"))
print(
print(format(
print(format(12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
```

```
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
```

```
print(format(-12345.56789, "12,.2f"))
print(format(-
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789,
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
```

```
print(format(12345.56789, "+12,.2f"))
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789, "+12,.2f"))
print(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789, "+12,.2f"))
print(format(
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789, "+12,.2f"))
print(format(0.
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
print(format(12345.56789, "+12,.2f"))
print(format(0.1234,
```

실행 결과

```
print(format(12345.56789, ">-12,.2f"))
print(format(12345.56789, "^-012,.2f"))
print(format(12345.56789, "<-012,.2f"))
print(format(12345.56789, "@>12,.2f"))
print(format(-12345.56789, "12,.2f"))
print(format(-12345.56789, "-12,.2f"))
```

```
print(format(12345.56789, "+12,.2f"))
print(format(0.1234, ".1%"))
12,
12,
1,
1,234,
1,234,567,
1.234568E+09
1.234568E+09
7b

    12,
012,
12,
@@@12,
  -
  -12,
  -
  -12,
  +
  +12,
12.3%
```

마지막 편집일시 : 2018년 7월 9일 11:12

02. Python Strings

1. 문자열이란?

문자열(String)이란 문자와 단어 등으로 구성된 문자들의 집합을 의미합니다.
다른 많은 인기있는 프로그래밍 언어와 마찬가지로 Python의 문자열은 유니 코드 문자를 나타내는 바이트 배열입니다. 그러나 파이썬에는 문자 데이터 유형이 없으며 단일 문자는 길이가 1 인 문자열입니다.
숫자도 따옴표안에 있으면 문자열 입니다.
파이썬의 문자열 리터럴은 작은 따옴표 또는 큰 따옴표로 묶습니다.
다른 방법으로는 작은 따옴표 또는 큰 따옴표 세개를 연달아 입력하는 방법이 있습니다.
다음은 모두 문자열 입니다.

```
'Python String! 한글 1234 !@#$%^'
"Python String! 한글 1234 !@#$%^"
'''Python String! 한글 1234 !@#$%^'''
"""Python String! 한글 1234 !@#$%^"""
```

2. 문자열 만드는 방법

파이썬에서는 문자열을 사용할 때 주로 작은따옴표를 사용합니다. 물론 " "(큰따옴표)로도

문자열을 사용할 수도 있습니다. 문자열을 만들어 바로 출력할 수도 있지만 가공하거나 여러가지 작업을 하기 위해서는 변수에 저장하여 처리합니다.

```
str1 = 'Hello Python!!!'
str2 = "Hello Python!!!"
str3 = '''Hello Python!!!'''
str4 = """Hello Python!!!"""
```

그런데 문자열을 만드는 방법이 왜 이렇게 많을까요?
문자열 안에 작은따옴표나 큰따옴표를 포함시키고 싶을 때 다른 언어들은 다음과 같이 확장문자열을 이용해야 합니다. 하지만 Python은 문자열을 만드는 여러 방법이 있으므로 편리하게 만들 수 있습니다.

```
# 문자열 안에 따옴표 자체를 포함하는 방법
str5 = 'Python String! "한글" 1234 !@#$%^'
str6 = 'Python String! \'한글\' 1234 !@#$%^'
str7 = "Python String! '한글' 1234 !@#$%^"
str8 = "Python String! \"한글\" 1234 !@#$%^"
```

Python은 기본 확장 문자(escape sequence)를 지원합니다. 확장문자의 사용은 표준 출력 부분을 참조하시기 바랍니다.

Ex05_string1.py

```
# 파이선 문자열
str1 = 'Python String! 한글 1234 !@#$%^'
str2 = "Python String! 한글 1234 !@#$%^"
str3 = '''Python String! 한글 1234 !@#$%^'''
str4 = """Python String! 한글 1234 !@#$%^"""
print(str1, str2, str3, str4, sep="\n")

# 문자열 안에 따옴표 자체를 포함하는 방법
str5 = 'Python String! "한글" 1234 !@#$%^'
str6 = 'Python String! \'한글\' 1234 !@#$%^'
str7 = "Python String! '한글' 1234 !@#$%^"
str8 = "Python String! \"한글\" 1234 !@#$%^"
print(str5, str6, str7, str8, sep="\n")
```

실행 결과

```
Python String! 한글 1234 !@#$%^
Python String! 한글 1234 !@#$%^
Python String! 한글 1234 !@#$%^
Python String! 한글 1234 !@#$%^
```

Python String! "한글" 1234 !@#$%^
Python String! '한글' 1234 !@#$%^
Python String! '한글' 1234 !@#$%^
Python String! "한글" 1234 !@#$%^
다른 이유로는 여러 줄의 문자열을 입력하고 싶을때 입니다.

Ex06_string2.py

```
# 파이선 문자열
str1 = '첫번째 줄 두번째 줄 세번째 줄'
str2 = '첫번째 줄\n두번째 줄\n세번째 줄\n'
str3 = '''첫번째 줄
두번째 줄
세번째 줄
'''
str4 = """첫번째 줄
두번째 줄
세번째 줄
"""
str5 = "첫번째 줄 " +  \
       "두번째 줄 " + \
       "세번째 줄"
print(str1, str2, str3, str4, sep="\n")
print(str5)
```

실행 결과

```
첫번째 줄 두번째 줄 세번째 줄
첫번째 줄
두번째 줄
세번째 줄

첫번째 줄
두번째 줄
세번째 줄

첫번째 줄
두번째 줄
세번째 줄

첫번째 줄 두번째 줄 세번째 줄
```

여러 줄을 입력할때 기본 확장 문자(escape sequence)을 사용하는것 보다 작은 따옴표 또는

큰 따옴표 세개를 연달아 입력하는 방법이 편리 합니다.
마지막의 str5는 여러 줄로 나누어 입력했지만 1줄입니다.

3. 파이썬 문서 문자열
Docstring이란 무엇입니까? 파이썬 문서화 문자열 (또는 문서화 문자열)은 파이썬 모듈, 함수, 클래스 및 메소드와 문자열을 연결시키는 것을 말합니다.

작성하는 모듈, 함수, 클래스 및 메소드의 첫번째 줄에 개체의 정의에 포함시킵니다. 주석과 같지만 소스 코드에서 문서화하기 위해 사용됩니다. 모든 함수에는 docstring이 있어야합니다.

Docstring은 객체의 doc 속성과 help명령을 통해 액세스 할 수 있습니다.

작은 따옴표 또는 큰 따옴표 세개를 연달아 입력하는 방법으로 Docstring을 지정합니다.
권장사항으로 큰따옴표 세개를 사용합니다.

Ex07_string3.py

```
# 파이선 문자열
def show1(message):
    '''
    메시지를 출력해주는 함수
    형식) show1(message)
    '''
    print(message)


def show2(message='Hello Python!!'):
    """
    메시지를 출력해주는 함수
    형식) show2(message='Hello Python!!')
    """
    print(message)


print(show1.__doc__)
print('-' * 50)

help(show1)
print('*' * 50)
```

```
print(show2.__doc__)
print('*' * 50)

help(show2)
print('*' * 50)

show1("반갑습니다. 파이썬!!!")
show2()
```

실행 결과

```
    메시지를 출력해주는 함수
    형식) show1(message)

--------------------------------------------------
Help on function show1 in module __main__:

show1(message)
    메시지를 출력해주는 함수
    형식) show1(message)

**************************************************

    메시지를 출력해주는 함수
    형식) show2(message='Hello Python!!')

**************************************************
Help on function show2 in module __main__:

show2(message='Hello Python!!')
    메시지를 출력해주는 함수
    형식) show2(message='Hello Python!!')

**************************************************
반갑습니다. 파이썬!!!
Hello Python!!
```

4. 간단한 문자열 처리 함수들

문자열을 다루는 방법은 세번째 계단에서 자세하게 다루도록 하겠습니다. 여기서는 간단하게 몇가지 자주 사용되는 몇가지 정보만 알아보도록 하겠습니다.

문자열 길이 구하기

len(문자열)

문자열의 길이를 구해줍니다. 한글, 공백, 특수문자도 1글자로 인식합니다.
참고로 파이썬 3에서 len은 문자의 개수를 구해주지만 파이썬 2.7에서는 실제 바이트 수를 구해주는 차이점이 있습니다. 즉, 한글 문자열의 길이를 구할 때 파이썬 버전에 따라 결과가 다르므로 주의해야 합니다.

문자열 연결과 반복하기
문자열1 + 문자열2
문자열 * 정수
+ 연산자를 이용하여 문자열을 더합니다.
문자열에 정수를 곱하면 동일한 문자열을 정수만큼 반복합니다. 문자열과 숫자를 + 연산자로 결합하면 아래와 같은 에러가 발생합니다. 이때는 숫자를 문자열로 변경해주는 str()함수를 사용해야 합니다.

```
Traceback (most recent call last):
  File "..../Ex08_string4.py", line 16, in <module>
    print("'" + str1 + "'의 길이는 " + len(str1) + "입니다.")
TypeError: must be str, not int
```

문자열 잘라내기
문자열[n:m]
[n] : 1글자 잘라내기
[n:m] : n부터 m-1까지 문자열을 리턴합니다.
[:m] : 앞의 숫자를 생략하면 0부터 시작합니다.
[n:] : 뒤의 숫자를 생략하면 끝 글자까지 입니다.
[:] : 모두 생략하면 전체입니다.
[-n] : 음수 값을 지정하면 뒤에서부터 카운팅합니다.

앞뒤 공백없애기
문자열.strip() , 문자열.lstrip(), 문자열.rstrip
strip() : 앞, 뒤 공백을 모두 없앱니다.
lstrip() : 왼쪽 공백을 없앱니다.
rstrip() : 오른쪽 공백을 없앱니다.

문자 개수세기
문자열.count(문자열)
문자열에서 지정 문자열의 개수를 세어 줍니다.

간단한 예제
Ex08_string4.py

```
# 파이선 문자열
str1 = "Hello Python!!"
```

```
str2 = "반갑다 파이썬!!"

# 문자열 길이
print(str1, "의길이", len(str1))  # 14
print(str2, "의길이", len(str2))  # 9 : 한글도 1글자로 인식

# 문자열 연결하기
str3 = str1 + str2
str4 = "^" * 10

print(str3)
print(str4)

# 문자열과 숫자형의 결합
# print("'" + str1 + "'의 길이는 " + len(str1) + "입니다.")
print("'" + str1 + "'의 길이는 " + str(len(str1)) + "입니다.")

# 문자열 잘라내기
print(str1[0], str1[6])  # 1글자 잘라내기
print(str1[3:7])  # [n:m] : n부터 m-1까지 문자열을 리턴합니다.
print(str1[:5])   # 앞의 숫자를 생략하면 0부터 입니다.
print(str1[6:])   # 뒤의 숫자를 생략하면 끝 글자까지 입니다.
print(str1[:])    # 모두 생략하면 전체입니다.
print(str1[:-2])  # 음수 값을 지정하면 뒤에서부터 카운팅합니다.

# 앞뒤 공백 없애기
str5 = "  앞 뒤에 공백이 있습니다.  "
print('[', str5, ']',sep='')
print('[', str5.lstrip(), ']',sep='')
print('[', str5.rstrip(), ']',sep='')
print('[', str5.strip(), ']',sep='')

print('"', str5,'"', '의 공백의 개수는 ', str5.count(' '), '개입니다.', sep='')
print('"', str1, '"', '의 ll의 개수는 ', str1.count('ll'), '개입니다.', sep='')
```

실행 결과

```
Hello Python!! 의길이 14
반갑다 파이썬!! 의길이 9
Hello Python!!반갑다 파이썬!!
^^^^^^^^^^
'Hello Python!!'의 길이는 14입니다.
```

H P
lo P
Hello
Python!!
Hello Python!!
Hello Python
[앞 뒤에 공백이 있습니다.]
[앞 뒤에 공백이 있습니다.]
[앞 뒤에 공백이 있습니다.]
[앞 뒤에 공백이 있습니다.]
" 앞 뒤에 공백이 있습니다. "의 공백의 개수는 7개입니다.
"Hello Python!!"의 ll의 개수는 1개입니다.
대, 소문자 바꾸기
문자열.upper(), 문자열.lower()
문자열을 대문자 또는 소문자로 바꿔줍니다.

문자열 찾기
문자열.find(문자열)
문자열.index(문자열)
두개의 차이점은 find()는 없으면 -1을 리턴하고 index()는 에러를 발생합니다.

문자열 바꾸기
문자열.replace('원본문자열', '바뀔문자열',개수=None)

※ Python에서 문자열은 불변객체입니다.
자신이 변경되는것이 아니라 항상 새로운 객체를 만들어 리턴합니다.
변경을 하려고하면 다음과 같이 에러를 발생합니다.

str1[0] = 'H' # 에러 : 불변객체

TypeError: 'str' object does not support item assignment
문자열 나누기, 합치기
문자열 나누기 : 문자열.split(self, sep=None, maxsplit=-1)
문자열 합치기 : 문자열.join(self, iterable)

간단한 예제
Ex09_string5.py

```
str1 = "hello python!!"
print(str1)
print(str1.lower())
```

```
print(str1.upper())
print(str1[0].upper())
print(str1.find(' '))
print(str1[0].upper() + str1[1:str1.find(' ')])
print(str1[str1.find(' ')+1].upper())
print(str1[str1.find(' ')+2:])

# 단어의 첫글자만 대문자로 바꾸기
print(str1[0].upper() + str1[1:str1.find(' ')] + ' ' +
      str1[str1.find(' ')+1].upper() + str1[str1.find(' ')+2:])

print(str1[0].upper() + str1[1:str1.index(' ')] + ' ' +
      str1[str1.index(' ')+1].upper() + str1[str1.index(' ')+2:])
print(str1.capitalize())
print('-' * 50)

print(str1.replace('python', 'world'))
print(str1)  # 원본 변경 않됨
print(str1.replace('!', '^-^', 1))  # 세번째 인수는 개수
print('-' * 50)

# str1[0] = 'H' # 에러 : 불변객체

str2 = "한놈 두식이 석삼 너구리 오징어"
str3 = str2.split()
print(str2, type(str2))
print(str3, type(str3))

str4 = "한놈, 두식이, 석삼, 너구리, 오징어"
str5 = str4.split(", ")
print(str4, type(str4))
print(str5, type(str5))

str6 = ","
print(str6.join(str3))
str6 = "|"
print(str6.join(str5))
```

실행 결과

```
hello python!!
hello python!!
```

```
HELLO PYTHON!!
H
5
Hello
P
ython!!
Hello Python!!
Hello Python!!
Hello python!!
--------------------------------------------------
hello world!!
hello python!!
hello python^-^!
--------------------------------------------------
한놈 두식이 석삼 너구리 오징어 <class 'str'>
['한놈', '두식이', '석삼', '너구리', '오징어'] <class 'list'>
한놈, 두식이, 석삼, 너구리, 오징어 <class 'str'>
['한놈', '두식이', '석삼', '너구리', '오징어'] <class 'list'>
한놈,두식이,석삼,너구리,오징어
한놈|두식이|석삼|너구리|오징어
```

간단한 변환함수

ord : 문자의 아스키 코드값을 리턴하는 함수

hex : hex(x)는 정수값을 입력받아 16진수(hexadecimal)로 변환하여 리턴하는 함수

otc : oct(x)는 정수 형태의 숫자를 8진수 문자열로 바꾸어 리턴하는 함수

chr : 아스키(ASCII) 코드값을 입력으로 받아 그 코드에 해당하는 문자를 리턴하는 함수

capitalize : 단어의 첫글자만 대문자로 변환하는 함수

간단한 예제

Ex10_string6.py

```
# 파이선 문자열
# ord : 문자의 아스키 코드값을 리턴하는 함수
# hex : hex(x)는 정수값을 입력받아 16진수(hexadecimal)로 변환하여 리턴하는 함수
# otc : oct(x)는 정수 형태의 숫자를 8진수 문자열로 바꾸어 리턴하는 함수이다.
print(ord("A"))          # 출력 결과: 65
print(hex(ord("A")))   # 출력 결과: 0x41
print(oct(ord("A")))   # 출력 결과: 0o101

# chr : 아스키(ASCII) 코드값을 입력으로 받아 그 코드에 해당하는 문자를 리턴하는 함수
print(chr(65))           # 출력 결과: A
print(chr(0x41))         # 출력 결과: A
```

```
# 대소문자 변환
print(chr(ord('A')+32))  # 출력 결과: a
print(chr(ord('Z')+32))  # 출력 결과: z
print(chr(ord('a')-32))  # 출력 결과: A
print(chr(ord('z')-32))  # 출력 결과: Z

# 1자리 숫자 문자열을 숫자로 변환
print(ord('1') - ord('0'), type(ord('1') - ord('0')))
print(ord('9') - ord('0'), type(ord('9') - ord('0')))

# 1자리 숫자를 숫자 문자열로 변환
print(chr(1 + ord('0')), type(chr(1 + ord('0'))))
print(chr(9 + ord('0')), type(chr(9 + ord('0'))))
```

실행 결과

```
65
0x41
0o101
A
A
a
z
A
Z
1 <class 'int'>
9 <class 'int'>
1 <class 'str'>
9 <class 'str'>
```

마지막 편집일시 : 2018년 7월 9일 5:43

==========

03. Python Tuple

1. tuple이란?

tuple 자료형은 데이터의 목록이라고 보면 됩니다.

콤마(,)로 데이터를 나열하면 tuple입니다.

괄호()안에 ,로 나열해도 tuple입니다.

길이 1개짜리 tuple을 만들려면 반드시 뒤에 콤마(,)를 붙여야 합니다.

tuple은 다른 자료형들로 만들 수 있습니다.

tuple은 tuple을 가질 수 있습니다.

Ex11_tuple1.py

```
# tuple 자료형이란?
# tuple 만들기
tuple1 = 1, 2, 3, 4, 5
tuple2 = (6, 7, 8, 9, 10)
print(tuple1, type(tuple1))
print(tuple2, type(tuple2))

# 다음은 tuple이 아닙니다.
tuple3 = 1
tuple4 = (6)
print(tuple3, type(tuple3))
print(tuple4, type(tuple4))

# 다음은 tuple입니다.
tuple3 = 1,
tuple4 = (6,)
print(tuple3, type(tuple3))
print(tuple4, type(tuple4))

# 다른 자료형들의 모임
tuple5 = '한놈', 123, True
print(tuple5, type(tuple5))

# tuple이 tuple을 가짐
tuple6 = '회원목록', ('한놈','두식이'),(33,25), (True, False)
print(tuple6, type(tuple6))
```

실행 결과

```
(1, 2, 3, 4, 5) <class 'tuple'>
(6, 7, 8, 9, 10) <class 'tuple'>
1 <class 'int'>
6 <class 'int'>
(1,) <class 'tuple'>
(6,) <class 'tuple'>
('한놈', 123, True) <class 'tuple'>
('회원목록', ('한놈', '두식이'), (33, 25), (True, False)) <class 'tuple'>
```

2. tuple에 접근하기

len(tuple)로 요소의 개수를 알아낼 수 있습니다.

[n:m]으로 요소에 접근이 가능합니다. 사용법은 문자열과 동일합니다.

반복문을 이용하여 접근이 가능합니다.

+연산을 이용하여 결합이 가능합니다.
*연산을 이용하여 반복할 수 있습니다.
Ex12_tuple2.py

```
# tuple 자료형이란?
tuple1 = 1, 2, 3
print(tuple1, type(tuple1))
print('*' * 50)

print("개수 : " + str(len(tuple1)))
print('*' * 50)

print(tuple1[0], tuple1[1],tuple1[2],tuple1[-1], type(tuple1[0]))
print(tuple1[0:1],type(tuple1[0:1]))
print(tuple1[1:3],type(tuple1[0:1]))
print('*' * 50)

for data in tuple1:
    print(data)
print('*' * 50)

tuple5 = '회원목록', ('한놈','두식이'), (33, 25), (True, False)
print(tuple5, type(tuple5))
print(tuple5[0])
title = ('', '이름', '나이', '성별')
for i in (1, 2, 3):
    print('[', title[i], ']')
    for j in (0, 1):
        print(tuple5[i][j])
    print('-' * 30)

print('*' * 50)
tuple2 = ('a', 'b', 'c', 'd')
tuple3 = tuple1 + tuple2
tuple4 = tuple2 * 3
print(tuple2)
print(tuple3)
print(tuple4)
```

실행 결과

```
(1, 2, 3) <class 'tuple'>
```

```
**************************************************
개수 : 3
**************************************************
1 2 3 3 <class 'int'>
(1,) <class 'tuple'>
(2, 3) <class 'tuple'>
**************************************************
1
2
3
**************************************************
('회원목록', ('한놈', '두식이'), (33, 25), (True, False)) <class 'tuple'>
회원목록
[ 이름 ]
한놈
두식이
------------------------------
[ 나이 ]
33
25
------------------------------
[ 성별 ]
True
False
------------------------------
**************************************************
('a', 'b', 'c', 'd')
(1, 2, 3, 'a', 'b', 'c', 'd')
('a', 'b', 'c', 'd', 'a', 'b', 'c', 'd', 'a', 'b', 'c', 'd')
3. tuple은 불변 객체
tuple의 요소 값 변경이 불가합니다.
tuple의 요소 삭제가 불가합니다.
값을 변경하면 다음과 같은 에러가 발생 합니다.

Traceback (most recent call last):
  File "..../Ex13_tuple3.py", line 7, in <module>
    tuple1[0] = 'A'
TypeError: 'tuple' object does not support item assignment
값을 삭제하면 다음과 같은 에러가 발생합니다.

Traceback (most recent call last):
```

```
  File "..../Ex13_tuple3.py", line 10, in <module>
    del tuple1[0]
TypeError: 'tuple' object doesn't support item deletion
```
그러므로 다음과 같이 새로운 tuple을 생성해서 대입하여야 합니다.

Ex13_tuple2.py

```
# Tuple 자료형이란?
# Tuple은 불변 객체
tuple1 = 1, 2, 3, 4
print(tuple1, type(tuple1))

# 요소 값 변경
# tuple1[0] = 'A'

# 요소 값 삭제
# del tuple1[0]

# 첫번째 값을 변경하려면 : 새로운 객체를 만들어야 합니다.
tuple1 = ('A',) + tuple1[1:]
print(tuple1)

# 세번째 값을 변경하려면
tuple1 = tuple1[:2] + ('C',) + tuple1[3:]
print(tuple1)

# 두번째 값을 삭제하려면
tuple1 = (tuple1[0],) + tuple1[2:]
print(tuple1)
```
실행 결과

```
(1, 2, 3, 4) <class 'tuple'>
('A', 2, 3, 4)
('A', 2, 'C', 4)
('A', 'C', 4)
```
마지막 편집일시 : 2018년 7월 9일 5:52

---

04. Python Set

1. set이란?

set 자료형은 중복을 허용하지 않습니다.

set은 입력된 순서는 중요하지 않습니다.

set() 생성자 함수를 이용해서 만들 수 있습니다.
{}를 이용해서 만들 수 있습니다.
add() 메서드를 이용하여 추가가 가능합니다.
update() 메서드를 이용하여 동시에 여러개 추가가 가능합니다.
remove() 메서드를 이용하여 삭제가 가능합니다.
copy() 메서드를 이용하여 복사가 가능합니다.
clear() 메서드를 이용하여 모든 요소 삭제가 가능합니다.
Ex14_set1.py

```
# set 자료형이란?
# 순서에 상관없이 중복을 제거하는 자료형
set1 = set("Hello Python!!!")
print(set1, type(set1))

set2 = {1,2,3, 'a', 'b', 'c', True, 3.14}
print(set2, type(set2))


set3 = set({1,2,3})
print(set3, type(set3))

set4 = set([1,2,3])
print(set4, type(set4))

set5 = set()
print(set5, type(set5))

for i in range(0,5):
    set5.add(i)

for i in range(0,5):
    set5.add(i)

print(set5, type(set5))

set5.add('a')
set5.add('문자열')
set5.add(34.6)
print(set5, type(set5))

set5.update({11, 22, 33})
```

```
print(set5, type(set5))

set5.remove(0)
set5.remove(2)
print(set5, type(set5))

set6 = set5.copy()
print(set6, type(set6))

set6.clear()
print(set6, type(set6))
```

실행 결과

```
{'t', ' ', 'l', 'P', 'h', '!', 'H', 'e', 'y', 'o', 'n'} <class 'set'>
{1, 2, 3, 3.14, 'a', 'c', 'b'} <class 'set'>
{1, 2, 3} <class 'set'>
{1, 2, 3} <class 'set'>
set() <class 'set'>
{0, 1, 2, 3, 4} <class 'set'>
{0, 1, 2, 3, 4, 34.6, 'a', '문자열'} <class 'set'>
{0, 1, 2, 3, 4, 34.6, 33, 'a', 11, 22, '문자열'} <class 'set'>
{1, 3, 4, 34.6, 33, 'a', 11, 22, '문자열'} <class 'set'>
{1, 34.6, 3, 4, 33, 'a', 11, 22, '문자열'} <class 'set'>
set() <class 'set'>
```

2. set활용하기

천자문은 동일한 글자는 세지 않는다면 한글로는 몇글자일까?

Ex15_set2.py

사용된 파일들

chunja.txt

chunja2.txt

national_anthem.txt

```
# 텍스트 파일 읽기전용으로 열기
f = open('chunja.txt', 'r',encoding='UTF-8')
# 모든 내용을 리스트로 읽기
lines = f.readlines();
# 타입 검사
print(type(lines))
# set 만들기
hanja_set = set()
```

```
hangul_set = set()
# 1줄씩 끝까지 반복
for ch in lines:
    # print(ch.split("|")[1], ch.split("|")[2], sep='')
    # set에 추가
    hanja_set.add(ch.split("|")[1])
    hangul_set.add(ch.split("|")[2])

print("한자수 : {}글자".format(len(hanja_set)))
print("한글수 : {}글자".format(len(hangul_set)))
```

실행 결과

```
<class 'list'>
한자수 : 1000글자
한글수 : 321글자
```

애국가는 몇개의 단어로 만들어 졌을까?

Ex16_set3.py

```
national_anthem = '''
동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리나라 만세
무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세
남산 위에 저 소나무 철갑을 두른듯 바람서리 불변함은 우리 기상일세
무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세
가을 하늘 공활한데 높고 구름 없이 밝은 달은 우리 가슴 일편단심일세
무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세
이 기상과 이 마음으로 충성을 다하여 괴로우나 즐거우나 나라사랑하세
무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세
'''
# print(national_anthem)
national_anthem = national_anthem.replace('\n', '')
# print(national_anthem)

# '대한'이란 단어의 개수
print('애국가에는 "대한"이란 문자열이 {}개
나타납니다.'.format(national_anthem.count('대한')))
# 공백으로 구분하여 list로 만들기
word_list = national_anthem.split(' ')
# 전체 단어 수
print('애국가에 사용된 단어 수는 총 {}개 입니다.'.format(len(word_list)))
# 단어를 set으로 만들기
word_set = set(word_list)
```

```
print('애국가에는 총 {}개 단어가 사용되었습니다.'.format(len(word_set)))
print('애국가에는 총 {}개 단어가 중복
사용되었습니다.'.format(len(word_list)-len(word_set)))


# 단어 출력
# for str in word_set:
#       print('[',str,']')
```

실행 결과

```
애국가에는 "대한"이란 문자열이 8개 나타납니다.
애국가에 사용된 단어 수는 총 70개 입니다.
애국가에는 총 44개 단어가 사용되었습니다.
애국가에는 총 26개 단어가 중복 사용되었습니다.
```

중복되지 않는 번호 뽑기?

random 모듈을 이용하면 더 편리한 방법이 있습니다. 여기서는 set의 특징을 보는 예제입니다.

Ex17_set4.py

```
import random as rnd

game = int(input("몇게임?"))
for i in range(1, game+1):
    # 중복되지 않은 숫자 6개 뽑기
    lotto_set = set()
    while len(lotto_set) < 6:
        lotto_set.add(rnd.randint(1,45))

    # print(lotto_set)
    print(i, '게임 :', sorted(lotto_set))
    game -= 1
```

실행 결과

```
몇게임?5
1 게임 : [2, 4, 11, 27, 37, 44]
2 게임 : [5, 20, 21, 32, 38, 42]
3 게임 : [1, 11, 12, 23, 33, 44]
4 게임 : [2, 8, 12, 23, 29, 43]
5 게임 : [1, 2, 12, 26, 37, 43]
```

마지막 편집일시 : 2018년 7월 10일 12:15

---
01. chunja1.txt
1|天|천|하늘 천
2|地|지|땅 지
3|玄|현|검을 현
4|黃|황|누를 황
5|宇|우|집 우
6|宙|주|집 주
7|洪|홍|넓을 홍
8|荒|황|거칠 황
9|日|일|날 일
10|月|월|달 월
11|盈|영|찰 영
12|昃|측|기울 측
13|辰|진|별 진
14|宿|숙|잘 숙
15|列|열|벌일 열
16|張|장|베풀 장
17|寒|한|찰 한
18|來|래|올 래
19|暑|서|더울 서
20|往|왕|갈 왕
21|秋|추|가을 추
22|收|수|거둘 수
23|冬|동|겨울 동
24|藏|장|감출 장
25|閏|윤|윤달 윤
26|餘|여|남을 여
27|成|성|이룰 성
28|歲|세|해 세
29|律|율|가락 률
30|呂|려|음률 려
31|調|조|고를 조
32|陽|양|볕 양
33|雲|운|구름 운
34|騰|등|오를 등
35|致|치|이를 치
36|雨|우|비 우
37|露|노|이슬 로
38|結|결|맺을 결
39|爲|위|할 위

40|霜|상|서리 상
41|金|금|쇠 금
42|生|생|낳을 생
43|麗|여|고울 려
44|水|수|물 수
45|玉|옥|구슬 옥
46|出|출|날 출
47|崑|곤|메 곤
48|岡|강|메 강
49|劍|검|칼 검
50|號|호|이름 호
51|巨|거|클 거
52|闕|궐|대궐 궐
53|珠|주|구슬 주
54|稱|칭|일컬을 칭
55|夜|야|밤 야
56|光|광|빛 광
57|果|과|과실 과
58|珍|진|보배 진
59|李|이|오얏 리
60|柰|내|능금나무 내
61|菜|채|나물 채
62|重|중|무거울 중
63|芥|개|겨자 개
64|薑|강|생강 강
65|海|해|바다 해
66|鹹|함|짤 함
67|河|하|물 하
68|淡|담|묽을 담
69|鱗|인|비늘 린
70|潛|잠|잠길 잠
71|羽|우|깃 우
72|翔|상|높이 날 상
73|龍|용|용 룡
74|師|사|스승 사
75|火|화|불 화
76|帝|제|임금 제
77|鳥|조|새 조
78|官|관|벼슬 관
79|人|인|사람 인
80|皇|황|임금 황

81|始|시|처음 시
82|制|제|지을 제
83|文|문|글월 문
84|字|자|글자 자
85|乃|내|이에 내
86|服|복|옷 복
87|衣|의|옷 의
88|裳|상|치마 상
89|推|추|밀 추
90|位|위|자리 위
91|讓|양|사양할 양
92|國|국|나라 국
93|有|유|있을 유
94|虞|우|헤아릴 우
95|陶|도|질그릇 도
96|唐|당|당나라 당
97|弔|조|슬퍼할 조
98|民|민|백성 민
99|伐|벌|칠 벌
100|罪|죄|허물 죄
101|周|주|두루 주
102|發|발|필 발
103|殷|은|나라이름 은
104|湯|탕|끓을 탕
105|坐|좌|앉을 좌
106|朝|조|아침 조
107|問|문|물을 문
108|道|도|길/말할 도
109|垂|수|드리울 수
110|拱|공|껴안을 공
111|平|평|평평할 평
112|章|장|글월 장
113|愛|애|사랑 애
114|育|육|기를 육
115|黎|여|검을 려
116|首|수|머리 수
117|臣|신|신하 신
118|伏|복|엎드릴 복
119|戎|융|오랑캐 융
120|羌|강|종족이름 강
121|遐|하|멀 하

122|邇|이|가까울 이
123|壹|일|한 일
124|體|체|몸 체
125|率|솔|거느릴 솔/비율 률
126|賓|빈|손 빈
127|歸|귀|돌아갈 귀
128|王|왕|임금 왕
129|鳴|명|울 명
130|鳳|봉|봉황새 봉
131|在|재|있을 재
132|樹|수|나무 수
133|白|백|흰 백
134|駒|구|망아지 구
135|食|식|밥 식
136|場|장|마당 장
137|化|화|될 화
138|被|피|입을 피
139|草|초|풀 초
140|木|목|나무 목
141|賴|뇌|힘입을 뢰
142|及|급|미칠 급
143|萬|만|일만 만
144|方|방|모 방
145|蓋|개|덮을 개
146|此|차|이 차
147|身|신|몸 신
148|髮|발|터럭 발
149|四|사|넉 사
150|大|대|큰 대
151|五|오|다섯 오
152|常|상|항상 상
153|恭|공|공손할 공
154|惟|유|오직 유
155|鞠|국|국문할 국
156|養|양|기를 양
157|豈|기|어찌 기
158|敢|감|감히 감
159|毁|훼|헐 훼
160|傷|상|상할 상
161|女|여|계집 녀
162|慕|모|사모할 모

163|貞|정|곧을 정
164|烈|열|매울 렬
165|男|남|사내 남
166|效|효|본받을 효
167|才|재|재주 재
168|良|량|어질 량
169|知|지|알 지
170|過|과|지날/허물 과
171|必|필|반드시 필
172|改|개|고칠 개
173|得|득|얻을 득
174|能|능|능할 능
175|莫|막|말 막
176|忘|망|잊을 망
177|罔|망|없을 망
178|談|담|말씀 담
179|彼|피|저 피
180|短|단|짧을 단
181|靡|미|아닐 미
182|恃|시|믿을 시
183|己|기|몸 기
184|長|장|길 장
185|信|신|믿을 신
186|使|사|하여금 사
187|可|가|옳을 가
188|覆|복|뒤집힐 복
189|器|기|그릇 기
190|欲|욕|하고자할 욕
191|難|난|어려울 난
192|量|량|헤아릴 량
193|墨|묵|먹 묵
194|悲|비|슬플 비
195|絲|사|실 사
196|染|염|물들일 염
197|詩|시|시 시
198|讚|찬|칭찬할 찬
199|羔|고|새끼양 고
200|羊|양|양 양
201|景|경|경치 경
202|行|행|다닐 행/항렬 항
203|維|유|벼리 유

204|賢|현|어질 현
205|克|극|이길 극
206|念|념|생각 념
207|作|작|지을 작
208|聖|성|성인 성
209|德|덕|덕 덕
210|建|건|세울 건
211|名|명|이름 명
212|立|립|설 립
213|形|형|모양 형
214|端|단|바를 단
215|表|표|겉 표
216|正|정|바를 정
217|空|공|빌 공
218|谷|곡|골 곡
219|傳|전|전할 전
220|聲|성|소리 성
221|虛|허|빌 허
222|堂|당|집 당
223|習|습|익힐 습
224|聽|청|들을 청
225|禍|화|재앙 화
226|因|인|인할 인
227|惡|악|악할 악/미워할 오
228|積|적|쌓을 적
229|福|복|복 복
230|緣|연|인연 연
231|善|선|착할 선
232|慶|경|경사 경
233|尺|척|자 척
234|壁|벽|구슬 벽
235|非|비|아닐 비
236|寶|보|보배 보
237|寸|촌|마디 촌
238|陰|음|그늘 음
239|是|시|옳을/이 시
240|競|경|다툴 경
241|資|자|자료 자
242|父|부|아비 부
243|事|사|일/섬길 사
244|君|군|임금 군

245|曰|왈|가로 왈
246|嚴|엄|엄할 엄
247|與|여|더불 여
248|敬|경|공경할 경
249|孝|효|효도 효
250|當|당|마땅할 당
251|竭|갈|다할 갈
252|力|력|힘 력
253|忠|충|충성 충
254|則|즉|곧 즉/법 칙
255|盡|진|다할 진
256|命|명|목숨 명
257|臨|임|임할 림
258|深|심|깊을 심
259|履|리|밟을 리
260|薄|박|얇을 박
261|夙|숙|일찍 숙
262|興|흥|흥할 흥
263|溫|온|따뜻할 온
264|凊|청|서늘할 정
265|似|사|같을 사
266|蘭|난|난초 란
267|斯|사|이 사
268|馨|형|향기 형
269|如|여|같을 여
270|松|송|소나무 송
271|之|지|갈 지
272|盛|성|성할 성
273|川|천|내 천
274|流|류|흐를 류
275|不|불|아니 불
276|息|식|쉴 식
277|淵|연|못 연
278|澄|징|맑을 징
279|取|취|취할 취
280|暎|영|비칠 영
281|容|용|얼굴 용
282|止|지|그칠 지
283|若|약|같을 약
284|思|사|생각 사
285|言|언|말씀 언

286|辭|사|말씀 사
287|安|안|편안 안
288|定|정|정할 정
289|篤|독|도타울 독
290|初|초|처음 초
291|誠|성|정성 성
292|美|미|아름다울 미
293|愼|신|삼갈 신
294|終|종|마지막 종
295|宜|의|마땅 의
296|令|령|하여금 령
297|榮|영|영화 영
298|業|업|업 업
299|所|소|바 소
300|基|기|터 기
301|籍|적|호적 적
302|甚|심|심할 심
303|無|무|없을 무
304|竟|경|마침내 경
305|學|학|배울 학
306|優|우|넉넉할 우
307|登|등|오를 등
308|仕|사|벼슬 사
309|攝|섭|잡을 섭
310|職|직|벼슬 직
311|從|종|좇을 종
312|政|정|정사 정
313|存|존|있을 존
314|以|이|써 이
315|甘|감|달 감
316|棠|당|해당화 당
317|去|거|갈 거
318|而|이|어조사 이
319|益|익|더할 익
320|詠|영|읊을 영
321|樂|악|풍류 악/즐길 락/좋아할 요
322|殊|수|다를 수
323|貴|귀|귀할 귀
324|賤|천|천할 천
325|禮|예|예도 례
326|別|별|다를 별

327|尊|존|높을 존
328|卑|비|낮을 비
329|上|상|위 상
330|和|화|화할 화
331|下|하|아래 하
332|睦|목|화목할 목
333|夫|부|지아비 부
334|唱|창|부를 창
335|婦|부|며느리 부
336|隨|수|따를 수
337|外|외|밖 외
338|受|수|받을 수
339|傅|부|스승 부
340|訓|훈|가르칠 훈
341|入|입|들 입
342|奉|봉|받들 봉
343|母|모|어미 모
344|儀|의|거동 의
345|諸|제|모두 제
346|姑|고|시어미 고
347|伯|백|맏 백
348|叔|숙|아재비 숙
349|猶|유|같을 유
350|子|자|아들 자
351|比|비|견줄 비
352|兒|아|아이 아
353|孔|공|구멍 공
354|懷|회|품을 회
355|兄|형|맏 형
356|弟|제|아우 제
357|同|동|한가지 동
358|氣|기|기운 기
359|連|연|이어질 연
360|枝|지|가지 지
361|交|교|사귈 교
362|友|우|벗 우
363|投|투|던질 투
364|分|분|나눌 분
365|切|절|끊을 절/모두 체
366|磨|마|갈 마
367|箴|잠|경계 잠

368|規|규|법 규
369|仁|인|어질 인
370|慈|자|사랑할 자
371|隱|은|숨을 은
372|惻|측|슬플 측
373|造|조|지을 조
374|次|차|버금 차
375|弗|불|아닐 불
376|離|리|떠날 리
377|節|절|마디 절
378|義|의|옳을 의
379|廉|염|청렴 렴
380|退|퇴|물러갈 퇴
381|顚|전|엎드러질 전
382|沛|패|자빠질 패
383|匪|비|아닐 비
384|虧|휴|이지러질 휴
385|性|성|성품 성
386|靜|정|고요할 정
387|情|정|뜻 정
388|逸|일|편안할 일
389|心|심|마음 심
390|動|동|움직일 동
391|神|신|귀신 신
392|疲|피|피곤할 피
393|守|수|지킬 수
394|眞|진|참 진
395|志|지|뜻 지
396|滿|만|찰 만
397|逐|축|쫓을 축
398|物|물|만물 물
399|意|의|뜻 의
400|移|이|옮길 이
401|堅|견|굳을 견
402|持|지|가질 지
403|雅|아|우아할 아
404|操|조|잡을 조
405|好|호|좋을 호
406|爵|작|벼슬 작
407|自|자|스스로 자
408|縻|미|얽을 미

409|都|도|도읍 도
410|邑|읍|고을 읍
411|華|화|빛날 화
412|夏|하|여름 하
413|東|동|동녘 동
414|西|서|서녘 서
415|二|이|두 이
416|京|경|서울 경
417|背|배|등 배
418|邙|망|산이름 망
419|面|면|낯 면
420|洛|낙|강이름 락
421|浮|부|뜰 부
422|渭|위|강이름 위
423|據|거|의거할 거
424|涇|경|통할 경
425|宮|궁|집 궁
426|殿|전|큰집 전
427|盤|반|서릴 반
428|鬱|울|답답 울
429|樓|누|다락 루
430|觀|관|볼 관
431|飛|비|날 비
432|驚|경|놀랄 경
433|圖|도|그림 도
434|寫|사|베낄 사
435|禽|금|날짐승 금
436|獸|수|짐승 수
437|畵|화|그림 화
438|采|채|채색 채
439|仙|선|신선 선
440|靈|령|신령 령
441|丙|병|남녘 병
442|舍|사|집 사
443|傍|방|곁 방
444|啓|계|열 계
445|甲|갑|갑옷 갑
446|帳|장|휘장 장
447|對|대|대답할 대
448|楹|영|기둥 영
449|肆|사|베풀 사

450|筵|연|자리 연
451|設|설|베풀 설
452|席|석|자리 석
453|鼓|고|북 고
454|瑟|슬|비파 슬
455|吹|취|불 취
456|笙|생|생황 생
457|陞|승|오른쪽 승
458|階|계|뜰 계
459|納|납|바칠 납
460|陛|폐|섬돌 폐
461|弁|변|고깔 변
462|轉|전|구를 전
463|疑|의|의심할 의
464|星|성|별 성
465|右|우|오를 우
466|通|통|통할 통
467|廣|광|넓을 광
468|內|내|안 내
469|左|좌|왼 좌
470|達|달|통달할 달
471|承|승|이을 승
472|明|명|밝을 명
473|既|기|이미 기
474|集|집|모을 집
475|墳|분|무덤 분
476|典|전|법 전
477|亦|역|또 역
478|聚|취|모을 취
479|群|군|무리 군
480|英|영|꽃부리 영
481|杜|두|막을 두
482|稿|고|볏짚 고
483|鍾|종|쇠북 종
484|隷|례|글씨 례
485|漆|칠|옻칠할 칠
486|書|서|글씨 서
487|壁|벽|벽 벽
488|經|경|날 경
489|府|부|마을 부
490|羅|라|벌릴 라

491|將|장|장수 장
492|相|상|서로 상
493|路|노|길 로
494|夾|협|낄 협
495|槐|괴|괴화나무 괴
496|卿|경|벼슬 경
497|戶|호|지게 호
498|封|봉|봉할 봉
499|八|팔|여덟 팔
500|縣|현|고을 현
501|家|가|집 가
502|給|급|줄 급
503|千|천|일천 천
504|兵|병|군사 병
505|高|고|높을 고
506|冠|관|갓 관
507|陪|배|더할 배
508|輦|련|손수레 련
509|驅|구|몰 구
510|轂|곡|바퀴 곡
511|振|진|떨친 진
512|纓|영|끈 영
513|世|세|세상 세
514|祿|록|녹 록
515|侈|치|사치할 치
516|富|부|부자 부
517|車|거|수레 거
518|駕|가|멍에 가
519|肥|비|살찔 비
520|輕|경|가벼울 경
521|策|책|꾀 책
522|功|공|공 공
523|茂|무|무성할 무
524|實|실|열매 실
525|勒|늑|굴레 륵
526|碑|비|비석 비
527|刻|각|새길 각
528|銘|명|새길 명
529|磻|반|강이름 반
530|溪|계|시내 계
531|伊|이|저 이

532|尹|윤|다스릴 윤
533|佐|좌|도울 좌
534|時|시|때 시
535|阿|아|언덕 아
536|衡|형|저울대 형
537|奄|엄|문득 엄
538|宅|택|집 댁/택
539|曲|곡|굽을 곡
540|阜|부|언덕 부
541|微|미|작을 미
542|旦|단|아침 단
543|孰|숙|누구 숙
544|營|영|경영 영
545|桓|환|굳셀 환
546|公|공|공변될 공
547|匡|광|바를 광
548|合|합|모을 합
549|濟|제|건널 제
550|弱|약|약할 약
551|扶|부|도울 부
552|傾|경|기울 경
553|綺|기|비단 기
554|回|회|돌아올 회
555|漢|한|한수 한
556|惠|혜|은혜 혜
557|兌|열|말씀 설/달랠 세/기뻐할 열
558|感|감|느낄 감
559|武|무|호반 무
560|丁|정|고무래 정
561|俊|준|준걸 준
562|乂|예|어질 예
563|密|밀|빽빽할 밀
564|勿|물|말 물
565|多|다|많을 다
566|士|사|선비 사
567|寔|식|이 식
568|寧|녕|편안 녕
569|晉|진|나라 진
570|楚|초|나라 초
571|更|갱|다시 갱/고칠 경
572|覇|패|으뜸 패

573|趙|조|나라 조
574|魏|위|나라 위
575|困|곤|곤할 곤
576|橫|횡|비낄 횡
577|假|가|거짓 가
578|途|도|길 도
579|滅|멸|멸할 멸
580|虢|괵|나라 괵
581|踐|천|밟을 천
582|土|토|흙 토
583|會|회|모일 회
584|盟|맹|맹세 맹
585|何|하|어찌 하
586|遵|준|좇을 준
587|約|약|약속할 약
588|法|법|법 법
589|韓|한|나라 한
590|弊|폐|해질 폐
591|煩|번|번거로울 번
592|刑|형|형벌 형
593|起|기|일어날 기
594|翦|전|자를 전
595|頗|파|자못 파
596|牧|목|칠 목
597|用|용|쓸 용
598|軍|군|군사 군
599|最|최|가장 최
600|精|정|정할 정
601|宣|선|베풀 선
602|威|위|위엄 위
603|沙|사|모래 사
604|漠|막|아득할 막
605|馳|치|달릴 치
606|譽|예|칭찬할 예
607|丹|단|붉을 단
608|靑|청|푸를 청
609|九|구|아홉 구
610|州|주|고을 주
611|禹|우|하우씨 우
612|跡|적|자취 적
613|百|백|일백 백

614|郡|군|고을 군
615|秦|진|나라 진
616|幷|병|아우를 병
617|嶽|악|산마루 악
618|宗|종|마루 종
619|恒|항|항상 항
620|岱|대|뫼 대
621|禪|선|터닦을 선
622|主|주|임금 주
623|云|운|이를 운
624|亭|정|정자 정
625|雁|안|기러기 안
626|門|문|문 문
627|紫|자|붉을 자
628|塞|새|변방 새
629|鷄|계|닭 계
630|田|전|밭 전
631|赤|적|붉을 적
632|城|성|성 성
633|昆|곤|맏 곤
634|池|지|못 지
635|碣|갈|돌 갈
636|石|석|돌 석
637|鉅|거|클 거
638|野|야|들 야
639|洞|동|골 동/꿰뚫을 통
640|庭|정|뜰 정
641|曠|광|빌 광
642|遠|원|멀 원
643|綿|면|이어질 면
644|邈|막|멀 막
645|巖|암|바위 암
646|峀|수|메뿌리 수
647|杳|묘|아득할 묘
648|冥|명|어두울 명
649|治|치|다스릴 치
650|本|본|근본 본
651|於|어|어조사 어
652|農|농|농사 농
653|務|무|힘쓸 무
654|玆|자|이 자

655|稼|가|심을 가
656|穡|색|거둘 색
657|俶|숙|비로소 숙
658|載|재|실을 재
659|南|남|남녘 남
660|畝|묘|이랑 묘
661|我|아|나 아
662|藝|예|재주 예
663|黍|서|기장 서
664|稷|직|피 직
665|稅|세|징수할 세
666|熟|숙|익을 숙
667|貢|공|바칠 공
668|新|신|새 신
669|勸|권|권할 권
670|賞|상|상줄 상
671|黜|출|물리칠 출
672|陟|척|오를 척
673|孟|맹|맏 맹
674|軻|가|수레 가
675|敦|돈|도타울 돈
676|素|소|흴 소
677|史|사|역사 사
678|魚|어|물고기 어
679|秉|병|잡을 병
680|直|직|곧을 직
681|庶|서|여러 서
682|幾|기|몇 기
683|中|중|가운데 중
684|庸|용|떳떳 용
685|勞|노|힘쓸 로
686|謙|겸|겸손 겸
687|謹|근|삼갈 근
688|勅|칙|칙서 칙
689|聆|영|들을 령
690|音|음|소리 음
691|察|찰|살필 찰
692|理|리|다스릴 리
693|鑑|감|거울 감
694|貌|모|모양 모
695|辨|변|분별 변

696|色|색|빛 색
697|貽|이|끼칠 이
698|厥|궐|그 궐
699|嘉|가|아름다울 가
700|猷|유|꾀 유
701|勉|면|힘쓸 면
702|其|기|그 기
703|祇|지|공경 지
704|植|식|심을 식
705|省|생|살필 성/덜 생
706|躬|궁|몸 궁
707|譏|기|나무랄 기
708|誡|계|경계 계
709|寵|총|고일 총
710|增|증|더할 증
711|抗|항|저항할 항
712|極|극|다할 극
713|殆|태|위태 태
714|辱|욕|욕할 욕
715|近|근|가까울 근
716|恥|치|부끄러울 치
717|林|임|수풀 림
718|皐|고|언덕 고
719|幸|행|다행 행
720|卽|즉|곧 즉
721|兩|양|두 량
722|疏|소|상소할 소
723|見|견|볼 견/나타날 현
724|機|기|틀 기
725|解|해|풀 해
726|組|조|짤 조
727|誰|수|누구 수
728|逼|핍|핍박할 핍
729|索|삭|찾을 색
730|居|거|살 거
731|閑|한|한가 한
732|處|처|곳 처
733|沈|침|잠길 침
734|黙|묵|잠잠할 묵
735|寂|적|고요할 적
736|寥|요|고요 요

737|求|구|구할 구
738|古|고|옛 고
739|尋|심|찾을 심
740|論|론|의논할 론
741|散|산|흩을 산
742|慮|려|생각 려
743|逍|소|거닐 소
744|遙|요|멀 요
745|欣|흔|기쁠 흔
746|奏|주|아뢸 주
747|累|누|여러 루
748|遣|견|보낼 견
749|慼|척|슬플 척
750|謝|사|사례 사
751|歡|환|기뻐할 환
752|招|초|부를 초
753|渠|거|개천 거
754|荷|하|연꽃 하
755|的|적|과녁 적
756|歷|력|지낼 력
757|園|원|동산 원
758|莽|망|풀 망
759|抽|추|빼낼 추
760|條|조|조목 조
761|枇|비|비파나무 비
762|杷|파|비파나무 파
763|晩|만|늦을 만
764|翠|취|푸를 취
765|梧|오|오동 오
766|桐|동|오동 동
767|早|조|이를 조
768|凋|조|시들 조
769|陳|진|베풀 진
770|根|근|뿌리 근
771|委|위|맡길 위
772|翳|예|가릴 예
773|落|낙|떨어질 락
774|葉|엽|잎사귀 엽
775|飄|표|나부낄 표
776|飄|요|나부낄 요
777|遊|유|헤엄칠 유

778|鵾|곤|곤새 곤
779|獨|독|홀로 독
780|運|운|운전 운
781|凌|능|업신여길 릉
782|摩|마|만질 마
783|絳|강|붉을 강
784|霄|소|하늘 소
785|耽|탐|즐길 탐
786|讀|독|읽을 독/이두 두
787|翫|완|가지고놀 완
788|市|시|저자 시
789|寓|우|붙일 우
790|目|목|눈 목
791|囊|낭|주머니 낭
792|箱|상|상자 상
793|易|이|쉬울 이/바꿀 역
794|輶|유|가벼울 유
795|攸|유|바 유
796|畏|외|두려워할 외
797|屬|속|붙을 속/이을 촉
798|耳|이|귀 이
799|垣|원|담 원
800|牆|장|담 장
801|具|구|갖출 구
802|膳|선|반찬 선
803|飧|손|밥 손
804|飯|반|밥 반
805|適|적|마침 적
806|口|구|입 구
807|充|충|채울 충
808|腸|장|창자 장
809|飽|포|배부를 포
810|飫|어|배부를 어
811|烹|팽|삶을 팽
812|宰|재|재상 재
813|饑|기|주릴 기
814|厭|염|싫을 염
815|糟|조|재강 조
816|糠|강|겨 강
817|親|친|친할 친
818|戚|척|겨레 척

819|故|고|연고 고
820|舊|구|옛 구
821|老|노|늙을 로
822|少|소|젊을 소
823|異|이|다를 이
824|糧|량|양식 량
825|妾|첩|첩 첩
826|御|어|모실 어
827|績|적|길쌈 적
828|紡|방|길쌈 방
829|侍|시|모실 시
830|巾|건|수건 건
831|帷|유|장막 유
832|房|방|방 방
833|紈|환|흰비단 환
834|扇|선|부채 선
835|圓|원|둥글 원
836|潔|결|깨끗할 결
837|銀|은|은 은
838|燭|촉|촛불 촉
839|煒|위|빛날 위
840|煌|황|빛날 황
841|晝|주|낮 주
842|眠|면|잘 면
843|夕|석|저녁 석
844|寐|매|잘 매
845|藍|남|쪽 람
846|筍|순|죽순 순
847|象|상|코끼리 상
848|床|상|상 상
849|弦|현|줄 현
850|歌|가|노래 가
851|酒|주|술 주
852|讌|연|잔치 연
853|接|접|이을 접
854|杯|배|잔 배
855|擧|거|들 거
856|觴|상|잔 상
857|矯|교|바로잡을 교
858|手|수|손 수
859|頓|돈|두드릴 돈

860|足|족|발 족
861|悅|열|기쁠 열
862|豫|예|미리 예
863|且|차|또 차
864|康|강|편안 강
865|嫡|적|정실 적
866|後|후|뒤 후
867|嗣|사|이을 사
868|續|속|이을 속
869|祭|제|제사 제
870|祀|사|제사 사
871|蒸|증|찔 증
872|嘗|상|맛볼 상
873|稽|계|조아릴 계
874|顙|상|이마 상
875|再|재|둘 재
876|拜|배|절 배
877|悚|송|두려워할 송
878|懼|구|두려워할 구
879|恐|공|두려워할 공
880|惶|황|두려워할 황
881|牋|전|편지 전
882|牒|첩|편지 첩
883|簡|간|편지 간
884|要|요|중요 요
885|顧|고|돌아볼 고
886|答|답|대답 답
887|審|심|살필 심
888|詳|상|자세할 상
889|骸|해|뼈 해
890|垢|구|때 구
891|想|상|생각할 상
892|浴|욕|목욕할 욕
893|執|집|잡을 집
894|熱|열|더울 열
895|願|원|원할 원
896|凉|량|서늘할 량
897|驢|여|나귀 려
898|騾|라|노새 라
899|犢|독|송아지 독
900|特|특|특별 특

901|駭|해|놀랄 해
902|躍|약|뛸 약
903|超|초|넘을 초
904|驤|양|달릴 양
905|誅|주|벨 주
906|斬|참|벨 참
907|賊|적|도적 적
908|盜|도|도적 도
909|捕|포|잡을 포
910|獲|획|얻을 획
911|叛|반|배반할 반
912|亡|망|망할 망/없을 무
913|布|포|베 포
914|射|사|쏠 사
915|僚|료|벗 료
916|丸|환|알 환
917|嵇|혜|산이름 혜
918|琴|금|거문고 금
919|阮|완|악기 완
920|嘯|소|휘파람 소
921|恬|염|편안 념
922|筆|필|붓 필
923|倫|륜|인륜 륜
924|紙|지|종이 지
925|鈞|균|고를 균
926|巧|교|공교할 교
927|任|임|맡길 임
928|釣|조|낚시 조
929|釋|석|놓을 석
930|紛|분|어지러울 분
931|利|리|이로울/날카로울 리
932|俗|속|풍속 속
933|竝|병|아우를 병
934|皆|개|다 개
935|佳|가|아름다울 가
936|妙|묘|묘할 묘
937|毛|모|털 모
938|施|시|베풀 시
939|淑|숙|맑을 숙
940|姿|자|모양 자
941|工|공|장인 공

942|顰|빈|찡그릴 빈
943|姸|연|고울 연
944|笑|소|웃을 소
945|年|연|해 년
946|矢|시|화살 시
947|每|매|매양 매
948|催|최|재촉 최
949|曦|희|햇빛 희
950|暉|휘|빛날 휘
951|朗|낭|밝을 랑
952|耀|요|빛날 요
953|璇|선|구슬 선
954|璣|기|구슬 기
955|懸|현|달 현
956|斡|알|빙빙돌 알
957|晦|회|그믐 회
958|魄|백|넋 백
959|環|환|고리 환
960|照|조|비칠 조
961|指|지|손가락 지
962|薪|신|섶나무 신
963|修|수|닦을 수
964|祐|우|복 우
965|永|영|길 영
966|綏|수|편안 수
967|吉|길|길할 길
968|劭|소|아름다울 소
969|矩|구|법 구
970|步|보|걸음 보
971|引|인|끌 인
972|領|령|거느릴 령
973|俯|부|굽을 부
974|仰|앙|우러를 앙
975|廊|낭|행랑 랑
976|廟|묘|사당 묘
977|束|속|묶을 속
978|帶|대|띠 대
979|矜|긍|자랑 긍
980|莊|장|씩씩할 장
981|徘|배|배회 배
982|徊|회|배회 회

983|瞻|첨|쳐다볼 첨
984|眺|조|바라볼 조
985|孤|고|외로울 고
986|陋|루|더러울 루
987|寡|과|적을 과
988|聞|문|들을 문
989|愚|우|어리석을 우
990|蒙|몽|어릴 몽
991|等|등|등급 등
992|誚|초|꾸짖을 초
993|謂|위|이를 위
994|語|어|말씀 어
995|助|조|도울 조
996|者|자|놈 자
997|焉|언|어찌 언
998|哉|재|어조사 재
999|乎|호|어조사 호
1000|也|야|어조사 야

마지막 편집일시 : 2018년 7월 10일 11:39
---
02. chunja2.txt
1|天地玄黃|천지현황|天(하늘 천)地(땅 지)玄(검을 현)黃(누를 황)|하늘은 위에 있어 그 빛이 검고 땅은 아래 있어서 그 빛이 누르다.
2|宇宙洪荒|우주홍황|宇(집 우)宙(집 주)洪(넓을 홍)荒(거칠 황)|하늘과 땅 사이는 넓고 커서 끝이 없다. 즉 세상의 넓음을 말한다.
3|日月盈昃|일월영측|日(날 일)月(달 월)盈(찰 영)昃(기울 측)|해는 서쪽으로 기울고 달도 차면 점차 이지러진다. 즉 우주의 진리를 말한다.
4|辰宿列張|진숙열장|辰(별 진)宿(잘 숙)列(벌일 열)張(베풀 장)|성좌가 해 달과 같이 하늘에 넓게 벌려져 있음을 말한다.
5|寒來暑往|한래서왕|寒(찰 한)來(올 래)暑(더울 서)往(갈 왕)|찬 것이 오면 더운 것이 가고 더운 것이 오면 찬 것이 간다. 즉 사철의 바뀜을 말한다.
6|秋收冬藏|추수동장|秋(가을 추)收(거둘 수)冬(겨울 동)藏(감출 장)|가을에 곡식을 거두고 겨울이 오면 그것을 감춰 들인다.
7|閏餘成歲|윤여성세|閏(윤달 윤)餘(남을 여)成(이룰 성)歲(해 세)|일년 이십사절기 나머지 시각을 모아 윤달로 하여 해를 이루었다.
8|律呂調陽|율려조양|律(가락 률)呂(음률 려)調(고를 조)陽(볕 양)|천지간의 양기를 고르게 하니 즉 율은 양이요 여는 음이다.
9|雲騰致雨|운등치우|雲(구름 운)騰(오를 등)致(이를 치)雨(비 우)|수증기가 올라가서 구름이 되고 냉기를 만나 비가 된다. 즉 자연의 기상을 말한다.
10|露結爲霜|노결위상|露(이슬 로)結(맺을 결)爲(할 위)霜(서리 상)|이슬이 맺어 서리가 되니

밤기운이 풀잎에 물방울처럼 이슬을 이룬다.
11|金生麗水|금생여수|金(쇠 금)生(낳을 생)麗(고울 려)水(물 수)|금은 여수에서 나니 여수는 중국의 지명이다.
12|玉出崑岡|옥출곤강|玉(구슬 옥)出(날 출)崑(메 곤)岡(메 강)|옥은 곤강에서 나니 곤강은 역시 중국의 산 이름이다.
13|劍號巨闕|검호거궐|劍(칼 검)號(이름 호)巨(클 거)闕(대궐 궐)|거궐은 칼이름이고 구야자가 지은 보검이다. 즉 조나라의 국보다.
14|珠稱夜光|주칭야광|珠(구슬 주)稱(일컬을 칭)夜(밤 야)光(빛 광)|구슬의 빛이 밤의 낮 같은 고로 야광이라 칭하였다.
15|果珍李柰|과진이내|果(과실 과)珍(보배 진)李(오얏 리)柰(능금나무 내)|과실 중에 오얏과 능금나무의 그 진미가 으뜸임을 말한다.
16|菜重芥薑|채중개강|菜(나물 채)重(무거울 중)芥(겨자 개)薑(생강 강)|나물은 겨자와 생강이 중하다.
17|海鹹河淡|해함하담|海(바다 해)鹹(짤 함)河(물 하)淡(묽을 담)|바다 물은 짜고 밀물은 맛도 없고 맑다.
18|鱗潛羽翔|인잠우상|鱗(비늘 린)潛(잠길 잠)羽(깃 우)翔(높이 날 상)|비늘 있는 고기는 물 속에 잠기고 날개 있는 새는 공중에 난다.
19|龍師火帝|용사화제|龍(용 룡)師(스승 사)火(불 화)帝(임금 제)|복희씨는 용으로써 벼슬을 기록하고 신농씨는 불로써 기록하였다.
20|鳥官人皇|조관인황|鳥(새 조)官(벼슬 관)人(사람 인)皇(임금 황)|소호는 새로써 벼슬을 기록하고 황제는 인문을 갖추었으므로 인황이라 하였다.
21|始制文字|시제문자|始(처음 시)制(지을 제)文(글월 문)字(글자 자)|복희의 신하 창힐이라는 사람이 새의 발자취를 보고 글자를 처음 만들었다.
22|乃服衣裳|내복의상|乃(이에 내)服(옷 복)衣(옷 의)裳(치마 상)|이에 의상을 입게 하니 황제가 의관을 지어 등분을 분별하고 위의를 엄숙케 하였다.
23|推位讓國|추위양국|推(밀 추)位(자리 위)讓(사양할 양)國(나라 국)|벼슬을 미루고 나라를 사양하니 제요가 제순에게 전위하였다.
24|有虞陶唐|유우도당|有(있을 유)虞(헤아릴 우)陶(질그릇 도)唐(당나라 당)|유우는 제순이요 도당은 제요이다. 즉 중국 고대 제왕이다.
25|弔民伐罪|조민벌죄|弔(슬퍼할 조)民(백성 민)伐(칠 벌)罪(허물 죄)|불쌍한 백성은 돕고 죄지은 백성은 벌주었다.
26|周發殷湯|주발은탕|周(두루 주)發(필 발)殷(나라이름 은)湯(끓을 탕)|주발은 무왕의 이름이고 은탕은 왕의 칭호이다.
27|坐朝問道|좌조문도|坐(앉을 좌)朝(아침 조)問(물을 문)道(길/말할 도)|좌조는 천하를 통일하여 왕위에 앉은 것이고 문도는 나라 다스리는 법을 말한다.
28|垂拱平章|수공평장|垂(드리울 수)拱(껴안을 공)平(평평할 평)章(글월 장)|밝고 평화스럽게 다스리는 길을 겸손히 생각함을 말한다.
29|愛育黎首|애육여수|愛(사랑 애)育(기를 육)黎(검을 려)首(머리 수)|明君이 천하를 다스림에 衆民을 사랑하고 양육함을 말한다.
30|臣伏戎羌|신복융강|臣(신하 신)伏(엎드릴 복)戎(오랑캐 융)羌(종족이름 강)|이상과 같이 나라를 다스리면 그 덕에 융과 강도 항복하고야 만다.

31|遐邇壹體|하이일체|遐(멀 하)邇(가까울 이)壹(한 일)體(몸 체)|멀고 가까운 나라가 전부 그 덕망에 귀순케 하며 일체가 될 수 있다.
32|率賓歸王|솔빈귀왕|率(거느릴 솔/비율 률)賓(손 빈)歸(돌아갈 귀)王(임금 왕)|거느리고 복종하여 왕에게 돌아오니 덕을 입어 복종치 않음이 없음을 말한다.
33|鳴鳳在樹|명봉재수|鳴(울 명)鳳(봉황새 봉)在(있을 재)樹(나무 수)|명군 성현이 나타나면 봉이 운다는 말과 같이 덕망이 미치는 곳마다 봉이 나무 위에서 울 것이다.
34|白駒食場|백구식장|白(흰 백)駒(망아지 구)食(밥 식)場(마당 장)|평화스러움을 말한 것이며, 즉 흰 망아지도 감화되어 사람을 따르며 마당 풀을 뜯어먹게 한다.
35|化被草木|화피초목|化(될 화)被(입을 피)草(풀 초)木(나무 목)|덕화가 사람이나 짐승에게만 미칠 뿐 아니라 초목에까지도 미침을 말한다.
36|賴及萬方|뇌급만방|賴(힘입을 뢰)及(미칠 급)萬(일만 만)方(모 방)|만방이 극히 넓으나 어진 덕이 고루 미치게 된다.
37|蓋此身髮|개차신발|蓋(덮을 개)此(이 차)身(몸 신)髮(터럭 발)|이 몸의 털은 대개 사람마다 없는 이가 없다.
38|四大五常|사대오상|四(넉 사)大(큰 대)五(다섯 오)常(항상 상)|네 가지 큰 것과 다섯 가지 떳떳함이 있으니 즉 사대는 천지 군부요 오상은 인의예지신이다.
39|恭惟鞠養|공유국양|恭(공손할 공)惟(오직 유)鞠(국문할 국)養(기를 양)|국양함을 공손히 하라. 이 몸은 부모의 기르신 은혜이기 때문이다.
40|豈敢毁傷|기감훼상|豈(어찌 기)敢(감히 감)毁(헐 훼)傷(상할 상)|부모께서 낳아 길러 주신 이 몸을 어찌 감히 훼상할 수 있으랴.
41|女慕貞烈|여모정열|女(계집 녀)慕(사모할 모)貞(곧을 정)烈(매울 렬)|여자는 정조를 굳게 지키고 행실을 단정하게 해야 함을 말한다.
42|男效才良|남효재량|男(사내 남)效(본받을 효)才(재주 재)良(어질 량)|남자는 재능을 닦고 어진 것을 본받아야 함을 말한다.
43|知過必改|지과필개|知(알 지)過(지날/허물 과)必(반드시 필)改(고칠 개)|누구나 허물이 있는 것이니 허물을 알면 즉시 고쳐야 한다.
44|得能莫忘|득능막망|得(얻을 득)能(능할 능)莫(말 막)忘(잊을 망)|사람으로써 알아야 할 것을 배운 후에는 잊지 않도록 노력하여야 한다.
45|罔談彼短|망담피단|罔(없을 망)談(말씀 담)彼(저 피)短(짧을 단)|자기의 단점을 말 안하는 동시에 남의 잘못을 욕하지 말라.
46|靡恃己長|미시기장|靡(아닐 미)恃(믿을 시)己(몸 기)長(길 장)|자신의 특기를 믿고 자랑하지 말라. 그럼으로써 더욱 발달한다.
47|信使可覆|신사가복|信(믿을 신)使(하여금 사)可(옳을 가)覆(뒤집힐 복)|믿음은 움직일 수 없는 진리이고 또한 남과의 약속은 지켜야 한다.
48|器欲難量|기욕난량|器(그릇 기)欲(하고자할 욕)難(어려울 난)量(헤아릴 량)|사람의 기량은 깊고 깊어서 헤아리기 어렵다.
49|墨悲絲染|묵비사염|墨(먹 묵)悲(슬플 비)絲(실 사)染(물들일 염)|흰 실에 검은 물이 들면 다시 희지 못함을 슬퍼한다. 즉 사람도 매사를 조심하여야 한다.
50|詩讚羔羊|시찬고양|詩(시 시)讚(칭찬할 찬)羔(새끼양 고)羊(양 양)|시전 고양편에 문왕의 덕을 입은 남국 대부의 정직함을 칭찬하였으니 사람의 선악을 말한 것이다.
51|景行維賢|경행유현|景(경치 경)行(다닐 행/항렬 항)維(벼리 유)賢(어질 현)|행실을 훌륭하게

하고 당당하게 행하면 어진 사람이 된다는 것을 말한다.
52|克念作聖|극념작성|克(이길 극)念(생각 념)作(지을 작)聖(성인 성)|성인의 언행을 잘 생각하여 수양을 쌓으면 자연 성인이 됨을 말한다.
53|德建名立|덕건명립|德(덕 덕)建(세울 건)名(이름 명)立(설 립)|항상 덕을 가지고 세상일을 행하면 자연 이름도 서게 된다.
54|形端表正|형단표정|形(모양 형)端(바를 단)表(겉 표)正(바를 정)|몸 형상이 단정하고 깨끗하면 마음도 바르며 또 표면에 나타난다.
55|空谷傳聲|공곡전성|空(빌 공)谷(골 곡)傳(전할 전)聲(소리 성)|산골짜기에서 크게 소리치면 그대로 전한다. 즉 악한 일을 당하게 된다.
56|虛堂習聽|허당습청|虛(빌 허)堂(집 당)習(익힐 습)聽(들을 청)|빈방에서 소리를 내면 울려서 다 들린다. 즉 착한 말을 하면 천리 밖에서도 응한다.
57|禍因惡積|화인악적|禍(재앙 화)因(인할 인)惡(악할 악/미워할 오)積(쌓을 적)|재앙은 악을 쌓음에 인한 것이므로 재앙을 받는 이는 평일에 악을 쌓았기 때문이다.
58|福緣善慶|복연선경|福(복 복)緣(인연 연)善(착할 선)慶(경사 경)|복은 착한 일에서 오는 것이니 착한 일을 하면 경사가 온다.
59|尺璧非寶|척벽비보|尺(자 척)璧(구슬 벽)非(아닐 비)寶(보배 보)|한 자 되는 구슬이라고 해서 결코 보배라고는 할 수 없다.
60|寸陰是競|촌음시경|寸(마디 촌)陰(그늘 음)是(옳을/이 시)競(다툴 경)|한 자 되는 구슬보다도 잠깐의 시간이 더욱 귀중하니 시간을 아껴야 한다.
61|資父事君|자부사군|資(자료 자)父(아비 부)事(일/섬길 사)君(임금 군)|아비를 자료로 하여 임금을 섬길지니 아비 섬기는 효도로 임금을 섬겨야 한다.
62|曰嚴與敬|왈엄여경|曰(가로 왈)嚴(엄할 엄)與(더불 여)敬(공경할 경)|임금을 대하는 데는 엄숙함과 공경함이 있어야 한다.
63|孝當竭力|효당갈력|孝(효도 효)當(마땅할 당)竭(다할 갈)力(힘 력)|부모를 섬길 때에는 마땅히 힘을 다하여야 한다.
64|忠則盡命|충즉진명|忠(충성 충)則(곧 즉/법 칙)盡(다할 진)命(목숨 명)|충성함에는 곧 목숨을 다하니 임금을 섬기는 데 몸을 사양해서는 안된다.
65|臨深履薄|임심리박|臨(임할 림)深(깊을 심)履(밟을 리)薄(얇을 박)|깊은 곳에 임하듯 하며 얇은 데를 밟듯이 세심 주의하여야 한다.
66|夙興溫凊|숙흥온청|夙(일찍 숙)興(흥할 흥)溫(따뜻할 온)淸(서늘할 정)|일찍 일어나서 추우면 덥게, 더우면 서늘케 하는 것이 부모 섬기는 절차이다.
67|似蘭斯馨|사난사형|似(같을 사)蘭(난초 란)斯(이 사)馨(향기 형)|난초같이 꽃다우니 군자의 지조를 비유한 것이다.
68|如松之盛|여송지성|如(같을 여)松(소나무 송)之(갈 지)盛(성할 성)|솔 나무같이 푸르러 성함은 군자의 절개를 말한 것이다.
69|川流不息|천류불식|川(내 천)流(흐를 류)不(아니 불)息(쉴 식)|내가 흘러 쉬지 아니하니 군자의 행지를 말한 것이다.
70|淵澄取暎|연징취영|淵(못 연)澄(맑을 징)取(취할 취)暎(비칠 영)|못이 맑아서 비치니 즉 군자의 마음을 말한 것이다.
71|容止若思|용지약사|容(얼굴 용)止(그칠 지)若(같을 약)思(생각 사)|행동을 덤비지 말고 형용과 행지를 조용히 생각하는 침착한 태도를 가져라.

72|言辭安定|언사안정|言(말씀 언)辭(말씀 사)安(편안 안)定(정할 정)|태도만 침착할 뿐 아니라 말도 안정케 하며 쓸데없는 말을 삼가라.
73|篤初誠美|독초성미|篤(도타울 독)初(처음 초)誠(정성 성)美(아름다울 미)|무엇이든지 처음에 성실하고 신중히 하여야 한다.
74|愼終宜令|신종의령|愼(삼갈 신)終(마지막 종)宜(마땅 의)令(하여금 령)|처음뿐만 아니라 끝맺음도 좋아야 한다.
75|榮業所基|영업소기|榮(영화 영)業(업 업)所(바 소)基(터 기)|이상과 같이 잘 지키면 번성하는 기본이 된다.
76|籍甚無竟|적심무경|籍(호적 적)甚(심할 심)無(없을 무)竟(마침내 경)|뿐만 아니라 자신의 명예스러운 이름이 길이 전하여질 것이다.
77|學優登仕|학우등사|學(배울 학)優(넉넉할 우)登(오를 등)仕(벼슬 사)|배운 것이 넉넉하면 벼슬에 오를 수 있다.
78|攝職從政|섭직종정|攝(잡을 섭)職(벼슬 직)從(좇을 종)政(정사 정)|벼슬을 잡아 정사를 좇으니 국가 정사에 종사하니라.
79|存以甘棠|존이감당|存(있을 존)以(써 이)甘(달 감)棠(해당화 당)|주나라 소공이 남국의 아가위나무 아래에서 백성을 교화하였다.
80|去而益詠|거이익영|去(갈 거)而(어조사 이)益(더할 익)詠(읊을 영)|소공이 죽은 후 남국의 백성이 그의 덕을 추모하여 감당시를 읊었다.
81|樂殊貴賤|악수귀천|樂(풍류 악/즐길 락/좋아할 요)殊(다를 수)貴(귀할 귀)賤(천할 천)|풍류는 귀천이 다르니 천자는 팔일 제후는 육일 사대부는 사일 선일은 이일이다.
82|禮別尊卑|예별존비|禮(예도 례)別(다를 별)尊(높을 존)卑(낮을 비)|예도에 존비의 분별이 있으니 군신, 부자, 부부, 장유, 붕우의 차별이 있다.
83|上和下睦|상화하목|上(위 상)和(화할 화)下(아래 하)睦(화목할 목)|위에서 사랑하고 아래에서 공경함으로써 화목이 된다.
84|夫唱婦隨|부창부수|夫(지아비 부)唱(부를 창)婦(며느리 부)隨(따를 수)|지아비가 부르면 지어미가 따른다. 즉 원만한 가정을 말한다.
85|外受傅訓|외수부훈|外(밖 외)受(받을 수)傅(스승 부)訓(가르칠 훈)|팔세면 바깥 스승의 가르침을 받아야 한다.
86|入奉母儀|입봉모의|入(들 입)奉(받들 봉)母(어미 모)儀(거동 의)|집에 들어서는 어머니를 받들어 종사하라.
87|諸姑伯叔|제고백숙|諸(모두 제)姑(시어미 고)伯(맏 백)叔(아재비 숙)|고모, 백부, 숙부 등 집안 내의 친척 등을 말한다.
88|猶子比兒|유자비아|猶(같을 유)子(아들 자)比(견줄 비)兒(아이 아)|조카들도 자기의 아들과 같이 취급하여야 한다.
89|孔懷兄弟|공회형제|孔(구멍 공)懷(품을 회)兄(맏 형)弟(아우 제)|형제는 서로 사랑하여 의좋게 지내야 한다.
90|同氣連枝|동기연지|同(한가지 동)氣(기운 기)連(이어질 연)枝(가지 지)|형제는 부모의 기운을 같이 받았으니 나무의 가지와 같다.
91|交友投分|교우투분|交(사귈 교)友(벗 우)投(던질 투)分(나눌 분)|벗을 사귈 때에는 서로가 분에 맞는 사람끼리 사귀어야 한다.
92|切磨箴規|절마잠규|切(끊을 절/모두 체)磨(갈 마)箴(경계 잠)規(법 규)|열심히 닦고 배워서

사람으로서의 도리를 지켜야 한다.
93|仁慈隱惻|인자은측|仁(어질 인)慈(사랑할 자)隱(숨을 은)惻(슬플 측)|어진 마음으로 남을 사랑하고 또는 이를 측은히 여겨야 한다.
94|造次弗離|조차불리|造(지을 조)次(버금 차)弗(아닐 불)離(떠날 리)|남을 위한 동정심을 잠시라도 잊지 말고 항상 가져야 한다.
95|節義廉退|절의염퇴|節(마디 절)義(옳을 의)廉(청렴 렴)退(물러갈 퇴)|청렴과 절개와 의리와 사양함과 물러감은 늘 지켜야 한다.
96|顚沛匪虧|전패비휴|顚(엎드러질 전)沛(자빠질 패)匪(아닐 비)虧(이지러질 휴)|엎드려지고 자빠져도 이지러지지 않으니 용기를 잃지 말라.
97|性靜情逸|성정정일|性(성품 성)靜(고요할 정)情(뜻 정)逸(편안할 일)|성품이 고요하면 뜻이 편안하니 고요함은 천성이요 동작함은 인정이다.
98|心動神疲|심동신피|心(마음 심)動(움직일 동)神(귀신 신)疲(피곤할 피)|마음이 움직이면 신기가 피곤하니 마음이 불안하면 신기가 불편하다.
99|守眞志滿|수진지만|守(지킬 수)眞(참 진)志(뜻 지)滿(찰 만)|사람의 도리를 지키면 뜻이 차고 군자의 도를 지키면 뜻이 편안하다.
100|逐物意移|축물의이|逐(쫓을 축)物(만물 물)意(뜻 의)移(옮길 이)|마음이 불안함은 욕심이 있어서 그렇다. 너무 욕심내면 마음도 변한다.
101|堅持雅操|견지아조|堅(굳을 견)持(가질 지)雅(우아할 아)操(잡을 조)|맑은 절조를 굳게 가지고 있으면 나의 도리를 극진히 함이라.
102|好爵自縻|호작자미|好(좋을 호)爵(벼슬 작)自(스스로 자)縻(얽을 미)|스스로 벼슬을 얻게 되니 찬작을 극진하면 인작이 스스로 이르게 된다.
103|都邑華夏|도읍화하|都(도읍 도)邑(고을 읍)華(빛날 화)夏(여름 하)|도읍은 왕성의 지위를 말한 것이고 화하는 당시 중국을 지칭하던 말이다.
104|東西二京|동서이경|東(동녘 동)西(서녘 서)二(두 이)京(서울 경)|동과 서에 두 서울이 있으니 동경은 낙양이고 서경은 장안이다.
105|背邙面洛|배망면낙|背(등 배)邙(산이름 망)面(낯 면)洛(강이름 락)|동경은 북에 북망산이 있고 낙양은 남에 낙천이 있다.
106|浮渭據涇|부위거경|浮(뜰 부)渭(강이름 위)據(의거할 거)涇(통할 경)|위수에 뜨고 경수를 눌렀으니 장안은 서북에 위천, 경수, 두물이 있었다.
107|宮殿盤鬱|궁전반울|宮(집 궁)殿(큰집 전)盤(서릴 반)鬱(답답 울)|궁전은 울창한 나무 사이에 서린 듯 정하고
108|樓觀飛驚|누관비경|樓(다락 루)觀(볼 관)飛(날 비)驚(놀랄 경)|궁전 가운데 있는 물견대는 높아서 올라가면 나는 듯하여 놀란다.
109|圖寫禽獸|도사금수|圖(그림 도)寫(베낄 사)禽(날짐승 금)獸(짐승 수)|궁전 내부에는 유명한 화가들이 그린 그림 조각 등으로 장식되어 있다.
110|畵綵仙靈|화채선령|畵(그림 화)采(채색 채)仙(신선 선)靈(신령 령)|신선과 신령의 그림도 화려하게 채색되어 있다.
111|丙舍傍啓|병사방계|丙(남녘 병)舍(집 사)傍(곁 방)啓(열 계)|병사 곁에 통고를 열어 궁전 내를 출입하는 사람들의 편리를 도모하였다.
112|甲帳對楹|갑장대영|甲(갑옷 갑)帳(휘장 장)對(대답할 대)楹(기둥 영)|아름다운 갑장이 기둥을 대하였으니 동방 삭이 갑장을 지어 임금이 잠시 정지하는 곳이다.

113|肆筵設席|사연설석|肆(베풀 사)筵(자리 연)設(베풀 설)席(자리 석)|자리를 베풀고 돗자리를 베푸니 연회하는 좌석이다.
114|鼓瑟吹笙|고슬취생|鼓(북 고)瑟(비파 슬)吹(불 취)笙(생황 생)|비파를 치고 저를 부니 잔치하는 풍류이다.
115|陞階納陛|승계납폐|陞(오른쪽 승)階(뜰 계)納(바칠 납)陛(섬돌 폐)|문무백관이 계단을 올라 임금께 납폐하는 절차이니라.
116|弁轉疑星|변전의성|弁(고깔 변)轉(구를 전)疑(의심할 의)星(별 성)|많은 사람들의 관에서 번쩍이는 구슬이 별안간 의심할 정도이다.
117|右通廣內|우통광내|右(오를 우)通(통할 통)廣(넓을 광)內(안 내)|오른편에 광내가 통하니 광내는 나라 비서를 두는 집이다.
118|左達承明|좌달승명|左(왼 좌)達(통달할 달)承(이을 승)明(밝을 명)|왼편에 승명이 사무치니 승명은 사기를 교열하는 집이다.
119|既集墳典|기집분전|既(이미 기)集(모을 집)墳(무덤 분)典(법 전)|이미 분과 전을 모았으니 삼황의 글은 삼분이요 오제의 글은 오전이다.
120|亦聚群英|역취군영|亦(또 역)聚(모을 취)群(무리 군)英(꽃부리 영)|또한 여러 영웅을 모으니 분전을 강론하여 치국하는 도를 밝힘이라.
121|杜藁鍾隷|두고종례|杜(막을 두)稿(볏짚 고)鍾(쇠북 종)隷(글씨 례)|초서를 처음으로 쓴 두고와 예서를 쓴 종례의 글로 비치되었다.
122|漆書壁經|칠서벽경|漆(옻칠할 칠)書(글씨 서)壁(벽 벽)經(날 경)|하나라 영제가 돌벽에서 발견한 서골과 공자가 발견한 육경도 비치되어 있다.
123|府羅將相|부라장상|府(마을 부)羅(벌릴 라)將(장수 장)相(서로 상)|마을 좌우에 장수와 정승이 벌려 있었다.
124|路俠槐卿|노협괴경|路(길 로)夾(낄 협)槐(괴화나무 괴)卿(벼슬 경)|길에 고위 고관인 삼공구경의 마차가 열지어 궁전으로 들어가는 모습이다.
125|戶封八縣|호봉팔현|戶(지게 호)封(봉할 봉)八(여덟 팔)縣(고을 현)|한나라가 천하를 통일하고 여덟 고을 민호를 주어 공신을 봉하였다.
126|家給千兵|가급천병|家(집 가)給(줄 급)千(일천 천)兵(군사 병)|제후 나라에 일천 군사를 주어 그의 집을 호위시켰다.
127|高冠陪輦|고관배련|高(높을 고)冠(갓 관)陪(더할 배)輦(손수레 련)|높은 관을 쓰고 연을 모시니 제후의 예로 대접했다.
128|驅轂振纓|구곡진영|驅(몰 구)轂(바퀴 곡)振(떨친 진)纓(끈 영)|수레를 몰며 갓끈이 떨치니 임금출행에 제후의 위엄이 있다
129|世祿侈富|세록치부|世(세상 세)祿(녹 록)侈(사치할 치)富(부자 부)|대대로 녹이 사치하고 부하니 제후 자손이 세세 관록이 무성하여라.
130|車駕肥輕|거가비경|車(수레 거)駕(멍에 가)肥(살찔 비)輕(가벼울 경)|수레의 말은 살찌고 몸의 의복은 가볍게 차려져 있다.
131|策功茂實|책공무실|策(꾀 책)功(공 공)茂(무성할 무)實(열매 실)|공을 꾀함에 무성하고 충실하러라.
132|勒碑刻銘|늑비각명|勒(굴레 륵)碑(비석 비)刻(새길 각)銘(새길 명)|비를 세워 이름을 새겨서 그 공을 찬양하며 후세에 전하였다.
133|磻溪伊尹|반계이윤|磻(강이름 반)溪(시내 계)伊(저 이)尹(다스릴 윤)|문왕은 반계에서

강태공을 맞고 은왕은 신야에서 이윤을 맞이하였다.
134|佐時阿衡|좌시아형|佐(도울 좌)時(때 시)阿(언덕 아)衡(저울대 형)|때를 돕는 아형이니 아형은 상나라 재상의 칭호이다.
135|奄宅曲阜|엄택곡부|奄(문득 엄)宅(집 댁/택)曲(굽을 곡)阜(언덕 부)|주공이 큰 공이 있는 고로 노국을 봉한 후 곡부에다 궁전을 세웠다.
136|微旦孰營|미단숙영|微(작을 미)旦(아침 단)孰(누구 숙)營(경영 영)|주공의 단이 아니면 어찌 큰 궁전을 세웠으리요.
137|桓公匡合|환공광합|桓(굳셀 환)公(공변될 공)匡(바를 광)合(모을 합)|제나라 환공은 바르게 하고 모두었으니 초를 물리치고 난을 바로잡았다.
138|濟弱扶傾|제약부경|濟(건널 제)弱(약할 약)扶(도울 부)傾(기울 경)|약한 나라를 구제하고 기울어지는 제신을 도와서 붙들어 주었다.
139|綺回漢惠|기회한혜|綺(비단 기)回(돌아올 회)漢(한수 한)惠(은혜 혜)|하나라 네 현인의 한 사람인 기가 한나라 혜제를 회복시켰다.
140|兌感武丁|열감무정|兌(말씀 설/달랠 세/기뻐할 열)感(느낄 감)武(호반 무)丁(고무래 정)|부열이 들에서 역사하매 무정의 꿈에 감동되어 곧 정승에 되었다.
141|俊乂密勿|준예밀물|俊(준걸 준)乂(어질 예)密(빽빽할 밀)勿(말 물)|준걸과 재사가 조정에 모여 빽빽하더라.
142|多士寔寧|다사식녕|多(많을 다)士(선비 사)寔(이 식)寧(편안 녕)|준걸과 재사가 조정에 많으니 국가가 태평함이라.
143|晋楚更霸|진초갱패|晋(나라 진)楚(나라 초)更(다시 갱/고칠 경)霸(으뜸 패)|진과 초가 다시 으뜸이 되니 진문공 초장왕이 패왕이 되니라.
144|趙魏困橫|조위곤횡|趙(나라 조)魏(나라 위)困(곤할 곤)橫(비낄 횡)|조와 위는 횡에 곤하니 육군때에 진나라를 섬기자 함을 횡이라 하니라.
145|假途滅虢|가도멸괵|假(거짓 가)途(길 도)滅(멸할 멸)虢(나라 괵)|길을 빌려 괵국을 멸하니 진헌공이 우국길을 빌려 괵국을 멸하였다.
146|踐土會盟|천토회맹|踐(밟을 천)土(흙 토)會(모일 회)盟(맹세 맹)|진문공이 제후를 천토에 모아 맹세하고 협천자영 제후하니라.
147|何遵約法|하준약법|何(어찌 하)遵(좇을 준)約(약속할 약)法(법 법)|소하는 한고조로 더불어 약법삼장을 정하여 준행하리라.
148|韓弊煩刑|한폐번형|韓(나라 한)弊(해질 폐)煩(번거로울 번)刑(형벌 형)|한비는 진왕을 달래 형벌을 펴다가 그 형벌에 죽는다.
149|起翦頗牧|기전파목|起(일어날 기)翦(자를 전)頗(자못 파)牧(칠 목)|백기와 왕전은 진나라 장수요 염파와 이목은 조나라 장수였다.
150|用軍最精|용군최정|用(쓸 용)軍(군사 군)最(가장 최)精(정할 정)|군사 쓰기를 가장 정결히 하였다
151|宣威沙漠|선위사막|宣(베풀 선)威(위엄 위)沙(모래 사)漠(아득할 막)|장수로서 그 위엄은 멀리 사막에까지 퍼졌다.
152|馳譽丹靑|치예단청|馳(달릴 치)譽(칭찬할 예)丹(붉을 단)靑(푸를 청)|그 이름은 생전뿐 아니라 죽은 후에도 전하기 위하여 초상을 기린각에 그렸다.
153|九州禹跡|구주우적|九(아홉 구)州(고을 주)禹(하우씨 우)跡(자취 적)|하우씨가 구주를 분별하니 기, 연, 청, 서, 양, 옹, 구주이다.

154|百郡秦幷|백군진병|百(일백 백)郡(고을 군)秦(나라 진)幷(아우를 병)|진시황이 천하봉군하는 법을 폐하고 일백군을 두었다.
155|嶽宗恒岱|악종항대|嶽(산마루 악)宗(마루 종)恒(항상 항)岱(뫼 대)|오악은 동태산, 서화산, 남형산, 북항산, 중숭산이니 항산과 태산이 조종이라.
156|禪主云亭|선주운정|禪(터닦을 선)主(임금 주)云(이를 운)亭(정자 정)|운과 정은 천자를 봉선하고 제사하는 곳이니 운정은 태산에 있다.
157|雁門紫塞|안문자새|雁(기러기 안)門(문 문)紫(붉을 자)塞(변방 새)|안문은 봄기러기 북으로 가는 고로 안문이고 흙이 붉은 고로 자색이라 하였다.
158|鷄田赤城|계전적성|鷄(닭 계)田(밭 전)赤(붉을 적)城(성 성)|계전은 옹주에 있는 고을이고 적성은 기주에 있는 고을이다.
159|昆池碣石|곤지갈석|昆(맏 곤)池(못 지)碣(돌 갈)石(돌 석)|곤지는 운남 곤명현에 있고 갈석은 부평현에 있다.
160|鉅野洞庭|거야동정|鉅(클 거)野(들 야)洞(골 동/꿰뚫을 통)庭(뜰 정)|거야는 태산 동편에 있는 광야 동전은 호남성에 있는 중국 제일의 호수이다.
161|曠遠綿邈|광원면막|曠(빌 광)遠(멀 원)綿(이어질 면)邈(멀 막)|산, 벌판, 호수 등이 아득하고 멀리 그리고 널리 줄지어 있음을 말한다.
162|巖峀杳冥|암수묘명|巖(바위 암)峀(메뿌리 수)杳(아득할 묘)冥(어두울 명)|큰 바위와 메뿌리가 묘연하고 아득함을 말한다.
163|治本於農|치본어농|治(다스릴 치)本(근본 본)於(어조사 어)農(농사 농)|다스리는 것은 농사를 근본으로 하니 중농 정치를 이른다.
164|務玆稼穡|무자가색|務(힘쓸 무)玆(이 자)稼(심을 가)穡(거둘 색)|때맞춰 심고 힘써 일하며 많은 수익을 거둔다.
165|俶載南畝|숙재남묘|俶(비로소 숙)載(실을 재)南(남녘 남)畝(이랑 묘)|비로소 남양의 밭에서 농작물을 배양한다.
166|我藝黍稷|아예서직|我(나 아)藝(재주 예)黍(기장 서)稷(피 직)|나는 기장과 피를 심는 일에 열중하겠다.
167|稅熟貢新|세숙공신|稅(징수할 세)熟(익을 숙)貢(바칠 공)新(새 신)|곡식이 익으면 부세하여 국용을 준비하고 신곡으로 종묘에 제사를 올린다.
168|勸賞黜陟|권상출척|勸(권할 권)賞(상줄 상)黜(물리칠 출)陟(오를 척)|농민의 의기를 앙양키 위하여 열심인 자는 상주고 게을리한 자는 출석하였다.
169|孟軻敦素|맹가돈소|孟(맏 맹)軻(수레 가)敦(도타울 돈)素(흴 소)|맹자는 그 모친의 교훈을 받아 자사문하에서 배웠다.
170|史魚秉直|사어병직|史(역사 사)魚(물고기 어)秉(잡을 병)直(곧을 직)|사어라는 사람은 위나라 태부였으며 그 성격이 매우 강직하였다.
171|庶幾中庸|서기중용|庶(여러 서)幾(몇 기)中(가운데 중)庸(떳떳 용)|어떠한 일이나 한쪽으로 기울어지게 일하면 안 된다.
172|勞謙謹勅|노겸근칙|勞(힘쓸 로)謙(겸손 겸)謹(삼갈 근)勅(칙서 칙)|근로하고 겸손하며 삼가고 신칙하면 중용의 도에 이른다.
173|聆音察理|영음찰리|聆(들을 령)音(소리 음)察(살필 찰)理(다스릴 리)|소리를 듣고 그 거동을 살피니 조그마한 일이라도 주의하여야 한다.
174|鑑貌辨色|감모변색|鑑(거울 감)貌(모양 모)辨(분별 변)色(빛 색)|모양과 거동으로 그

마음속을 분별할 수 있다.
175|貽厥嘉猷|이궐가유|貽(끼칠 이)厥(그 궐)嘉(아름다울 가)猷(꾀 유)|도리를 지키고 착함으로 자손에 좋은 것을 끼쳐야 한다.
176|勉其祗植|면기지식|勉(힘쓸 면)其(그 기)祗(공경 지)植(심을 식)|착한 것으로 자손에 줄 것을 힘써야 좋은 가정을 이룰 것이다.
177|省躬譏誡|생궁기계|省(살필 성/덜 생)躬(몸 궁)譏(나무랄 기)誡(경계 계)|나무람과 경계함이 있는가 염려하며 몸을 살피라.
178|寵增抗極|총증항극|寵(고일 총)增(더할 증)抗(저항할 항)極(다할 극)|총애가 더할수록 교만한 태도를 부리지 말고 더욱 조심하여야 한다.
179|殆辱近恥|태욕근치|殆(위태 태)辱(욕할 욕)近(가까울 근)恥(부끄러울 치)|총애를 받는다고 욕된 일을 하면 머지 않아 위태함과 치욕이 온다.
180|林皐幸卽|임고행즉|林(수풀 림)皐(언덕 고)幸(다행 행)卽(곧 즉)|부귀할지라도 겸토하여 산간 수풀에서 편히 지내는 것도 다행한 일이다.
181|兩疏見機|양소견기|兩(두 량)疏(상소할 소)見(볼 견/나타날 현)機(틀 기)|한나라의 소광과 소수는 기틀을 보고 상소하고 낙향했다.
182|解組誰逼|해조수핍|解(풀 해)組(짤 조)誰(누구 수)逼(핍박할 핍)|관의 끈을 풀어 사직하고 돌아가니 누가 핍박하리요.
183|索居閑處|삭거한처|索(찾을 색)居(살 거)閑(한가 한)處(곳 처)|퇴직하여 한가한 곳에서 세상을 보냈다.
184|沈默寂寥|침묵적요|沈(잠길 침)默(잠잠할 묵)寂(고요할 적)寥(고요 요)|세상에 나와서 교제하는 데도 언행에 침착해야 한다.
185|求古尋論|구고심론|求(구할 구)古(옛 고)尋(찾을 심)論(의논할 론)|예를 찾아 의논하고 고인을 찾아 토론한다.
186|散慮逍遙|산려소요|散(흩을 산)慮(생각 려)逍(거닐 소)遙(멀 요)|세상일을 잊어버리고 자연 속에서 한가하게 즐긴다.
187|欣奏累遣|흔주누견|欣(기쁠 흔)奏(아뢸 주)累(여러 루)遣(보낼 견)|기쁨은 아뢰고 더러움은 보내니.
188|慼謝歡招|척사환초|慼(슬플 척)謝(사례 사)歡(기뻐할 환)招(부를 초)|심중의 슬픈 것은 없어지고 즐거움만 부른 듯이 오게 된다.
189|渠荷的歷|거하적력|渠(개천 거)荷(연꽃 하)的(과녁 적)歷(지낼 력)|개천의 연꽃도 아름다우니 향기를 잡아볼 만하다.
190|園莽抽條|원망추조|園(동산 원)莽(풀 망)抽(빼낼 추)條(조목 조)|동산의 풀은 땅속 양분으로 가지가 뻗고 크게 자란다.
191|枇杷晩翠|비파만취|枇(비파나무 비)杷(비파나무 파)晩(늦을 만)翠(푸를 취)|비파나무는 늦은 겨울에도 그 빛은 푸르다.
192|梧桐早凋|오동조조|梧(오동 오)桐(오동 동)早(이를 조)凋(시들 조)|오동잎은 가을이면 다른 나무보다 먼저 마른다.
193|陳根委翳|진근위예|陳(베풀 진)根(뿌리 근)委(맡길 위)翳(가릴 예)|가을이 오면 오동뿐 아니라 고목의 뿌리는 시들어 마른다.
194|落葉飄颻|낙엽표요|落(떨어질 락)葉(잎사귀 엽)飄(나부낄 표)颻(나부낄 요)|가을이 오면 낙엽이 펄펄 날리며 떨어진다.

195|遊鵾獨運|유곤독운|游(헤엄칠 유)鯤(곤새 곤)獨(홀로 독)運(운전 운)|곤새가 자유로이 홀로 날개를 펴고 運回하고 있다.
196|凌摩絳霄|능마강소|凌(업신여길 릉)摩(만질 마)絳(붉을 강)霄(하늘 소)|적색의 大空을 업신여기는 듯이 선회하고 있다.
197|耽讀翫市|탐독완시|耽(즐길 탐)讀(읽을 독/이두 두)翫(가지고놀 완)市(저자 시)|하나라의 왕총은 독서를 즐겨 서점에 가서 탐독하였다.
198|寓目囊箱|우목낭상|寓(붙일 우)目(눈 목)囊(주머니 낭)箱(상자 상)|왕총이 한번 읽으면 잊지 아니하여 글을 주머니나 상자에 둠과 같다고 하였다.
199|易輶攸畏|이유유외|易(쉬울 이/바꿀 역)輶(가벼울 유)攸(바 유)畏(두려워할 외)|매사를 소홀히 하고 경솔함은 군자가 진실로 두려워하는 바이다.
200|屬耳垣牆|속이원장|屬(붙을 속/이을 촉)耳(귀 이)垣(담 원)牆(담 장)|담장에도 귀가 있다는 말과 같이 경솔히 말하는 것을 조심하라.
201|具膳飧飯|구선손반|具(갖출 구)膳(반찬 선)飧(밥 손)飯(밥 반)|반찬을 갖추고 밥을 먹으니
202|適口充腸|적구충장|適(마침 적)口(입 구)充(채울 충)腸(창자 장)|훌륭한 음식이 아니라도 입에 맞으면 배를 채운다.
203|飽飫烹宰|포어팽재|飽(배부를 포)飫(배부를 어)烹(삶을 팽)宰(재상 재)|배부를 때에는 아무리 좋은 음식이라도 그 맛을 모른다.
204|饑厭糟糠|기염조강|饑(주릴 기)厭(싫을 염)糟(재강 조)糠(겨 강)|반대로 배가 고플 때에는 겨와 재강도 맛있게 되는 것이다.
205|親戚故舊|친척고구|親(친할 친)戚(겨레 척)故(연고 고)舊(옛 구)|친은 동성지친이고 척은 이성지친이요 고구는 오랜 친구를 말한다.
206|老少異糧|노소이량|老(늙을 로)少(젊을 소)異(다를 이)糧(양식 량)|늙은이와 젊은이의 식사가 다르다.
207|妾御績紡|첩어적방|妾(첩 첩)御(모실 어)績(길쌈 적)紡(길쌈 방)|남자는 밖에서 일하고 여자는 안에서 길쌈을 짜니라.
208|侍巾帷房|시건유방|侍(모실 시)巾(수건 건)帷(장막 유)房(방 방)|유방에서 모시고 수건을 받드니 처첩이 하는 일이다.
209|紈扇圓潔|환선원결|紈(흰비단 환)扇(부채 선)圓(둥글 원)潔(깨끗할 결)|흰 비단으로 만든 부채는 둥글고 깨끗하다.
210|銀燭煒煌|은촉위황|銀(은 은)燭(촛불 촉)煒(빛날 위)煌(빛날 황)|은촛대의 촛불은 빛나서 휘황 찬란하다.
211|晝眠夕寐|주면석매|晝(낮 주)眠(잘 면)夕(저녁 석)寐(잘 매)|낮에 낮잠 자고 밤에 일찍 자니 한가한 사람의 일이다.
212|藍筍象床|남순상상|藍(쪽 람)筍(죽순 순)象(코끼리 상)牀(상 상)|푸른 대순과 코끼리 상이니 즉 한가한 사람의 침대이다.
213|絃歌酒讌|현가주연|弦(줄 현)歌(노래 가)酒(술 주)讌(잔치 연)|거문고를 타며 술과 노래로 잔치하니.
214|接杯擧觴|접배거상|接(이을 접)杯(잔 배)擧(들 거)觴(잔 상)|작고 큰 술잔을 서로 주고받으며 즐기는 모습이다.
215|矯手頓足|교수돈족|矯(바로잡을 교)手(손 수)頓(두드릴 돈)足(발 족)|손을 들고 발을 두드리며 춤을 춘다.

216|悅豫且康|열예차강|悅(기쁠 열)豫(미리 예)且(또 차)康(편안 강)|이상과 같이 마음 편히 즐기고 살면 단란한 가정이다.
217|嫡後嗣續|적후사속|嫡(정실 적)後(뒤 후)嗣(이을 사)續(이을 속)|적자된 자, 즉 장남은 뒤를 계승하여 대를 이룬다.
218|祭祀蒸嘗|제사증상|祭(제사 제)祀(제사 사)蒸(찔 증)嘗(맛볼 상)|제사하되 겨울 제사는 증이라 하고 가을 제사는 상이라 한다.
219|稽顙再拜|계상재배|稽(조아릴 계)顙(이마 상)再(둘 재)拜(절 배)|이마를 조아려 선조에게 두 번 절한다.
220|悚懼恐惶|송구공황|悚(두려워할 송)懼(두려워할 구)恐(두려워할 공)惶(두려워할 황)|송구하고 공황하니 엄중, 공경함이 지극함이라.(3년상 이후의 제사시의 몸가짐이다.)
221|牋牒簡要|전첩간요|牋(편지 전)牒(편지 첩)簡(편지 간)要(중요 요)|글과 편지는 간략함을 요한다.
222|顧答審詳|고답심상|顧(돌아볼 고)答(대답 답)審(살필 심)詳(자세할 상)|편지의 회답도 자세히 살펴 써야 한다.
223|骸垢想浴|해구상욕|骸(뼈 해)垢(때 구)想(생각할 상)浴(목욕할 욕)|몸에 때가 끼면 목욕하기를 생각하고.
224|執熱願涼|집열원량|執(잡을 집)熱(더울 열)願(원할 원)涼(서늘할 량)|더우면 서늘하기를 원한다.
225|驢騾犢特|여라독특|驢(나귀 려)騾(노새 라)犢(송아지 독)特(특별 특)|나귀와 노새와 송아지, 즉 가축을 말한다.
226|駭躍超驤|해약초양|駭(놀랄 해)躍(뛸 약)超(넘을 초)驤(달릴 양)|뛰고 달리며 노는 가축의 모습을 말한다.
227|誅斬賊盜|주참적도|誅(벨 주)斬(벨 참)賊(도적 적)盜(도적 도)|역적과 도적을 베어 물리침.
228|捕獲叛亡|포획반망|捕(잡을 포)獲(얻을 획)叛(배반할 반)亡(망할 망/없을 무)|배반하고 도망하는 자를 잡아 죄를 다스린다.
229|布射僚丸|포사료환|布(베 포)射(쏠 사)僚(벗 료)丸(알 환)|한나라 여포는 화살을 잘 쐈고 의료는 탄자를 잘 던졌다.
230|嵇琴阮嘯|혜금완소|嵇(산이름 혜)琴(거문고 금)阮(악기 완)嘯(휘파람 소)|위국 혜강은 거문고를 잘 타고 완적은 휘파람을 잘 불었다.
231|恬筆倫紙|염필륜지|恬(편안 념)筆(붓 필)倫(인륜 륜)紙(종이 지)|진국 봉념은 토끼털로 처음 붓을 만들었고 후한 채윤은 처음 종이를 만들었다.
232|鈞巧任釣|균교임조|鈞(고를 균)巧(공교할 교)任(맡길 임)釣(낚시 조)|위국 마균은 지남거를 만들고 전국시대 임공자는 낚시를 만들었다.
233|釋紛利俗|석분리속|釋(놓을 석)紛(어지러울 분)利(이로울/날카로울 리)俗(풍속 속)|이상 팔인의 재주를 다하여 어지러움을 풀어 풍속에 이롭게 하였다.
234|竝皆佳妙|병개가묘|竝(아우를 병)皆(다 개)佳(아름다울 가)妙(묘할 묘)|모두가 아름다우며 묘한 재주였다.
235|毛施淑姿|모시숙자|毛(털 모)施(베풀 시)淑(맑을 숙)姿(모양 자)|모는 오의 모타라는 여자이고 시는 월의 시라는 여자인데 모두 절세 미인이었다.
236|工嚬妍笑|공빈연소|工(장인 공)嚬(찡그릴 빈)妍(고울 연)笑(웃을 소)|이 두 미인의 웃는 모습이 매우 곱고 아름다웠다.

237|年矢每催|연시매최|年(해 년)矢(화살 시)每(매양 매)催(재촉 최)|세월이 빠른 것을 말한다. 즉 살같이 매양 재촉하니
238|曦暉朗耀|희휘낭요|曦(햇빛 희)暉(빛날 휘)朗(밝을 랑)耀(빛날 요)|태양 빛과 달빛은 온 세상을 비추어 만물에 혜택을 주고 있다.
239|璇璣懸斡|선기현알|璇(구슬 선)璣(구슬 기)懸(달 현)斡(빙빙돌 알)|선기는 천기를 보는 기구이고 그 기구가 높이 걸려 도는 것을 말한다.
240|晦魄環照|회백환조|晦(그믐 회)魄(넋 백)環(고리 환)照(비칠 조)|달이 고리와 같이 돌며 천지를 비치는 것을 말한다.
241|指薪修祐|지신수우|指(손가락 지)薪(섶나무 신)修(닦을 수)祐(복 우)|불타는 나무와 같이 정열로 도리를 닦으면 복을 얻는다.
242|永綏吉邵|영수길소|永(길 영)綏(편안 수)吉(길할 길)邵(아름다울 소)|그리고 영구히 편안하고 길함이 높으리라.
243|矩步引領|구보인령|矩(법 구)步(걸음 보)引(끌 인)領(거느릴 령)|걸음을 바로 걷고 따라서 얼굴도 바르니 위의가 당당하다.
244|俯仰廊廟|부앙낭묘|俯(굽을 부)仰(우러를 앙)廊(행랑 랑)廟(사당 묘)|항상 남묘에 있는 것으로 생각하고 머리를 숙여 예의를 지키라.
245|束帶矜莊|속대긍장|束(묶을 속)帶(띠 대)矜(자랑 긍)莊(씩씩할 장)|의복에 주의하여 단정히 함으로써 긍지를 갖는다.
246|徘徊瞻眺|배회첨조|徘(배회 배)徊(배회 회)瞻(쳐다볼 첨)眺(바라볼 조)|같은 장소를 배회하며 선후를 보는 모양이다.
247|孤陋寡聞|고루과문|孤(외로울 고)陋(더러울 루)寡(적을 과)聞(들을 문)|하등의 식견도 재능도 없다.(천자문의 저자가 자기 자신을 겸손해서 말한 것이다.)
248|愚蒙等誚|우몽등초|愚(어리석을 우)蒙(어릴 몽)等(등급 등)誚(꾸짖을 초)|적고 어리석어 몽매함을 면치 못한다는 것을 말한다.
249|謂語助者|위어조자|謂(이를 위)語(말씀 어)助(도울 조)者(놈 자)|어조라 함은 한문의 조사, 즉 다음 글자이다.
250|焉哉乎也|언재호야|焉(어찌 언)哉(어조사 재)乎(어조사 호)也(어조사 야)|'언재호야' 이 네 글자는 어조사이다.

마지막 편집일시 : 2018년 7월 10일 11:39

---

03. national_anthem.txt

동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리나라 만세

무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세

남산 위에 저 소나무 철갑을 두른듯 바람서리 불변함은 우리 기상일세

무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세

가을 하늘 공활한데 높고 구름 없이 밝은 달은 우리 가슴 일편단심일세

무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세

이 기상과 이 마음으로 충성을 다하여 괴로우나 즐거우나

나라사랑하세 무궁화 삼천리 화려 강산 대한사람 대한으로 길이 보전하세

마지막 편집일시 : 2018년 7월 10일 11:41

---

05. Python List
1. list란?
list는 자료들의 모임입니다.
입력된 순서가 유지 됩니다.
list()생성자나 []로 리스트를 만듭니다.
set과 다르게 1개짜리 리스트도 만들 수 있습니다.
어떤 자료 형도 저장이 가능합니다.
Ex19_list1.py

```
list1 = list()
print(type(list1), list1)

list2 = []
print(type(list2), list2)

list3 = [1, 2, 3, 4, 5]
print(type(list3), list3)

list4 = [1]
print(type(list4), list4)

list5 = [1, 2, 3, 1.1, 2.2, 'A', True,(1,2,3)]
print(type(list5), list5)
```

실행 결과

```
<class 'list'> []
<class 'list'> []
<class 'list'> [1, 2, 3, 4, 5]
<class 'list'> [1]
```

<class 'list'> [1, 2, 3, 1.1, 2.2, 'A', True, (1, 2, 3)]
2. 리스트의 길이, 추가, 삽입, 수정,
len() 함수를 이용하여 요소 개수를 얻을 수 있습니다.
[n:m] 값을 이용하여 접근이 가능합니다.
append() 메서드를 이용하여 추가가 가능합니다.
insert() 메서드를 이용하여 삽입이 가능합니다.
remove() 메서드를 이용하여 삭제가 가능합니다.
clear() 메서드를 이용하여 모든 요소 삭제가 가능합니다.
Ex20_list2.py

```
import random as rnd
list1 = [];
print("리스트의 길이 :",len(list1))
for i in range(1,10):
    list1.append(rnd.randrange(1,100))  # 추가

print("리스트의 길이 :",len(list1))
print(list1)
print('2번째부터 4번째까지  :',list1[2:5])

print(list1[0])
list1[0] = 99 # 수정
print(list1[0])

list1.insert(3,88) # 삽입
list1.insert(3,77)
print(list1)

# 삭제 : 인덱스가 변하므로 주의해야 함
for i in list1:
    if i > 50:
        list1.remove(i)
print(list1)

# 삭제 뒤에서 부터 해야함
for index in range(len(list1)-1, -1, -1):
    if list1[index] > 50:
        list1.remove(list1[index])
print(list1)

# 모두 지우기
```

```
list1.clear()
print(list1)
```

실행 결과

```
리스트의 길이 : 0
리스트의 길이 : 9
[61, 20, 63, 11, 98, 4, 51, 35, 75]
2번째부터 4번째까지  : [63, 11, 98]
61
99
[99, 20, 63, 77, 88, 11, 98, 4, 51, 35, 75]
[20, 77, 11, 4, 35]
[20, 11, 4, 35]
[]
```

3. 정렬, 뒤집기, 복사

reverse()메서드를 이용하여 뒤집기가 가능합니다.

sort() 메서드를 이용하여 정렬이 가능합니다.

sorted() 함수를 이용하여 정렬이 가능합니다.

copy() 메서드를 이용하여 복사가 가능합니다.

Ex21_list3.py

```
import random as rnd
list1 = []
for i in range(1,10):
    list1.append(rnd.randrange(1,100))  # 추가
print(list1)
# 뒤집기
list1.reverse()
print(list1)

# 오름 차순 정렬
list1.sort()  # 자신이 변경됨
print(list1)
# 내림 차순 정렬
list1.sort(reverse=True)
print(list1)

list2 = sorted(list1)  # 정렬된 리스트 반환
print(list1, list2)
```

```
list2 = sorted(list1, reverse=True)
print(list1, list2)

# 얕은복사 : 주소가 복사됨
list2 = list1
print(list1, list2)
list2[0] = 999  # 1개변경
print(list1, list2) # 둘다 변경 : 같은 객체이다.


# 깊은복사 : 값이 복사됨
list2 = list1.copy()
print(list1, list2)
list2[0] = 777  # 1개 변경
print(list1, list2) # 1개만 변경 : 다른 객체이다.
```

실행 결과

```
[34, 46, 58, 56, 22, 62, 34, 97, 57]
[57, 97, 34, 62, 22, 56, 58, 46, 34]
[22, 34, 34, 46, 56, 57, 58, 62, 97]
[97, 62, 58, 57, 56, 46, 34, 34, 22]
[97, 62, 58, 57, 56, 46, 34, 34, 22] [22, 34, 34, 46, 56, 57, 58, 62, 97]
[97, 62, 58, 57, 56, 46, 34, 34, 22] [97, 62, 58, 57, 56, 46, 34, 34, 22]
[97, 62, 58, 57, 56, 46, 34, 34, 22] [97, 62, 58, 57, 56, 46, 34, 34, 22]
[999, 62, 58, 57, 56, 46, 34, 34, 22] [999, 62, 58, 57, 56, 46, 34, 34, 22]
[999, 62, 58, 57, 56, 46, 34, 34, 22] [999, 62, 58, 57, 56, 46, 34, 34, 22]
[999, 62, 58, 57, 56, 46, 34, 34, 22] [777, 62, 58, 57, 56, 46, 34, 34, 22]
```

마지막 편집일시 : 2018년 7월 10일 2:06

---

06. Python Dictionary

1. dictionary(사전) 란?

dictionary는 "키(Key)/값(Value)" 쌍을 요소로 갖는 자료형입니다.
dictionary는 흔히 Map 이라고도 불립니다. 키(Key)로 신속하게 값(Value)을 찾아내는 해시테이블(Hash Table) 구조를 갖습니다.
파이썬에서 dictionary는 "dict" 클래스로 구현되어 있습니다.
dictionary의 키(key)는 그 값을 변경할 수 없는 Immutable 타입이어야 하며, dictionary 값(value)은 Immutable과 Mutable 모두 가능합니다. 예를 들어, dictionary의 키(key)로 문자열이나 Tuple은 사용될 수 있는 반면, 리스트는 키로 사용될 수 없습니다.
dictionary의 요소들은 {} 를 사용하여 만듭니다. 여러개일 경우 콤마(,)로 구분합니다.
{}를 이용하여 만들 수 있습니다.
dict() 생성자를 이용하여 만들 수 있습니다.

Ex22_dictionary1.py

```
# dictionary(사전)란?
dict1 = {}
print(dict1, type(dict1))

dict2 = {'name':'한사람','age':22, 'gender': True}
print(dict2, type(dict2))
print(dict2['name'])
print(dict2['age'])
print(dict2['gender'])

tuple1 = ('이름','나이','성별')
dict3 = {tuple1 : ('한사람', 22, True)}
print(dict3, type(dict3))

print(dict3[tuple1])
for index in range(0,3):
    print(tuple1[index],":", dict3[tuple1][index])

dict4 = dict(kor=87, eng=80, mat=85, edps=99)
print(dict4, type(dict4))
print(dict4['kor'])
```

실행 결과

```
{} <class 'dict'>
{'name': '한사람', 'age': 22, 'gender': True} <class 'dict'>
한사람
22
True
{('이름', '나이', '성별'): ('한사람', 22, True)} <class 'dict'>
('한사람', 22, True)
이름 : 한사람
나이 : 22
성별 : True
{'kor': 87, 'eng': 80, 'mat': 85, 'edps': 99} <class 'dict'>
87
```

2. dictionary 조작하기

keys() 메서드로 키값 만을 얻을 수 있습니다.

values() 메서드로 값만을 얻을 수 있습니다.

"dict객체[키]=값"으로 새로운 요소를 추가 가능합니다.

update()메서드로 한번에 여러게 요소를 추가 가능합니다.
같은 키에 다른값을 넣으면 수정이 됩니다.
"del dict객체[키]"으로 요소 삭제가 가능합니다.
items() 메서드로 (key,value)쌍 얻을 수 있습니다.
Ex23_dictionary2.py

```
# Dictionary(사전)란?
dict1 = {'name': '한사람', 'age': 22, 'gender': True}
print(dict1, type(dict1))

# 키 반복
for key in dict1.keys():
    print(key, ':', dict1[key])

# 값 반복
for value in dict1.values():
    print(value)

# 추가
dict1['hobby'] = ['술', '춤', '잠']
dict1['weight'] = 78.65
print(dict1)

# 동시에 여러요소 추가
dict1.update({'weight':67.8,'height': 189.7})
print(dict1)

# 수정
dict1['hobby'] = ['술', '춤', '잠','게임','등산']
print(dict1)

# 요소 삭제
del dict1['weight']
print(dict1)

# (key,value)쌍 얻기
print(dict1.items())
for data in dict1.items():
    # print(data, type(data))
    print(data[0], ':', data[1])
```

실행 결과

{'name': '한사람', 'age': 22, 'gender': True} <class 'dict'>
name : 한사람
age : 22
gender : True
한사람
22
True
{'name': '한사람', 'age': 22, 'gender': True, 'hobby': ['술', '춤', '잠'], 'weight': 78.65}
{'name': '한사람', 'age': 22, 'gender': True, 'hobby': ['술', '춤', '잠'], 'weight': 67.8, 'height': 189.7}
{'name': '한사람', 'age': 22, 'gender': True, 'hobby': ['술', '춤', '잠', '게임', '등산'], 'weight': 67.8, 'height': 189.7}
{'name': '한사람', 'age': 22, 'gender': True, 'hobby': ['술', '춤', '잠', '게임', '등산'], 'height': 189.7}
dict_items([('name', '한사람'), ('age', 22), ('gender', True), ('hobby', ['술', '춤', '잠', '게임', '등산']), ('height', 189.7)])
name : 한사람
age : 22
gender : True
hobby : ['술', '춤', '잠', '게임', '등산']
height : 189.7
마지막 편집일시 : 2018년 7월 10일 2:59
---

04. Python(파이썬) 연산자

목차

- 산술 연산자 (Arithmetic Operators)
- 비교 (즉, 관계형) 연산자 (Comparison (i.e., Relational) Operators)
- 할당 연산자 (Assignment Operators)
- 논리 연산자 (Logical Operators)
- 비트 연산자 (Bitwise Operators)
- 구성원 연산자 (Membership Operators)
- 식별 연산자 (Identity Operators)

마지막 편집일시 : 2018년 7월 10일 4:08

01. 산술 연산자 (Arithmetic Operators)
1. 산술 연산자 (Arithmetic Operators)
+ : 더하기
- : 빼기
* : 곱하기
/ : 나누기
// : 정수 몫
% : 나머지
* * : 누승

학교에서 초등학교때 부터 많이 배웠기 때문에 특별한 설명은 없습니다.
배우지 않은 생소한것 몇가지를 집중적으로 보면서 어떻게 사용하는지 알아보도록 하겠습니다.
기본 연산
Ex01.py

```
# 산술 연산자
a = 10
b = 3
print(a, type(a))                # 10
print(b, type(b))                # 3
print(a + b, type(a + b))        # 13
print(a - b, type(a - b))        # 7
print(a * b, type(a * b))        # 30
print(a / b, type(a / b))        # 3.3333333333333335
print(a // b, type(a // b))      # 3
print(a % b, type(a % b))        # 1
print(a ** b, type(a ** b))      # 1000
print("-"*20)                    # --------------------
```

※ int 나누기 int의 결과가 float형 입니다.

실행 결과

```
10 <class 'int'>
3 <class 'int'>
13 <class 'int'>
7 <class 'int'>
30 <class 'int'>
3.3333333333333335 <class 'float'>
```

```
3 <class 'int'>
1 <class 'int'>
1000 <class 'int'>
--------------------
```

형식화된 결과 출력

Ex02.py

```
# 산술 연산자 : + - * / // % **
i = 10
j = 3;
# int 와 int 연산의 결과는 int형 입니다.
# 특별히 int 나누기 int의 결과가 float형 입니다.
print("{0} + {1} = {2}".format(i, j, i+j))
print("{0} - {1} = {2}".format(i, j, i-j))
print("{0} * {1} = {2}".format(i, j, i*j))
print("{0} / {1} = {2}".format(i, j, i/j))
print("{0} // {1} = {2}".format(i, j, i//j))
print("{0} % {1} = {2}".format(i, j, i%j))
print("{0} ** {1} = {2}".format(i, j, i**j))
print()

# int 와 float 연산의 결과는 float형 입니다.
i = 10
j = 3.0;
print("{0} + {1} = {2}".format(i, j, i+j))
print("{0} - {1} = {2}".format(i, j, i-j))
print("{0} * {1} = {2}".format(i, j, i*j))
print("{0} / {1} = {2}".format(i, j, i/j))
print("{0} // {1} = {2}".format(i, j, i//j))
print("{0} % {1} = {2}".format(i, j, i%j))
print("{0} ** {1} = {2}".format(i, j, i**j))
print()

# 나눗셈을 연산하는 divmod() 라는 함수도 있는데,
# 몫과 나머지를 튜플로 반환한다.
print(divmod(i, j))  # (3.0, 1.0)
```

실행 결과

```
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
```

```
10 / 3 = 3.3333333333333335
10 // 3 = 3
10 % 3 = 1
10 ** 3 = 1000

10 + 3.0 = 13.0
10 - 3.0 = 7.0
10 * 3.0 = 30.0
10 / 3.0 = 3.3333333333333335
10 // 3.0 = 3.0
10 % 3.0 = 1.0
10 ** 3.0 = 1000.0

(3.0, 1.0)
```

/(나눗셈) 와 //(정수몫) 연산자의 고찰

Ex03.py

```
# 산술 연산자 : / 와 //
print(3//2)          # 1
print(-3//2)         # -2
print(10//3)         # 3
print(-10//3)        # -4
print("="*50)
# 몫을 넘지않는 최대 정수 ?
print(3/2)          # 1.5
print(3//2)         # 1
print(-3/2)         # -1.5
print(-3//2)         # -2
print("="*50)
# 몫을 넘지않는 최대 정수 ?
print(10/3)         # 3.3333333333333335
print(10//3)        # 3
print(-10/3)        # -3.3333333333333335
print(-10//3)       # -4
print(-10/3.0)        # -3.3333333333333335
print(-10//3.0)       # -4.0
print("="*50)
# 결론 : // 연산자는 몫을 넘지않는 "최대 정수 ?"를 구합니다.
import math
# math.floor() : 정수 부분을 넘지않는 최대 정수
print(math.floor(3.3))
```

```
print(math.floor(-3.3))
# math.ceil() : 정수 부분을 넘지않는 최소 정수
print(math.ceil(3.3))
print(math.ceil(-3.3))
# int로 형변환
print(int(3.3))
print(int(-3.3))
```

※ // 연산자는 몫을 넘지않는 "최대 정수 ?"를 구합니다.

실행 결과

```
1
-2
3
-4
==================================================
1.5
1
-1.5
-2
==================================================
3.3333333333333335
3
-3.3333333333333335
-4
-3.3333333333333335
-4.0
==================================================
3
-4
4
-3
3
-3
```

%(나머지) 연산자의 고찰

Ex03.py

```
# 산술 연산자 : %
print('10/3의 결과')
print('결과 :', 10 / 3)          # 3.3333333333333335
print('몫 :', 10 // 3)           # 3
```

```
print('나머지 :', 10 % 3)          # 1
print("-" * 20)

print('3/10의 결과')
print('결과 :', 3 / 10)            #  0.3
print('몫 :', 3 // 10)             # 0
print('나머지 :', 3 % 10)          # 3
print("-" * 20)

print('-10/3의 결과')
print('결과 :', -10 / 3)           # -3.3333333333333335
print('몫 :', -10 // 3)            # -4
print('나머지 :', -10 % 3)         #  2
print("-" * 20)

print('-3/10의 결과')
print('결과 :', -3 / 10)          # -0.3
print('몫 :', -3 // 10)           # -1
print('나머지 :', -3 % 10)        #  7
print("-" * 20)

print('10/-3의 결과')
print('결과 :', 10 / -3)           # -3.3333333333333335
print('몫 :', 10 // -3)            # -4
print('나머지 :', 10 % -3)         # -2
# "나머지 = 피제수 -(제수*몫)"
print('나머지 :', 10 -(-3 * (10//-3)))
print("-" * 20)

print('3/-10의 결과')
print('결과 :', 3 / -10)          # -0.3
print('몫 :', 3 // -10)           # -1
print('나머지 :', 3 % -10)        # -7
# "나머지 = 피제수 -(제수*몫)"
print('나머지 :', 3 -(-10 * (3//-10)))
print("-" * 20)

# %연산의 결론
#  제수 * 몫 + 나머지 = 피제수 에서 "나머지 = 피제수 -(제수*몫)"
#  10/3 = 3 * 3 + (1) = 10
#  3/10 = 10 * 0 + (3) =  3
```

```
# -10/3 = 3 * -4 + (2) = -10
# -3/10 = 10 * -1 + (7) = -3
# 10/-3 = -3 * -4 + (-2) = 10
# 3/-10 = -10 * -1 +(-7) = 3
```

실행 결과

```
10/3의 결과
결과 : 3.3333333333333335
몫 : 3
나머지 : 1
--------------------
3/10의 결과
결과 : 0.3
몫 : 0
나머지 : 3
--------------------
-10/3의 결과
결과 : -3.3333333333333335
몫 : -4
나머지 : 2
--------------------
-3/10의 결과
결과 : -0.3
몫 : -1
나머지 : 7
--------------------
10/-3의 결과
결과 : -3.3333333333333335
몫 : -4
나머지 : -2
나머지 : -2
--------------------
3/-10의 결과
결과 : -0.3
몫 : -1
나머지 : -7
나머지 : -7
--------------------
```

2. 산술 연산자 (Arithmetic Operators) 활용

숫자 분리하기

Ex05.py

```
# 숫자 분리하기
birth = 19920822
year = birth//10000
month1 = birth%10000//100
month2 = birth//100%100
date = birth%100
print(year, month1, date)
print(year, month2, date)
print("생년월일 : {}년 {}월 {}일".format(year,month1,date))
print("생년월일 : {}년 {}월 {}일".format(year,month2,date))
```

실행 결과

```
1992 8 22
1992 8 22
생년월일 : 1992년 8월 22일
생년월일 : 1992년 8월 22일
```

숫자 뒤집어서 출력하기

while 반복문 : 조건이 참인 동안 계속 실행 됩니다.

```
while 조건:
    조건이 참인동안 실행할 명령들....
```

Ex06.py

```
# 숫자 뒤집어서 출력하기
num = 123456
print("원본 :", num)
print("뒤집기 : ", end='')
while num > 0:
    print(num % 10, end='')
    num = num // 10
print()
```

실행 결과

```
원본 : 123456
뒤집기 : 654321
```

내가 입력한 숫자는 몇자리 숫자일까?

if조건문 : 조건이 참인 경우에만 실행됩니다.

break문 : 반복문을 탈출합니다.

```
    if 조건:
```

조건이 참인동안 실행할 명령들....

Ex07.py

```
# 내가 입력한 숫자는 몇자리 숫자일까?
while True:
    num = int(input("숫자입력(0은 종료) : "))
    if num == 0:
        break
    count = 0
    print(num,'의 자릿수 ', end='')
    while num > 0:
        count = count + 1
        num = num // 10
    print(count, '자리입니다.')
```

실행 결과

```
숫자입력(0은 종료) : 12345
12345 의 자릿수 5 자리입니다.
숫자입력(0은 종료) : 321432143
321432143 의 자릿수 9 자리입니다.
숫자입력(0은 종료) : 1
1 의 자릿수 1 자리입니다.
숫자입력(0은 종료) : 10001
10001 의 자릿수 5 자리입니다.
숫자입력(0은 종료) : 0001
1 의 자릿수 1 자리입니다.
숫자입력(0은 종료) : 10000
10000 의 자릿수 5 자리입니다.
숫자입력(0은 종료) : 0
```

통화 단위별 매수 출력하기

풀이 1 : 복사하여 붙여넣기 방법

Ex08.py

```
# 통화 단위별 매수 출력하기!!
while True:
    salary = int(input("월급입력(0은 종료) : "))
    if salary == 0:
        break
    print('{0:12,}원권 {1}매'.format(1000000, salary//1000000))
    salary %= 1000000
    print('{0:12,}원권 {1}매'.format(500000, salary//500000))
```

```
    salary %= 500000
    print('{0:12,}원권 {1}매'.format(100000, salary//100000))
    salary %= 100000
    print('{0:12,}원권 {1}매'.format(50000, salary//50000))
    salary %= 50000
    print('{0:12,}원권 {1}매'.format(10000, salary//10000))
    salary %= 10000
    print('{0:12,}원권 {1}매'.format(5000, salary//5000))
    salary %= 5000
    print('{0:12,}원권 {1}매'.format(1000, salary//1000))
    salary %= 1000
    print('{0:12,}원권 {1}매'.format(500, salary//500))
    salary %= 500
    print('{0:12,}원권 {1}매'.format(100, salary//100))
    salary %= 100
    print('{0:12,}원권 {1}매'.format(50, salary//50))
    salary %= 50
    print('{0:12,}원권 {1}매'.format(10, salary//10))
    salary %= 10
    print('{0:12,}원권 {1}매'.format(5, salary//5))
    salary %= 5
    print('{0:12,}원권 {1}매'.format(1, salary))
```

풀이 2 : 컬렉션과 반복문 사용

Ex09.py

```
# 통화 단위별 매수 출력하기!!
currency = (1000000, 500000, 100000, 50000, 1000, 5000, 1000, 500, 100, 50, 10, 5,
1)
while True:
    salary = int(input("월급입력(0은 종료) : "))
    if salary == 0:
        break
    for i in currency:
        print('{0:12,}원권 {1}매'.format(i, salary//i))
        salary %= i
```

실행 결과

```
월급입력(0은 종료) : 7896789
   1,000,000원권 7매
     500,000원권 1매
     100,000원권 3매
```

```
        50,000원권 1매
        10,000원권 4매
         5,000원권 1매
         1,000원권 1매
           500원권 1매
           100원권 2매
            50원권 1매
            10원권 3매
             5원권 1매
             1원권 4매
월급입력(0은 종료) : 0
마지막 편집일시 : 2018년 7월 11일 11:37
```

---

02. 관계 연산자 (Relational Operators)

1. 관계 연산자 (Relational Operators)

> : 크다
>= : 크거나 같다
< : 작다
<= : 작거나 같다
== : 같다
!= : 같지 않다

대소 관계와 상등관계를 나타내는 연산자로 결과값이 논리값입니다.
학교에서 초등학교때 부터 많이 배웠기 때문에 특별한 설명은 없습니다.
마지막의 두개는 상등연산자라고도 합니다.
같다의 기호는 "="라는 대입연산자와 겹치므로 "=="를 사용합니다.
같지 않다라는 기호도 키보드에 "≠"이 없으므로 "!="를 사용합니다.
기본 연산
Ex01.py

```
# 관계 연산자(Relational Operators)
# 대소 관계와 상등관계를 나타내는 연산자로 결과값이 논리값입니다.
a = 10
b = 3

print("10 == 3 =>", a == b)
print("10 != 3 =>", a != b)
print("10 > 3 =>", a > b)
print("10 >= 3 =>", a >= b)
print("10 < 3 =>", a < b)
print("10 <= 3 =>", a <= b)
```

print()
실행 결과

```
10 == 3 => False
10 != 3 => True
10 > 3 => True
10 >= 3 => True
10 < 3 => False
10 <= 3 => False
```

2. 관계 연산자(Relational Operators) 활용
내가 입력한 숫자는 홀수일까? 짝수일까?
"짝수가 어떤수일까요?"라고 물으면 대부분의 사람이 "2, 4, 6, 8, 10 .... "이라고 대답을 합니다. "그러면 홀수는?"하고 다시 물으면 "1,3,5,7,9...."라고 대답을 합니다. 그렇다면 위의 대답을 프로그램으로 작성할 수 있을까요? 아닙니다. 앞으로는 정의를 내릴때 컴퓨터로 표현가능하게 정의를 내리는 연습을 하시기 바랍니다. 조금 진보된 답변이 "2N 또는 2의 배수"입니다. 그렇다면 2의 배수는 어떻게 표현을 해야 프로그램으로 작성할 수 있을까요? 다음 처럼 정의를 내리면 프로그램으로 옮길 수 있습니다.

짝수 : 수를 2로 나누어서 나머지가 0인 수
홀수 : 수를 2로 나누어서 나머지가 0이 아닌 수
Ex02.py

```
# 내가 입력한 숫자는 짝수일까 홀수일까?
# 3항연산자 : 참 if 조건 else 거짓
while True:
    num = int(input("숫자입력(0은 종료) : "))
    if num == 0:
        break
    print("짝수" if num % 2 == 0 else "홀수")
    print("홀수" if num % 2 != 0 else "짝수")
    print("홀수" if num % 2 else "짝수")
```

실행 결과

```
숫자입력(0은 종료) : 34
짝수
짝수
짝수
숫자입력(0은 종료) : 21
홀수
홀수
홀수
```

숫자입력(0은 종료) : 0
참/거짓의 판단은?
True는 참, False는 거짓입니다.
조건문에서 number타입을 조건으로 지정하면 0은 거짓 그 외의 숫자는 참입니다.
True와 False를 직접 비교시 True는 1 False는 0입니다.
산술식에 True와 False를 사용하면 True는 1 False는 0입니다.
Ex03.py

```
# 내가 입력한 숫자는 짝수일까 홀수일까?
# 3항연산자 : 참 if 조건 else 거짓
# 참고적으로 Python에서 True(참)/False(거짓) 판단은 어떻게 될까요?

# True/False
print("True는 참" if True else "True는 거짓")
print("False는 참" if False else "False는 거짓")

# 조건문에서 number타입을 조건으로 지정하면 0은 거짓 그 외의 숫자는 참입니다.
print("1은 참" if 1 else "1은 거짓")
print("0은 참" if 0 else "0은 거짓")
print("-1은 참" if -1 else "-1은 거짓")
print("2는 참" if 2 else "2는 거짓")
print("1.0은 참" if 1.0 else "1.0은 거짓")
print("-1.0은 참" if -1.0 else "-1.0은 거짓")
print("0.0은 참" if 0.0 else "-1.0은 거짓")
print("0.1은 참" if 0.1 else "0.1은 거짓")

# True와 False를 직접 비교시 True는 1 False는 0입니다.
print(True == 1)
print(True == 2)
print(False == 0)
print(False == 2)
# 산술식에 True와 False를 사용하면 True는 1 False는 0입니다.
print(True + False + True)
```

실행 결과

```
True는 참
False는 거짓
1은 참
0은 거짓
-1은 참
2는 참
```

1.0은 참
-1.0은 참
-1.0은 거짓
0.1은 참
True
False
True
False
2
마지막 편집일시 : 2018년 7월 11일 2:24
---
03. 논리 연산자(Logical Operators)
1. 논리 연산자(Logical Operators)
and : 논리곱
or : 논리합
not : 논리부정

논리 값을 판단해주는 연산자 입니다.
진리표

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

기본 연산
Ex01.py

```
# 논리 연산자(Logical operator)
# and, or, not
print(True and True)
print(True and False)
print(False and True)
print(False and False)
print()
print(True or True)
print(True or False)
print(False or True)
print(False or False)
print()
print(not True)
print(not False)
```

실행 결과

True
False
False
False

True
True
True
False

False
True
2. 논리 연산자 활용
1~10사이의 수중에서 3의 배수 또는 5의 배수 출력하려면
Ex02.py

```
# 1~10사이의 수중에서 3의 배수 또는 5의 배수 출력
for i in range(1,11):
    if i%3==0 or i%5==0:
        print(i, end=' ')
print()
for i in range(1,11):
    if not i%3 or not i%5:
        print(i, end=' ')
print()
for i in range(1,11):
    if not (i%3 and i%5):
        print(i, end=' ')
print()
```

실행 결과

3 5 6 9 10
3 5 6 9 10
3 5 6 9 10
1~10사이의 수중에서 2의 배수 이면서 3의 배수인 수를 출력
Ex03.py

```
# 1~10사이의 수중에서 2의 배수 이면서 3의 배수인 수를 출력
for i in range(1,11):
```

```
    if i%2==0 and i%3==0:
        print(i, end=' ')
print()

for i in range(1,11):
    if not(i%2 or i%3):
        print(i, end=' ')
print()

for i in range(1,11):
    if not i%6:
        print(i, end=' ')
print()
```

실행 결과

```
6
6
6
```

1~10사이의 3의 배수만 빼고 출력

Ex04.py

```
for i in range(1,11):
    if not (i%3==0):
        print(i, end=' ')
print()

for i in range(1,11):
    if i%3:
        print(i, end=' ')
print()
```

실행 결과

```
1 2 4 5 7 8 10
1 2 4 5 7 8 10
```

년도를 입력받아 윤년 판단하기

윤년(閏年)은 역법을 실제 태양년에 맞추기 위해 여분의 하루를 끼워넣는 해를 말합니다. 태양년은 정수의 하루로 나누어떨어지지 않기 때문에 태양력에서는 하루(윤일)를 적절한 시기에 끼워서 이를 보정합니다.

윤년(閏年)은 태양력에서 보통 윤일이 들어 있는 해를 말하는데, 이 경우 1년은 366일이 되며 이것이 바로 윤년입니다. 지구가 태양을 한 바퀴 도는 데에는 365일 5시간 48분 46초가 걸리므로 365일을 제외한 시간들을 모아 태양력에서는 4년마다 한 번 2월 29일을 두어

하루를 늘 윤년이라 합니다. 윤년이 아닌 해는 평년이라고 합니다.
윤년 판단 규칙
1. 년도가 4로 나누어떨어지는 해는 윤년입니다. (1996년, 2004년, 2008년, 2012년 …)
2. 이 중에서 100으로 나누어떨어지는 해는 평년입니다. (1900년, 2100년, 2200년, 2300년 …)
3. 그중에 400으로 나누어떨어지는 해는 윤년입니다. (1600년, 2000년, 2400년 …)

Ex05.py

```
# 윤년 판단 규칙
# 1. 년도가 4로 나누어떨어지는 해는 윤년입니다. (1996년, 2004년, 2008년, 2012년 …)
# 2. 이 중에서 100으로 나누어떨어지는 해는 평년입니다. (1900년, 2100년, 2200년, 2300년 …)
# 3. 그중에 400으로 나누어떨어지는 해는 윤년입니다. (1600년, 2000년, 2400년 …)

while True:
    year = int(input('년도를 입력하시오(0은 종료)'))
    if not year:
        break
    # print('윤년' if year%400==0 or year%4==0 and year%100!=0 else '평년')
    # print('윤년' if year%400==0 or year%4==0 and year%100 else '평년')
    print('윤년' if not year%400 or not year%4 and year%100 else '평년')
```

실행 결과

```
년도를 입력하시오(0은 종료)1
평년
년도를 입력하시오(0은 종료)4
윤년
년도를 입력하시오(0은 종료)96
윤년
년도를 입력하시오(0은 종료)100
평년
년도를 입력하시오(0은 종료)200
평년
년도를 입력하시오(0은 종료)300
평년
년도를 입력하시오(0은 종료)400
윤년
년도를 입력하시오(0은 종료)0
```

마지막 편집일시 : 2018년 7월 11일 2:19
---

04. 삼항 연산자(Ternary operators)
1. 삼항 연산자(Ternary operators)
참인경우 값 if 조건 else 거짓인경우 값

연산 대상의 개수에 따라 연산자를 분리하면 단항 연산자, 이항 연산자, 삼항 연산자로 분리 합니다.
단항 연산자는 부호(+, -), not 등이 있으며 +, -, *, / .... 등 대부분의 연산자가 이항 연산자 입니다.
삼항 연산자는 1개가 존재 합니다.
기본 연산
Ex01.py

```
# 삼항 연산자
# 참인경우 값 if 조건 else 거짓인경우 값
print("True는 참" if True else "True는 거짓")
print("False는 참" if False else "False는 거짓")
year = 2000;
isLeapYear = year%400==0 or year%4==0 and year%100
print('{0}년은 "{1}"입니다.'.format(year,'윤년' if isLeapYear else '평년'))
print('{0}년은 "{1}"년입니다.'.format(year,'홀수' if year%2 else '짝수'))
```

실행 결과

```
True는 참
False는 거짓
2000년은 "윤년"입니다.
2000년은 "홀수"년입니다.
```

2. 삼항 연산자 활용
내가 입력한 숫자는 홀수일까? 짝수일까?
Ex02.py

```
while True:
    num = int(input("숫자입력(0은 종료) : "))
    if num == 0:
        break
    print("짝수" if num % 2 == 0 else "홀수")
    print("홀수" if num % 2 != 0 else "짝수")
    print("홀수" if num % 2 else "짝수")
```

실행 결과

```
숫자입력(0은 종료) : 34
짝수
```

짝수
짝수
숫자입력(0은 종료) : 21
홀수
홀수
홀수
숫자입력(0은 종료) : 0
년도와 월을 입력받아 월을 마지막 날짜 구하기
Ex03.py

```
# 년도와 월을 입력받아 월을 마지막 날짜 구하기

while True:
    year = int(input('년도을 입력하시오(0은 종료)'))
    if not year:
        break
    month = int(input('월을 입력하시오(0은종료)'))
    if not month:
        break

    last_day = (29 if year % 400 == 0 or year % 4 == 0 and year % 100 else 28) \
               if (month==2) else (30 if month%2 else 31) \
               if (month>=8) else (31 if month%2 else 30)

    print('{0}년 {1}월의 마지막 날짜는 {2}일 입니다.'.format(year,month,last_day))
```

실행 결과

년도을 입력하시오(0은 종료)2000
월을 입력하시오(0은종료)2
2000년 2월의 마지막 날짜는 29일 입니다.
년도을 입력하시오(0은 종료)1989
월을 입력하시오(0은종료)2
1989년 2월의 마지막 날짜는 28일 입니다.
년도을 입력하시오(0은 종료)2018
월을 입력하시오(0은종료)5
2018년 5월의 마지막 날짜는 31일 입니다.
년도을 입력하시오(0은 종료)2018
월을 입력하시오(0은종료)9
2018년 9월의 마지막 날짜는 30일 입니다.
년도을 입력하시오(0은 종료)0
년도와 월을 입력받아 월을 마지막 날짜 구하기 2

리스트를 이용하면 편리합니다.
Ex04.py

```
# 년도와 월을 입력받아 월을 마지막 날짜 구하기
months = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
while True:
    year = int(input('년도을 입력하시오(0은 종료)'))
    if not year:
        break
    month = int(input('월을 입력하시오(0은종료)'))
    if not month:
        break

    months[1] = 29 if year % 400 == 0 or year % 4 == 0 and year % 100 else 28

    print('{0}년 {1}월의 마지막 날짜는 {2}일 입니다.'.format(year, month,
months[month-1]))
```

실행 결과

```
년도을 입력하시오(0은 종료)2000
월을 입력하시오(0은종료)2
2000년 2월의 마지막 날짜는 29일 입니다.
년도을 입력하시오(0은 종료)2001
월을 입력하시오(0은종료)2
2001년 2월의 마지막 날짜는 28일 입니다.
년도을 입력하시오(0은 종료)2018
월을 입력하시오(0은종료)4
2018년 4월의 마지막 날짜는 30일 입니다.
년도을 입력하시오(0은 종료)2018
월을 입력하시오(0은종료)3
2018년 3월의 마지막 날짜는 31일 입니다.
년도을 입력하시오(0은 종료)0
```

입력한 두개의 숫자 중 큰수, 작은수 출력하기
Ex05.py

```
# 입력한 두개의 숫자 중 큰수, 작은수 출력하기
while True:
    num1 = int(input('첫번째 정수를 입력하시오(0은 종료)'))
    if not num1:
        break
    num2 = int(input('두번째 정수를 입력하시오(0은종료)'))
```

```
    if not num2:
        break

    print('{0}과(와) {1}중에서 큰수는 {2}입니다.'.format(num1,num2, num1 if
num1>num2 else num2))
    print('{0}과(와) {1}중에서 작은수는 {2}입니다.'.format(num1,num2, num2 if
num1>num2 else num1))
```

실행 결과

```
첫번째 정수를 입력하시오(0은 종료)32
두번째 정수를 입력하시오(0은종료)34
32과 34중에서 큰수는 34입니다.
32과 34중에서 작은수는 32입니다.
첫번째 정수를 입력하시오(0은 종료)12
두번째 정수를 입력하시오(0은종료)44
12과 44중에서 큰수는 44입니다.
12과 44중에서 작은수는 12입니다.
첫번째 정수를 입력하시오(0은 종료)0
```

입력한 3개의 숫자 최대값, 최소값 출력하기

Ex06.py

```
# 입력한 3개의 숫자 최대값, 최소값 출력하기
while True:
    num1 = int(input('첫번째 정수를 입력하시오(0은 종료)'))
    if not num1:
        break
    num2 = int(input('두번째 정수를 입력하시오(0은종료)'))
    if not num2:
        break
    num3 = int(input('세번째 정수를 입력하시오(0은종료)'))
    if not num3:
        break

    # 앞의 두수 중에서 큰값을 max에 넣기
    max = num1 if num1 > num2 else num2
    # max와 마지막 수에서 큰값을 max에 넣기
    max = num3 if num3 > max else max

    # 앞의 두수 중에서 작은값을 max에 넣기
    min = num1 if num1 < num2 else num2
    # min과 마지막 수에서 작은값을 min에 넣기
```

```
    min = num3 if num3 < min else min

    print('최대값은 {0}입니다.'.format(max))
    print('최소값은 {0}입니다.'.format(min))
```

실행 결과

```
첫번째 정수를 입력하시오(0은 종료)1
두번째 정수를 입력하시오(0은종료)2
세번째 정수를 입력하시오(0은종료)3
최대값은 3입니다.
최소값은 1입니다.
첫번째 정수를 입력하시오(0은 종료)3
두번째 정수를 입력하시오(0은종료)2
세번째 정수를 입력하시오(0은종료)1
최대값은 3입니다.
최소값은 1입니다.
첫번째 정수를 입력하시오(0은 종료)1
두번째 정수를 입력하시오(0은종료)3
세번째 정수를 입력하시오(0은종료)2
최대값은 3입니다.
최소값은 1입니다.
첫번째 정수를 입력하시오(0은 종료)0
```

입력한 3개의 숫자 최대값, 최소값 출력하기

Ex07.py

```
# 입력한 3개의 숫자 정렬하여 출력하기
while True:
    num1 = int(input('첫번째 정수를 입력하시오(0은 종료)'))
    if not num1:
        break
    num2 = int(input('두번째 정수를 입력하시오(0은종료)'))
    if not num2:
        break
    num3 = int(input('세번째 정수를 입력하시오(0은종료)'))
    if not num3:
        break

    # 첫 번째 수가 두번째 수보다 크면 교환
    [num1, num2] = [num1, num2] if num1 < num2 else [num2, num1]
    # 두 번째 수가 세번째 수보다 크면 교환
    [num2, num3] = [num2, num3] if num2 < num3 else [num3, num2]
```

```
    # 여기까지오면 세번째에 제일큰수가 위치

    # 첫 번째 수가 두번째 수보다 크면 교환
    [num1, num2] = [num1, num2] if num1 < num2 else [num2, num1]

    # 여기까지 오면 세개의 수는 정렬 완료
    print('차례대로 배열하면 {0},{1},{2}'.format(num1, num2, num3))
```

실행 결과

```
첫번째 정수를 입력하시오(0은 종료)1
두번째 정수를 입력하시오(0은종료)2
세번째 정수를 입력하시오(0은종료)3
차례대로 배열하면 1,2,3
첫번째 정수를 입력하시오(0은 종료)3
두번째 정수를 입력하시오(0은종료)2
세번째 정수를 입력하시오(0은종료)1
차례대로 배열하면 1,2,3
첫번째 정수를 입력하시오(0은 종료)2
두번째 정수를 입력하시오(0은종료)3
세번째 정수를 입력하시오(0은종료)1
차례대로 배열하면 1,2,3
첫번째 정수를 입력하시오(0은 종료)0
```

마지막 편집일시 : 2018년 7월 11일 3:45

---

05. 비트 연산자(Bitwise Operators)

1. 비트 연산자(Bitwise Operators)

& (Binary AND) : bit 단위로 and연산을 합니다.
| (Binary OR) : bit 단위로 or연산을 합니다.
^ (Binary XOR) : bit 단위로 xor연산을 합니다.
~ (Binary NOT) : bit 단위로 not연산을 합니다.(1의 보수)
<< (Binary left Shift) : bit 단위로 왼쪽으로 비트단위 밀기 연산을 합니다.
>> (Binary right Shift) : bit 단위로 오른쪽으로 비트단위 밀기 연산을 합니다.

비트 단위로 연산을 수행합니다.
0은 거짓으로 1은 참으로 연산하여 결과를 1과 0으로 반환합니다.
"^(xor)"연산은 두개의 값이 다를 때만 참인 연산입니다.
" ~(not)" 연산은 1의 보수를 구합니다. 컴퓨터에서는 뺄셈을 2의 보수를 덧셈하여 처리합니다.
"<<"는 연산은 왼쪽으로 1비트 밀때마다 두 배씩 늘어납니다.
">>" 연산은 오른쪽으로 1비트 밀때마다 1/2씩 줄어듭니다.
n << m : n * 2의 m승

n >> m : n / 2의 m승
진법의 초기화 및 출력 그리고 변환
Ex01.py

```
# 진법 초기화
a = 10
print(type(a), a)
a = 0b10
print(type(a), a)
a = 0o10
print(type(a), a)
a = 0x10
print(type(a), a)
print()
a = 0B10
print(type(a), a)
a = 0O10
print(type(a), a)
a = 0X10
print(type(a), a)
print("-"*20)

# 출력
i = 0x100
print("i = ", i)
print("i = ", bin(i))
print("i = ", int(i))
print("i = ", oct(i))
print("i = ", hex(i))
print("-"*20)

# 문자열을 형변환
i = "111"
print("i = ", int(i))
print("i = ", int(i, 2))
print("i = ", int(i, 8))
print("i = ", int(i, 16))
print("i = ", int(i, 5))
print("-"*20)
```

실행 결과

```
<class 'int'> 10
<class 'int'> 2
<class 'int'> 8
<class 'int'> 16

<class 'int'> 2
<class 'int'> 8
<class 'int'> 16
--------------------
i =  256
i =  0b100000000
i =  256
i =  0o400
i =  0x100
--------------------
i =  111
i =  7
i =  73
i =  273
i =  31
--------------------
```

기본 연산

Ex02.py

```
# 비트 연산자(Bitwise Operators)
a = 0b10101010
b = 0b01110011

print('a = ',  a, ":", bin(a))
print('b = ',  b, ":", bin(b))
print('a & b = ',  a & b, ":", bin(a & b))
print('a | b = ',  a | b, ":", bin(a | b))
print('a ^ b = ',  a ^ b, ":", bin(a ^ b))
print('~a = ',  ~a, ":", bin(~a))

a = 0b1
print('a = ', a)
a = a << 1     # * 2
print('a = ', a)
a = a << 1     # * 2
print('a = ', a)
```

```
a = a << 3     # * 2**3
print('a = ', a)

a = a >> 1     # / 2
print('a = ', a)
a = a >> 1     # / 2
print('a = ', a)
a = a >> 2     # / 2**2
print('a = ', a)
```

실행 결과

```
a =  170 : 0b10101010
b =  115 : 0b1110011
a & b =  34 : 0b100010
a | b =  251 : 0b11111011
a ^ b =  217 : 0b11011001
~a =  -171 : -0b10101011
a =  1
a =  2
a =  4
a =  32
a =  16
a =  8
a =  2
```

2. 비트 연산자(Bitwise Operators) 활용

2의 누승 출력하기

Ex03.py

```
# 비트 연산자(Bitwise Operators)
# 2의 누승 출력하기
a = 1
for i in range(0, 11):
    print('{0:6,}'.format(a << i), end=' ')
print()

a = 2
for i in range(0, 11):
    print('{0:6,}'.format(a ** i), end=' ')
print()

a = 1024
```

```
for i in range(0, 11):
    print('{0:6,}'.format(a >> i), end=' ')
print()

a = 2
for i in range(10, -1, -1):
    print('{0:6,}'.format(a ** i), end=' ')
print()
```

실행 결과

```
     1     2     4     8    16    32    64   128   256   512 1,024
     1     2     4     8    16    32    64   128   256   512 1,024
 1,024   512   256   128    64    32    16     8     4     2     1
 1,024   512   256   128    64    32    16     8     4     2     1
```

보수를 이용한 밸셈

Ex04.py

```
# 2의 보수를 이용한 뺄셈
# ~(1의보수) + 1 = 2의 보수
a = 10
b = 3
c = a - b
d = a + (~b + 1)
print('a - b = ', c)
print('a + (~b + 1) = ', d)
```

실행 결과

```
a - b =  7
a + (~b + 1) =  7
```

두개의 숫자를 교환하기

Ex05.py

```
# 두개의 숫자를 교환하기
a = 100
b = 200
print('a={0}, b={1}'.format(a,b))

# 임시 변수를 사용하는 방법
temp = a
a = b
b = temp
```

```
print('a={0}, b={1}'.format(a,b))

# Python의 tuple을 이용하는 방법
a, b = b, a
print('a={0}, b={1}'.format(a,b))

# ^(xor)연산자를 이용하는 방법
a = a ^ b
b = a ^ b
a = a ^ b
print('a={0}, b={1}'.format(a,b))
```

실행 결과

```
a=100, b=200
a=200, b=100
a=100, b=200
a=200, b=100
```

10진수를 입력받아 16진수로 출력하기

Ex06.py

```
# 10진수를 입력받아 16진수로 출력하기
while True:
    n = int(input('정수입력(1~255사이, 0은종료)'))
    if not n:
        break
    if n<0 or n>255:  # 0x00 ~ 0xFF 사이의 숫자
        print('올바른 범위의 정수를 입력하시오')
        continue
    high = n >> 4 & 0XF      # 상위 4비트만 뽑아내기
    low = n & 0XF            # 하위 4비트만 뽑아내기
    print('원본 : ', bin(n))
    print('8비트중 상위 4비트 : ', bin(high))
    print('8비트중 하위 4비트 : ', bin(low))
    print('사용자 : 0X{0:1}'.format(high if high < 10 else chr(ord('A') + high-10)),
end='')
    print('{0:1}'.format(low if low < 10 else chr(ord('A') + low-10)))
    print('파이선 : {}'.format(hex(n)))
```

실행 결과

```
정수입력(1~255사이, 0은종료)255
원본 :  0b11111111
```

8비트중 상위 4비트 : 0b1111
8비트중 하위 4비트 : 0b1111
사용자 : 0XFF
파이선 : 0xff
정수입력(1~255사이, 0은종료)128
원본 : 0b10000000
8비트중 상위 4비트 : 0b1000
8비트중 하위 4비트 : 0b0
사용자 : 0X80
파이선 : 0x80
정수입력(1~255사이, 0은종료)78
원본 : 0b1001110
8비트중 상위 4비트 : 0b100
8비트중 하위 4비트 : 0b1110
사용자 : 0X4E
파이선 : 0x4e
정수입력(1~255사이, 0은종료)0
10진수를 입력받아 2진수 출력하기
Ex07.py

```
# 10진수를 입력받아 2진수로 출력하기
while True:
    n = int(input('정수입력(1~255사이, 0은종료)'))
    if not n:
        break
    if n<0 or n>255:  # 0x00 ~ 0xFF 사이의 숫자
        print('올바른 범위의 정수를 입력하시오')
        continue

    mask = 0b10000000  # 8비트 중 상위 1비트만 1인 수
    print('사용자 : {}을 2진수로 출력하면 0b'.format(n), end='')
    # 8번 반복
    # 1이 오른쪽으로 1칸씩 이동하며 mask와 and 연산을 수행
    # 지정 위치의 값이 1일때만 같은 수가되고 0이면 0이된다.
    # 같은때는 1을 다를때는 0을 출력하면 2진수
    for i in range(0,8):
        print('1' if (mask & n) == mask else '0', end='')
        mask = mask >> 1
    print()
    print('파이썬 : {0}을 2진수로 출력하면 {1}'.format(n, bin(n)))
```

실행 결과

정수입력(1~255사이, 0은종료)1
사용자 : 1을 2진수로 출력하면 0b00000001
파이썬 : 1을 2진수로 출력하면 0b1
정수입력(1~255사이, 0은종료)2
사용자 : 2을 2진수로 출력하면 0b00000010
파이썬 : 2을 2진수로 출력하면 0b10
정수입력(1~255사이, 0은종료)3
사용자 : 3을 2진수로 출력하면 0b00000011
파이썬 : 3을 2진수로 출력하면 0b11
정수입력(1~255사이, 0은종료)4
사용자 : 4을 2진수로 출력하면 0b00000100
파이썬 : 4을 2진수로 출력하면 0b100
정수입력(1~255사이, 0은종료)5
사용자 : 5을 2진수로 출력하면 0b00000101
파이썬 : 5을 2진수로 출력하면 0b101
정수입력(1~255사이, 0은종료)6
사용자 : 6을 2진수로 출력하면 0b00000110
파이썬 : 6을 2진수로 출력하면 0b110
정수입력(1~255사이, 0은종료)7
사용자 : 7을 2진수로 출력하면 0b00000111
파이썬 : 7을 2진수로 출력하면 0b111
정수입력(1~255사이, 0은종료)8
사용자 : 8을 2진수로 출력하면 0b00001000
파이썬 : 8을 2진수로 출력하면 0b1000
정수입력(1~255사이, 0은종료)255
사용자 : 255을 2진수로 출력하면 0b11111111
파이썬 : 255을 2진수로 출력하면 0b11111111
정수입력(1~255사이, 0은종료)0
색상 값 분리하기/결합하기
색상 값은 어떻게 나타낼까?
대부분 4Byte로 Alpha, Red, Green, Blue를 이용하여 나타냅니다.
2진수 4자리는 16진수 1자리이다. 1Byte는 8Bit이므로 16진수 2자리로 표현 가능합니다.
and 연산은 값을 지울 때 사용합니다.
0으로 &연산하면 0이고 1로 &연산하면 통과입니다.
or 연산은 값을 설정할 때 사용합니다.
0으로 |연산하면 통과이고 1로 |연산하면 1입니다.
Ex08.py

```
# 색상 값은 어떻게 나타낼까?
# 대부분 4Byte로 Alpha, Red, Green, Blue를 이용하여 나타낸다.
```

```
# 색상값이 AABBCCDD일때 각각 1바이트씩 분리하고 다시 결합해 보자
# 2진수 4자리는 16진수 1자리이다. 1Byte는 8Bit이므로 16진수 2자리로 표현 가능하다.
# and 연산은 값을 지울 때 사용합니다.
# 0으로 &연산하면 0이고 1로 &연산하면 통과입니다.
# or 연산은 값을 설정할 때 사용합니다.
# 0으로 |연산하면 통과이고 1로 |연산하면 1입니다.

# 색상분리
color = 0xAABBCCDD
alpha = color >> 24 & 0xFF
red = color >> 16 & 0xFF
green = color >> 8 & 0xFF
blue = color & 0xFF

print(color, hex(color))
print(alpha, hex(alpha))
print(red, hex(red))
print(green, hex(green))
print(blue, hex(blue))

# 색상 결합
color2 = 0
print(color2, hex(color2))
color2 = color2 | alpha << 24
print(color2, hex(color2))
color2 = color2 | red << 16
print(color2, hex(color2))
color2 = color2 | green << 8
print(color2, hex(color2))
color2 = color2 | blue
print(color2, hex(color2))

# red값 만 삭제
color2 = color2 & 0xFF00FFFF
print(color2, hex(color2))

# red값을 56으로 설정
color2 = color2 | 0x00560000
print(color2, hex(color2))
```

실행 결과

```
2864434397 0xaabbccdd
170 0xaa
187 0xbb
204 0xcc
221 0xdd
0 0x0
2852126720 0xaa000000
2864381952 0xaabb0000
2864434176 0xaabbcc00
2864434397 0xaabbccdd
2852179165 0xaa00ccdd
2857815261 0xaa56ccdd
```

테트리스 벽돌 출력하기

Ex08.py

```
# 테트리스 벽돌을 출력해보자
brick = (0x0660, 0x0F00, 0x2222, 0x6220, 0x4460, 0x4640, 0x7200, 0x6300,0x4620)
for i in brick:
    mask = 0b1000000000000000
    for j in range(0,16):
        print('■' if (mask & i) == mask else '□', end='')
        if (j+1)%4==0:
            print()
        mask >>= 1
    print()
```

실행 결과

```
□□□□
□■■□
□■■□
□□□□

□□□□
■■■■
□□□□
□□□□

□□■□
□□■□
□□■□
□□■□
```

□■■□
□□■□
□□■□
□□□□

□■□□
□■□□
□■■□
□□□□

□■□□
□■■□
□■□□
□□□□

□■■■
□□■□
□□□□
□□□□

□■■□
□□■■
□□□□
□□□□

□■□□
□■■□
□□■□
□□□□

아직까지 이진수와 16진수 그리고 반복문과 제어문을 모르기 때문에 어려울 것입니다. 하지만 많은 응용프로그램에서 자주 사용되는 연산자 입니다. 숙지하시면 일반 프로그램 보다도 적은 소스로 우수한 성능을 발휘할 수 있는 연산자들입니다. 기본 명령어들을 숙지하시고 다시한번 보시기 바랍니다.
마지막 편집일시 : 2018년 7월 12일 10:43
---
06. 복합 대입 연산자(Assignment Operators)
1. 복합 대입 연산자(Assignment Operators)
연산자 동치
a += b a = a + b

a -= b a = a - b
a *= b a = a * b
a /= b a = a / b
a %= b a = a % b
a //= ba = a // b
a **= ba = a ** b
a &= b a = a & b
a |= b a = a | b
a^= b a = a ^ b
a <<= ba = a << b
a >>= ba = a >>b
복합 대입 연산자는 연산과 할당을 합쳐놓은 것입니다.
사용하면 식을 간결하게 사용이 가능합니다.
변수가 이전에 가졌던 값을 수정하여 할당하는 일 같은 것에 쓸 수 있습니다.
너무 많이 쓰다보면 코드를 읽기가 어려워질 수도 있습니다.
기본 연산
Ex01.py

```
# 복합 대입 연산자(Assignment Operators)
a = 1
print('a =', a)
a = a + 1
print('a =', a)
a += 1
print('a =', a)
a -= 1
print('a =', a)
a *= 2
print('a =', a)
a //= 2
print('a =', a)
a %= 5
print('a =', a)
a <<= 3
print('a =', a)
a >>= 2
print('a =', a)
a **= 2
print('a =', a)
a &= 0b101010
print('a =', a)
```

```
a |= 0b11110000
print('a =', a)
a ^= 0b10101010
print('a =', a)
```

실행 결과

```
a = 1
a = 2
a = 3
a = 2
a = 4
a = 2
a = 2
a = 16
a = 4
a = 16
a = 0
a = 240
a = 90
```

2. 복합 대입 연산자(Assignment Operators) 활용

a *= 3+2의 결과는?(연산순서 주의)

Ex02.py

```
# a *= 3+2의 결과는?(연산순서 주의)
a = 10
print('a =', a)
a *= 3 + 2
# a = a * 3 + 2 일까?
# a = a * (3 + 2) 일까?
print('a =', a)
# 대입 연산자 이므로 우변이 제일 먼저 계산된다.
```

실행 결과

```
a = 10
a = 50
```

합계 구하기

Ex03.py

```
i = 1
sum = 0
while i <= 100:
```

```
    sum += i
    i += 1
print('1~100의 전체 합 :', sum)

odd = 0
i = 1
while i <= 100:
    odd += i
    i += 2
print('1~100의 홀수 합 :', odd)

even = 0
i = 2
while i <= 100:
    even += i
    i += 2
print('1~100의 짝수 합 :', even)
print()

# 한번에 구하기
odd = even = 0
i = 1
while i<100:
    odd += i
    i += 1
    even += i
    i += 1
sum = odd + even
print('1~100의 홀수 합 :', odd)
print('1~100의 짝수 합 :', even)
print('1~100의 전체 합 :', sum)
```

실행 결과

```
1~100의 전체 합 : 5050
1~100의 홀수 합 : 2500
1~100의 짝수 합 : 2550

1~100의 홀수 합 : 2500
1~100의 짝수 합 : 2550
1~100의 전체 합 : 5050
```

마지막 편집일시 : 2018년 7월 12일 11:27 오

---
07. 멤버 연산자(Membership Operators)
1. 멤버 연산자(Membership Operators)
in 연산자 : 포함하는지 검사합니다.
not in 연산자 : 포함되어 있지 않은지 검사 합니다.

포함 여부를 판단하는 연산자 입니다.
기본 연산
Ex01.py

```
list1 = [2, 3, 4, 5, 6, 7, 8]
print(list1)
print()
print('1은 있다' if 1 in list1 else '1이 없다')
print('3은 있다' if 3 in list1 else '3이 없다')
print()
print('1은 없다' if 1 not in list1 else '1이 있다')
print('3은 없다' if 3 not in list1 else '3이 있다')
```

실행 결과

```
[2, 3, 4, 5, 6, 7, 8]

1이 없다
3은 있다

1은 없다
3이 있다
```

2. 멤버 연산자(Membership Operators) 활용
회원검색
Ex02.py

```
members = ['초롱', '아름', '별님', '보람', '슬기', '하나','고와', '봄내', '힘차']

while True:
    name = input('찾는 회원의 이름은(Enter 종료)?')
    if name=='':
        break
    print('회원' if name in members else '손님', '입니다.')
```

실행 결과

```
찾는 회원의 이름은(Enter 종료)?아름
```

회원 입니다.
찾는 회원의 이름은(Enter 종료)?철수
손님 입니다.
찾는 회원의 이름은(Enter 종료)?
마지막 편집일시 : 2018년 7월 12일 12:11
---
08. 아이디 연산자(Identity Operators)
1. 아이디 연산자(Identity Operators)
is : 양쪽 Operand가 동일한 Object를 가리키는지 아닌지를 검사합니다.
is not : 양쪽 Operand가 다른 Object를 가리키는지 아닌지를 검사합니다.

동일한 객체 여부를 판별하는 연산자입니다.
id() 함수는 객체를 입력값으로 받아서 객체의 고유값(레퍼런스)을 반환하는 함수입니다.
id는 파이썬이 객체를 구별하기 위해서 부여하는 일련번호입니다. 숫자로서 의미는 없습니다.
id는 동일한 객체 여부를 판별할 때 사용합니다.
기본 연산
Ex01.py

```
# 아이디 연산자(Identity Operators)
a = 10
b = 10
c = 11
print(a, 'id =', id(a))
print(b, 'id =', id(b))
print(c, 'id =', id(c))
print('같은 객체' if a is b else '다른 객체')
print('같은 객체' if a is c else '다른 객체')
print('다른 객체' if a is not c else '같은 객체')
print()

name1 = '한사람'
name2 = '한사람'
name3 = '두사람'
print(name1, 'id =', id(name1))
print(name2, 'id =', id(name2))
print(name3, 'id =', id(name3))
print('같은 객체' if name1 is name2 else '다른 객체')
print('같은 객체' if name1 is name3 else '다른 객체')
print('다른 객체' if name1 is not name3 else '같은 객체')
print()
```

```
name4 = name3
print(name3, 'id =', id(name3))
print(name4, 'id =', id(name4))
print('같은 객체' if name3 is name4 else '다른  객체')
print()

name3 = '세사람'
print(name3, 'id =', id(name3))
print(name4, 'id =', id(name4))
print('같은 객체' if name3 is name4 else '다른  객체')
print()
```

실행 결과

```
10 id = 1879076608
10 id = 1879076608
11 id = 1879076640
같은 객체
다른 객체
다른 객체

한사람 id = 35783088
한사람 id = 35783088
두사람 id = 35783280
같은 객체
다른 객체
다른 객체

두사람 id = 35783280
두사람 id = 35783280
같은 객체

세사람 id = 35783376
두사람 id = 35783280
다른  객체
```

객체에 관해서는 뒤에 자세하게 설명하도록 하겠습니다. 여기서는 이런 연산자가 있다 정도만 알고 지나가도록 하겠습니다.

마지막 편집일시 : 2018년 7월 12일 12:36

---

09. 연산 우선 순위(Operators Precedence)

* 연산 순서를 잘못 이해하면 원하지 않는 결과를 나타낼 수 있습니다.

* 연산 순서를 잘 기억하여 연산 순서를 변경할 때는 괄호를 이용하시기 바랍니다.
순위 연산자 설명
1 (), {}, [] Tuple, Set, List, Dictionary
2 collection[index]
collection[index1, index2]
function(aguments ...)
object.attribute 컬렉션의 첨자
슬라이싱
함수의 인수
객체의 속성 등
3 ** 거듭제곱
4 + , - , ~ 단항 연산자(부호, bitwise not)
5 * / // % 곱하기, 나누기, 정수 몫, 나머지
6 + - 더하기, 빼기
7 << >> 시프트 연산
8 & bitwise and
9 ^ bitwise xor
10
11 in, not in
is, is not
<, <=, >, >=, ==, != 멤버 연산자
아이디 연산자
관계 연산자
12 not 논리 not
13 and 논리 and
14 or 논리 or
15 if else 삼항 연산자
16 lambda 람다 표현식
마지막 편집일시 : 2018년 7월 12일 2:15
---

05. Python(파이썬) 조건문 - if문

프로그램을 작성하면 위에서부터 아래로 순서대로 실행됩니다. 하지만 프로그래이 꼭 순서대로만 실행되어서는 올바른 결과를 얻을 수 없습니다. 경우에 따라서 실행 순서를 변경해야 하는 경우가 생기는데 이때에 조건문을 사용합니다.
예를 들어 늦은 밤 집에가는데 왼쪽길은 어둡지만 빠르게 갈 수 있고 오른쪽 길은 밝지만 돌아간다면 어떻게 할까요? 사람은 생각을 할 수 잇는 동물이므로 자신이 스스로 판단을 하여 길을 선택합니다. 하지만 컴퓨터는 생각을 할 수 없으므로 프로그램에 직접 이럴때는 이렇게 하고 저럴때는 저렇게하라라고 꼭 지시를 해주어야만 합니다. 이때 사용되는 명령어가 조건문입니다.
※ 다른 언어에는 switch문이 있지만 Python에는 switch문은 없습니다. if문 하나만으로도 모

든 조건문을 사용할 수 있기때문에 굳이 switc문의 필요성을 느끼지 못합니다. 다른 언어에서도 switch문을 사용하지 않고도 모두 처리가 가능합니다.

마지막 편집일시 : 2018년 7월 12일 5:58

---

01. 조건이 참인 경우만 실행

1. 조건이 참인 경우에만 실행하기

```
if 조건:
    참인경우 실행할 명령 1
    참인경우 실행할 명령 2
    참인경우 실행할 명령 3
    ....
다음 명령
```

조건이 참인 경우에만 실행할 때 if조건문을 사용합니다.
참인 경우에 실행할 명령들은 들여쓰기를 합니다. 대부분 4칸 들여쓰기를 합니다.
들여쓰기가 끝나면 조건문을 종료합니다.
조건이 거짓이면 아무런 일도 하지 않습니다.
기본 사용
Ex01.py

```
# 조건이 참이면 if문 아래 들여쓰기 한 문장들이 실행됩니다.

a = 4
if a == 4:
    # 조건이 참이므로 실행됩니다.
    print('한국 사람이 싫어하는 숫자 입니다.')
    print('그래서 4층을 "F"층으로 표기하기도 합니다.')
    print()

if a == 1:
    # 조건이 거짓이므로 아무런 일도 하지 않습니다.
    print('한국 사람이 좋아하는 숫자 입니다.')
    print('1등 만능주의입니다.')
    print()

a = 3
if a == 4:
    print('한국 사람이 싫어하는 숫자 입니다.')
# 이 줄은 if문에 포함되지 않습니다. 그래서 실행됩니다.
print('그럼 나는 실행이 될까요? 안될까요?')
```

실행 결과

한국 사람이 싫어하는 숫자 입니다.
그래서 4층을 "F"층으로 표기하기도 합니다.

그럼 나는 실행이 될까요? 안될까요?
2. if문에서 자료형의 참/거짓
숫자형 : 0은 거짓, 그외의 숫자는 참
문자열 : 빈문자열만 거짓
컬렉션 : {}, [], ()만 거짓
None : 거짓

Ex02.py

```
n = 0
if not n:
    print('조건문에서 0은 "거짓"입니다.')

n = 1
if n:
    print('조건문에서 1은 "참"입니다.')

string = ''
if not string:
    print('조건문에서 빈문자열은 "거짓"입니다.')

string = ' '
if string:
    print('조건문에서 " "(공백)은 "참"입니다.')

if not None:
    print('조건문에서 "None"은 "거짓"입니다.')

my_tuple = ()
print(type(my_tuple), my_tuple)
if not my_tuple:
    print('조건문에서 원소가 없는 "tuple"은 "거짓"입니다.')

my_list = []
print(type(my_list), my_list)
if not my_list:
    print('조건문에서 원소가 없는 "list"는 "거짓"입니다.')
```

```
my_set = set()
print(type(my_set), my_set)
if not my_set:
    print('조건문에서 원소가 없는 "set"은 "거짓"입니다.')

my_dict = {}
print(type(my_dict), my_dict)
if not my_dict:
    print('조건문에서 원소가 없는 "dictionary"는 "거짓"입니다.')
```

실행 결과

```
조건문에서 0은 "거짓"입니다.
조건문에서 1은 "참"입니다.
조건문에서 빈문자열은 "거짓"입니다.
조건문에서 " "(공백)은 "참"입니다.
조건문에서 "None"은 "거짓"입니다.
<class 'tuple'> ()
조건문에서 원소가 없는 "tuple"은 "거짓"입니다.
<class 'list'> []
조건문에서 원소가 없는 "list"는 "거짓"입니다.
<class 'set'> set()
조건문에서 원소가 없는 "set"은 "거짓"입니다.
<class 'dict'> {}
조건문에서 원소가 없는 "dictionary"는 "거짓"입니다.
```

3. 활용

양수, 0, 음수 판단

Ex03.py

```
# 양수, 0, 음수 판단
# 무한루프입니다. 될 수 있으면 쓰지 않는것이 좋습니다.
while True:
    num = int(input('정수를 입력하세요(0은 종료)'))
    if num < 0:
        print('{}은 음수입니다.'.format(num))
    if num == 0:
        print('{}은 "Zero"입니다.'.format(num))
        break
    if num > 0:
        print('{}은 양수입니다.'.format(num))
```

실행 결과

정수를 입력하세요(0은 종료)3
3은 양수입니다.
정수를 입력하세요(0은 종료)-4
-4은 음수입니다.
정수를 입력하세요(0은 종료)0
0은 "Zero"입니다.
합격/불합격 판단
Ex04.py

```
# 합격/불합격 판단
while True:
    num = int(input('점수를 입력하세요(음수는 종료)'))
    if num < 0:
        break

    if num > 100:
        print('올바른 점수가 아닙니다.\n0 ~ 100 사이의 점수를 입력하세요')
        continue  # 반복문의 처음으로 가라!!
    if num < 60:
        print('점수가 {}점이므로 "불합격"입니다.'.format(num))
    if num >= 60:
        print('점수가 {}점이므로 "합격"입니다.'.format(num))
```

실행 결과

점수를 입력하세요(음수는 종료)234
올바른 점수가 아닙니다.
0 ~ 100 사이의 점수를 입력하세요
점수를 입력하세요(음수는 종료)56
점수가 56점이므로 "불합격"입니다.
점수를 입력하세요(음수는 종료)78
점수가 78점이므로 "합격"입니다.
점수를 입력하세요(음수는 종료)-1
그런데 위의 예제들을 따라하면서 느끼는 의문점은 없었나요? 위의 예제를 보면 무언가 냄새가 난다는 생각이 들지 않나요? 프로그램계에서는 “코드 스멜(code smell)”이라는 용어가 있습니다. 코드 스멜(code smell←코드 냄새)은 컴퓨터 프로그래밍 코드에서 더 심오한 문제를 일으킬 가능성이 있는 프로그램 소스 코드의 증상을 나타내는 용어입니다. 무슨 냄새가 날까요? 더 좋은 방법은 없을까요? 다음 절을 보겠습니다.
마지막 편집일시 : 2018년 7월 12일 4:20
---
02. 참일 때와 거짓일 때 다른 코드 실행

1. if ~ else 문

```
if 조건:      참인 경우 실행할 명령 1
    참인 경우 실행할 명령 2
    참인 경우 실행할 명령 3
else:      거짓인 경우 실행할 명령 1
    거짓인 경우 실행할 명령 2
    거짓인 경우 실행할 명령 3
다음 명령
```

조건에 따라 참인 경우와 거짓인 경우에 다른 명령을 실행할 때 사용합니다.
들여쓰기를 합니다. 대부분 4칸 들여쓰기를 합니다.
들여쓰기가 끝나면 조건문을 종료합니다.
기본 사용
Ex05.py

```
a = 10
if a < 60:
    print("{}점이므로 불합격입니다.".format(a))
else:
    print("{}점이므로 합격입니다.".format(a))

a = 67
if a < 60:
    print("{}점이므로 불합격입니다.".format(a))
else:
    print("{}점이므로 합격입니다.".format(a))
```

실행 결과

```
10점이므로 불합격입니다.
67점이므로 합격입니다.
```

2. 활용
양수, 0, 음수 판단
Ex06.py

```
# 양수, 0, 음수 판단
while True:
    num = int(input('정수를 입력하세요(0은 종료)'))
    if num == 0:
        print('{}은 "Zero"입니다.'.format(num))
        break
    if num < 0:
```

```
        print('{}은 음수입니다.'.format(num))
    else:
        print('{}은 양수입니다.'.format(num))
```

실행 결과

```
정수를 입력하세요(0은 종료)56
56은 양수입니다.
정수를 입력하세요(0은 종료)-34
-34은 음수입니다.
정수를 입력하세요(0은 종료)0
0은 "Zero"입니다.
```

홀수/짝수 판별

Ex07.py

```
# 홀수/짝수 판단
while True:
    num = int(input('정수를 입력하세요(0은 종료)'))
    if num == 0:
        break
    if num < 0:
        print("양수로 입력하세요")
        continue

    if num%2:
        print('{}은 홀수입니다.'.format(num))
    else:
        print('{}은 짝수입니다.'.format(num))
```

실행 결과

```
정수를 입력하세요(0은 종료)-9
양수로 입력하세요
정수를 입력하세요(0은 종료)1
1은 홀수입니다.
정수를 입력하세요(0은 종료)2
2은 짝수입니다.
정수를 입력하세요(0은 종료)0
```

합격/불합격 판정

Ex08.py

```
# 합격/불합격 판단
while True:
```

```
    num = int(input('점수를 입력하세요(음수는 종료)'))
    if num < 0:
        break

    if num > 100:
        print('올바른 점수가 아닙니다.\n0 ~ 100 사이의 점수를 입력하세요')
        continue  # 반복문의 처음으로 가라!!
    if num < 60:
        print('점수가 {}점이므로 "불합격"입니다.'.format(num))
    else:
        print('점수가 {}점이므로 "합격"입니다.'.format(num))
```

실행 결과

```
점수를 입력하세요(음수는 종료)345
올바른 점수가 아닙니다.
0 ~ 100 사이의 점수를 입력하세요
점수를 입력하세요(음수는 종료)56
점수가 56점이므로 "불합격"입니다.
점수를 입력하세요(음수는 종료)78
점수가 78점이므로 "합격"입니다.
점수를 입력하세요(음수는 종료)-1
```

체 질량지수(BMI) 판별

Ex09.py

```
# 체 질량지수(BMI) 판별
# 체질량지수 = 체중/키의제곱(m)
# 저체중 : 18.5 미만
# 정상 : 18.5 ~ 24.9
# 과체중 : 24.5 ~ 29.9
# 비만 : 30.0 이상
while True:
    weight = float(input('체중을 입력하세요(0은 종료)'))
    if weight==0:
        break
    if weight < 0:
        print('올바른 값이 아닙니다.\n다시 입력하세요')
        continue  # 반복문의 처음으로 가라!!

    height = float(input('신장을 입력하세요(0은 종료)'))
    if height==0:
        break
```

```
    if height < 0:
        print('올바른 값이 아닙니다.\n다시 입력하세요')
        continue  # 반복문의 처음으로 가라!!

    bmi = weight/(height/100)**2

    if bmi<18.5:
        print('BMI값이 {:.2f}이므로 "저체중"입니다.'.format(bmi))
    else:
        if bmi >= 18.5 and bmi <=24.9:
            print('BMI값이 {:.2f}이므로 "정상"입니다.'.format(bmi))
        else:
            if bmi >= 24.5 and bmi <= 29.9:
                print('BMI값이 {:.2f}이므로 "과체중"입니다.'.format(bmi))
            else:
                print('BMI값이 {:.2f}이므로 "비만"입니다.'.format(bmi))
```

실행 결과

```
체중을 입력하세요(0은 종료)65
신장을 입력하세요(0은 종료)164
BMI값이 24.17이므로 "정상"입니다.
체중을 입력하세요(0은 종료)80
신장을 입력하세요(0은 종료)175
BMI값이 26.12이므로 "과체중"입니다.
체중을 입력하세요(0은 종료)101
신장을 입력하세요(0은 종료)178
BMI값이 31.88이므로 "비만"입니다.
체중을 입력하세요(0은 종료)-1
올바른 값이 아닙니다.
다시 입력하세요
체중을 입력하세요(0은 종료)0
```

위의 코드 역시 코드 스멜(code smell←코드 냄새)입니다.

년도를 입력받아 윤/평년 판단

Ex10.py

```
# 윤년 판단 규칙
# 1. 년도가 4로 나누어떨어지는 해는 윤년입니다. (1996년, 2004년, 2008년, 2012년 …)
# 2. 이 중에서 100으로 나누어떨어지는 해는 평년입니다. (1900년, 2100년, 2200년, 2300년 …)
# 3. 그중에 400으로 나누어떨어지는 해는 윤년입니다. (1600년, 2000년, 2400년 …)
```

```
while True:
    year = int(input('년도를 입력하시오(0은 종료)'))
    if not year:
        break

    # 이상한(?) 코드
    if not year%400:
        print('{}년은 "윤년"입니다.'.format(year))
    else:
        if not year%4:
            if year%100:
                print('{}년은 "윤년"입니다.'.format(year))
            else:
                print('{}년은 "평년"입니다.'.format(year))
        else:
            print('{}년은 "평년"입니다.'.format(year))

    # 조금 향상된 코드
    isLeapYear = False
    if not year%400:
        isLeapYear = True
    else:
        if not year%4:
            if year%100:
                isLeapYear = True
    print('{0}년은 "{1}년"입니다.'.format(year, '윤' if isLeapYear else '평'))

    # 현명한(?) 코드
    isLeapYear = year%400==0 or year%4==0 and year%100!=0
    print('{0}년은 "{1}년"입니다.'.format(year, '윤' if isLeapYear else '평'))

    # 현명한(?) 코드
    print('{0}년은 "{1}년"입니다.'
          .format(year, '윤' if year%400==0 or year%4==0 and year%100!=0 else
'평'))
```

실행 결과

```
년도를 입력하시오(0은 종료)1
1년은 "평년"입니다.
1년은 "평년"입니다.
```

1년은 "평년"입니다.
1년은 "평년"입니다.
년도를 입력하시오(0은 종료)4
4년은 "윤년"입니다.
4년은 "윤년"입니다.
4년은 "윤년"입니다.
4년은 "윤년"입니다.
년도를 입력하시오(0은 종료)100
100년은 "평년"입니다.
100년은 "평년"입니다.
100년은 "평년"입니다.
100년은 "평년"입니다.
년도를 입력하시오(0은 종료)400
400년은 "윤년"입니다.
400년은 "윤년"입니다.
400년은 "윤년"입니다.
400년은 "윤년"입니다.
년도를 입력하시오(0은 종료)0
위의 코드중에서 어떤 방법이 제일 좋은 방법일까요? 한 번 고민해 보시기 바랍니다.
위의 예제는 조건문이 얼마든지 중첩될 수 있다는 것을 보여주려고 일부러 만든 코드입니다.
마지막 편집일시 : 2018년 7월 12일 5:06
---
03. 조건이 복잡한 경우
1. if ~ elif ~ else

```
if 조건1:
    조건1이 참인 경우 실행할 명령....
elif 조건2:
    조건2가 참인 경우 실행할 명령....
.
.
else:
    모든 조건이 거짓인 경우 실행할 명령....
다음 명령
```

여러 개의 조건이 있을 경우 사용합니다.
모든 명령문은 얼마든지 중첩이 가능합니다.
기본 사용
Ex11.py

```
# 양수, 0, 음수 판단
while True:
```

```
    num = int(input('정수를 입력하세요(0은 종료)'))
    if num < 0:
        print('{}은 음수입니다.'.format(num))
    elif num > 0:
        print('{}은 양수입니다.'.format(num))
    else:
        print('{}은 "Zero"입니다.'.format(num))
        break
```

실행 결과

```
정수를 입력하세요(0은 종료)43
43은 양수입니다.
정수를 입력하세요(0은 종료)-4
-4은 음수입니다.
정수를 입력하세요(0은 종료)0
0은 "Zero"입니다.
```

2. 활용

체 질량지수(BMI) 판별

Ex12.py

```
# 체 질량지수(BMI) 판별
# 체질량지수 = 체중/키의제곱(m)
# 저체중 : 18.5 미만
# 정상 : 18.5 ~ 24.9
# 과체중 : 24.5 ~ 29.9
# 비만 : 30.0 이상
while True:
    weight = float(input('체중을 입력하세요(0은 종료)'))
    if weight==0:
        break
    if weight < 0:
        print('올바른 값이 아닙니다.\n다시 입력하세요')
        continue  # 반복문의 처음으로 가라!!

    height = float(input('신장을 입력하세요(0은 종료)'))
    if height==0:
        break
    if height < 0:
        print('올바른 값이 아닙니다.\n다시 입력하세요')
        continue  # 반복문의 처음으로 가라!!
```

```
    bmi = weight/(height/100)**2

    if bmi<18.5:
        print('BMI 값이 {:.2f}이므로 "저체중"입니다.'.format(bmi))
    elif bmi >= 18.5 and bmi <=24.9:
        print('BMI 값이 {:.2f}이므로 "정상"입니다.'.format(bmi))
    elif bmi >= 24.5 and bmi <= 29.9:
        print('BMI 값이 {:.2f}이므로 "과체중"입니다.'.format(bmi))
    else:
        print('BMI 값이 {:.2f}이므로 "비만"입니다.'.format(bmi))

    result = '비만'
    if bmi<18.5:
        result = '저체중'
    elif 18.5 <= bmi <= 24.9:
        result = '정상'
    elif 24.5 <= bmi <= 29.9:
        result = '과체중'
    else:
        result = '비만'

    print('BMI 값이 {0:.2f}이므로 "{1}"입니다.'.format(bmi, result))

    result = '비만'
    if bmi < 18.5:
        result = '저체중'
    elif bmi <= 24.9:
        result = '정상'
    elif bmi <= 29.9:
        result = '과체중'

    print('BMI 값이 {0:.2f}이므로 "{1}"입니다.'.format(bmi, result))
```

실행 결과

```
체중을 입력하세요(0은 종료)67
신장을 입력하세요(0은 종료)170
BMI 값이 23.18이므로 "정상"입니다.
BMI 값이 23.18이므로 "정상"입니다.
BMI 값이 23.18이므로 "정상"입니다.
체중을 입력하세요(0은 종료)98
신장을 입력하세요(0은 종료)172.5
```

BMI 값이 32.93이므로 "비만"입니다.
BMI 값이 32.93이므로 "비만"입니다.
BMI 값이 32.93이므로 "비만"입니다.
체중을 입력하세요(0은 종료)0
3가지 방법을 사용했습니다. 어떤 방법이 제일 좋은 방법일까요? 한 번 고민해 보시기 바랍니다.
학점 계산하기 1
Ex13.py

```
# 학점 구하기
# 90~100 A
# 80~89  B
# 70~79  C
# 60~69  D
# 60미만 F
while True:
    num = int(input('점수를 입력하세요(음수는 종료)'))
    if num < 0:
        break

    if num > 100:
        print('올바른 점수가 아닙니다.\n0 ~ 100 사이의 점수를 입력하세요')
        continue  # 반복문의 처음으로 가라!!

    # 학점 구하기
    grade = 'F'
    if num >= 90:
        grade = 'A'
    elif num >= 80:
        grade = 'B'
    elif num >= 70:
        grade = 'C'
    elif num >= 60:
        grade = 'D'

    print('{0}점의 학점은 "{1}"입니다.'.format(num,grade))
```

실행 결과

점수를 입력하세요(음수는 종료)54
54점의 학점은 "F"입니다.
점수를 입력하세요(음수는 종료)69

```
69점의 학점은 "D"입니다.
점수를 입력하세요(음수는 종료)78
78점의 학점은 "C"입니다.
점수를 입력하세요(음수는 종료)84
84점의 학점은 "B"입니다.
점수를 입력하세요(음수는 종료)98
98점의 학점은 "A"입니다.
점수를 입력하세요(음수는 종료)123
올바른 점수가 아닙니다.
0 ~ 100 사이의 점수를 입력하세요
점수를 입력하세요(음수는 종료)0
0점의 학점은 "F"입니다.
점수를 입력하세요(음수는 종료)-1
```

학점 구하기 2

Ex14.py

```
# 학점 구하기
# 95~100 A+, 90~94 A-
# 85~89  B+, 80~84  B+
# 75~79  C+, 70~74  C-
# 65~69  D+, 60~64  D-
# 60미만 F
while True:
    num = int(input('점수를 입력하세요(음수는 종료)'))
    if num < 0:
        break

    if num > 100:
        print('올바른 점수가 아닙니다.\n0 ~ 100 사이의 점수를 입력하세요')
        continue  # 반복문의 처음으로 가라!!

    # 학점 구하기 1
    grade = 'F'
    if num >= 95:
        grade = 'A+'
    elif num >= 90:
        grade = 'A-'
    elif num >= 85:
        grade = 'B+'
    elif num >= 80:
        grade = 'B-'
```

```
    elif num >= 75:
        grade = 'C+'
    elif num >= 70:
        grade = 'C-'
    elif num >= 65:
        grade = 'D+'
    elif num >= 60:
        grade = 'D-'
    else:
        grade = 'F'

    print('{0}점의 학점은 "{1}"입니다.'.format(num,grade))

    # 학점 구하기 2
    grade = 'F'
    if num >= 90:
        grade = 'A'
    elif num >= 80:
        grade = 'B'
    elif num >= 70:
        grade = 'C'
    elif num >= 60:
        grade = 'D'

    # 1의자리가 5이상이면 +, 미만이면 -이다.
    # 예외가 두 가지있는데 100점은 0이지만 +이고 F학점은 +,-가 없다
    if num == 100:
        grade += '+'
    elif num > 60:
        if num%10 >= 5:
            grade += '+'
        else:
            grade += '-'

    print('{0}점의 학점은 "{1}"입니다.'.format(num,grade))
```

실행 결과

```
점수를 입력하세요(음수는 종료)100
100점의 학점은 "A+"입니다.
100점의 학점은 "A+"입니다.
점수를 입력하세요(음수는 종료)87
```

87점의 학점은 "B+"입니다.
87점의 학점은 "B+"입니다.
점수를 입력하세요(음수는 종료)62
62점의 학점은 "D-"입니다.
62점의 학점은 "D-"입니다.
점수를 입력하세요(음수는 종료)54
54점의 학점은 "F"입니다.
54점의 학점은 "F"입니다.
점수를 입력하세요(음수는 종료)-1
위의 코드 중에서 어떤 방법이 제일 좋은 방법일까요? 한 번 고민해 보시기 바랍니다.
마지막 편집일시 : 2018년 7월 12일 5:38
---

06. Python(파이썬) 반복문

Python(파이썬) 반복문은 두가지가 존재합니다.

1. while loops
2. for loops

마지막 편집일시 : 2018년 7월 12일 6:00

---

01. range와 enumerate 함수
1. range 함수
range 함수
range([strat,] stop [,step])
필요한 만큼의 숫자를 만들어내는 유용한 기능입니다.
for문과 함께 자주 사용되는 함수입니다.
이 함수는 입력받은 숫자에 해당되는 범위의 값을 반복 가능한 객체로 만들어 리턴합니다.

기본사용
Ex01.py

```
# range 함수
# range(stop) : 0 ~ stop-1 까지의 정수
print(range(5), type(range(5)))
print(tuple(range(5)))
print(set(range(5)))
print(list(range(5)))

for i in range(5):
    print(i, end=' ')
print()
```

```
# range(start,stop) : start ~ stop-1 까지의 정수
print(list(range(5, 10)))

# range(start,stop, step) : start 부터 step 씩 증감하며 stop-1 까지의 정수
print(list(range(2, 11, 2)))
print(list(range(1, 11, 2)))
print(list(range(9, 0, -2)))
print(list(range(10, 0, -2)))
```

실행 결과

```
range(0, 5) <class 'range'>
(0, 1, 2, 3, 4)
{0, 1, 2, 3, 4}
[0, 1, 2, 3, 4]
0 1 2 3 4
[5, 6, 7, 8, 9]
[2, 4, 6, 8, 10]
[1, 3, 5, 7, 9]
[9, 7, 5, 3, 1]
[10, 8, 6, 4, 2]
```

2. enumerate 함수

리스트가 있는 경우 순서와 리스트의 값을 전달하는 기능을 가집니다.
enumerate는 “열거하다”라는 뜻입니다.
이 함수는 순서가 있는 자료형(list, set, tuple, dictionary, string)을 입력으로 받아 인덱스 값을 포함하는 enumerate 객체를 리턴합니다.
보통 enumerate 함수는 for문과 함께 자주 사용됩니다.

Ex02.py

```
# enumerate 함수
data = enumerate((1, 2, 3))
print(data, type(data))

for i, value in data:
    print(i, ":", value)
print()

data = enumerate({1, 2, 3})
for i, value in data:
    print(i, ":", value)
print()
```

```
data = enumerate([1, 2, 3])
for i, value in data:
    print(i, ":", value)
print()

dict1 = {'이름': '한사람', '나이': 33}
data = enumerate(dict1)
for i, key in data:
    print(i, ":", key, dict1[key])
print()

data = enumerate("재미있는 파이썬")
for i, value in data:
    print(i, ":", value)
print()
```

실행 결과

```
<enumerate object at 0x0000000002424EA0> <class 'enumerate'>
0 : 1
1 : 2
2 : 3

0 : 1
1 : 2
2 : 3

0 : 1
1 : 2
2 : 3

0 : 이름 한사람
1 : 나이 33

0 : 재
1 : 미
2 : 있
3 : 는
4 :
5 : 파
```

6 : 이
7 : 썬
마지막 편집일시 : 2018년 7월 13일 5:07
---
02. while loops
1. while 반복문
while 조건:
    조건이 참인동안 실행할 명령들....

break : 반복문을 탈출 합니다.
continue : 반복문의 처음으로 갑니다.

조건이 참인 동안 반복 실행할 때 while 반복문을 사용합니다.
참인 경우에 실행할 명령들은 들여쓰기를 합니다. 대부분 4칸 들여쓰기를 합니다.
들여쓰기가 끝나면 반복문을 종료합니다.
조건이 거짓이면 아무런 일도 하지 않습니다.
반복문은 얼나든지 중첩이 가능합니다. 중첩할때 들여쓰기에 주의 하세요
반복문 내에서 break 명령을 만나면 현재의 반복문을 탈출 합니다.
반복문 내에서 continue 명령을 만나면 현재의 반복문의 처음으로 돌아가서 조건을 비교합니다.
기본 사용
Ex01.py

```
# while 반복문
i = 1
# 10번 반복 합니다.
while i <= 10:
    print('{0}번째 계단밟고'.format(i))
    i += 1
print('Python 정상에 섯다!!!!')
```
실행 결과

```
1번째 계단밟고
2번째 계단밟고
3번째 계단밟고
4번째 계단밟고
5번째 계단밟고
6번째 계단밟고
7번째 계단밟고
8번째 계단밟고
9번째 계단밟고
```

10번째 계단밟고
Python 정상에 섯다!!!!
2. while의 활용
피보나치 수(Fibonacci Numbers)를 1줄에 5개씩 30개 출력하기
피보나치 수(Fibonacci Numbers)란 1 1 2 3 5 8 .... 처럼 앞의 두개를 더해서 다음수가 되는 수열을 말합니다.

풀이 방법
1. 두개의 변수를 선언하여 0과 1로 초기화합니다.
2. 두번째 변수 값을 출력합니다.
3. 임시변수에 두번째 변수를 대입하여 보관 합니다.
4. 두개의 변수값을 더하여 두번째 변수에 대입합니다.
5. 임시 변수값을 첫번째 변수에 대입합니다.
6. 2번부터 5번을 반복 합니다.

Ex02.py

```
# 피보나치 수(Fibonacci Numbers)를 1줄에 5개씩 30개 출력하기

# 풀이 1
count = 0
first = 0
second = 1
while count<30: # 30번 반복
    print('{0:10d}'.format(second),end=' ')  # 출력
    count += 1 # 개수 증가
    if count%5 == 0: # 개수가 5의 배수면 줄바꿈
        print()
    temp = second # 임시변수에 피신
    second = first + second # 두개 더한 값을 두번째 변수에
    first = temp # 첫번째 변수에 두번째 변수가 가지고 있던값 대입


# 풀이 2
# 임시 변수를 왜 사용했을까?
# second값이 지워지기 때문에 지워지기 전에 피신을 시킨 것이다.
# 지워지기 전의 값을 알 수 있다면 임시변수를 사용하지 않아도 된다.
count = 0
first = 0
second = 1
while count<30: # 30번 반복
```

```
    print('{0:10d}'.format(second),end=' ')  # 출력
    count += 1 # 개수 증가
    if count%5 == 0: # 개수가 5의 배수면 줄바꿈
        print()
    second = first + second # 두개 더한 값을 두번째 변수에
    first = second - first  # second값이 first만큼 늘어났으니 빼주면 원래 값
```

실행 결과

```
         1         1         2         3         5
         8        13        21        34        55
        89       144       233       377       610
       987      1597      2584      4181      6765
     10946     17711     28657     46368     75025
    121393    196418    317811    514229    832040
         1         1         2         3         5
         8        13        21        34        55
        89       144       233       377       610
       987      1597      2584      4181      6765
     10946     17711     28657     46368     75025
    121393    196418    317811    514229    832040
```

약수와 약수 개수 구하기

Ex03.py

```
# 약수와 약수 개수 구하기
while True:
    num = int(input('정수를 입력하시오(0은종료)'))
    if not num:
        break  # 반복문을 탈출합니다.
    if num < 0:
        print('양의 정수로 입력하시오')
        continue  # 반복문의 처음으로 갑니다.
    count = 0  # 약수의 개수를 세어줄 변수
    i = 1  # 1~num까지 증가할 변수
    while i <= num:
        if num % i == 0:  # 나누어지면 약수
            print('{0:5}'.format(i), end=' ')
            count += 1
        i += 1  # 1증가
    print()
    print('{0}의 약수의 개수 {1}개입니다.'.format(num, count))
```

실행 결과

정수를 입력하시오(0은종료)88
    1   2   4   8   11   22   44   88
88의 약수의 개수 8개입니다.
정수를 입력하시오(0은종료)90
    1   2   3   5   6   9   10   15   18   30   45   90
90의 약수의 개수 12개입니다.
정수를 입력하시오(0은종료)-1
양의 정수로 입력하시오
정수를 입력하시오(0은종료)0
제곱수 구하기
위의 문제에서 약수의 개수가 홀수인 수는 제곱수임을 알 수 있다.
4, 9, 16, 25 ....
Ex04.py

```
# 1~100 사이의 모든 제곱수 구하기
# 1, 4, 9, 16,25 ....
num = 1
while num <= 100:
    count = 0  # 약수의 개수를 세어줄 변수
    i = 1  # 1~num까지 증가할 변수
    while i <= num:
        if num % i == 0:  # 나누어지면 약수
            count += 1
        i += 1  # 1증가
    if count % 2:  # 약수의 개수가 홀수일 때만 출력
        print('{0}의 약수가 {1}개이므로 "제곱수"입니다.'.format(num, count))

    num += 1  # 100까지 증가
```

실행 결과

1의 약수가 1개이므로 "제곱수"입니다.
4의 약수가 3개이므로 "제곱수"입니다.
9의 약수가 3개이므로 "제곱수"입니다.
16의 약수가 5개이므로 "제곱수"입니다.
25의 약수가 3개이므로 "제곱수"입니다.
36의 약수가 9개이므로 "제곱수"입니다.
49의 약수가 3개이므로 "제곱수"입니다.
64의 약수가 7개이므로 "제곱수"입니다.
81의 약수가 5개이므로 "제곱수"입니다.
100의 약수가 9개이므로 "제곱수"입니다.

소수 구하기 1
소수란 1과 자신만을 약수로 가지는 수 입니다.
2, 3, 5, 7, 11, ...
위의 문제를 조금 변경해서 풀어 보겠습니다. 효율성은 떨어집니다.
Ex05.py

```
# 2~100 사이의 모든 소수 구하기 1
# 소수란 1과 자신만을 소수로 가지는 수입니다.
num = 2
while num <= 100:
    count = 0  # 약수의 개수를 세어줄 변수
    i = 1  # 1~num까지 증가할 변수
    while i <= num:
        if num % i == 0:  # 나누어지면 약수
            count += 1
        i += 1  # 1증가
    if count == 2:  # 약수의 개수가 2개면 출력
        print('{0}의 약수가 {1}개이므로 "소수"입니다.'.format(num, count))

    num += 1  # 100까지 증가
```

실행 결과

```
2의 약수가 2개이므로 "소수"입니다.
3의 약수가 2개이므로 "소수"입니다.
5의 약수가 2개이므로 "소수"입니다.
7의 약수가 2개이므로 "소수"입니다.
11의 약수가 2개이므로 "소수"입니다.
13의 약수가 2개이므로 "소수"입니다.
17의 약수가 2개이므로 "소수"입니다.
19의 약수가 2개이므로 "소수"입니다.
23의 약수가 2개이므로 "소수"입니다.
29의 약수가 2개이므로 "소수"입니다.
31의 약수가 2개이므로 "소수"입니다.
37의 약수가 2개이므로 "소수"입니다.
41의 약수가 2개이므로 "소수"입니다.
43의 약수가 2개이므로 "소수"입니다.
47의 약수가 2개이므로 "소수"입니다.
53의 약수가 2개이므로 "소수"입니다.
59의 약수가 2개이므로 "소수"입니다.
61의 약수가 2개이므로 "소수"입니다.
67의 약수가 2개이므로 "소수"입니다.
```

71의 약수가 2개이므로 "소수"입니다.
73의 약수가 2개이므로 "소수"입니다.
79의 약수가 2개이므로 "소수"입니다.
83의 약수가 2개이므로 "소수"입니다.
89의 약수가 2개이므로 "소수"입니다.
97의 약수가 2개이므로 "소수"입니다.
소수 구하기 2
임의의 수를 2부터 나누어질 때까지 반복합니다. 나누어 졌을때 자기 자신이면 중간에 약수가 없는것 입니다. 즉 소수 입니다.

Ex06.py

```
# 2~100 사이의 모든 소수 구하기 2
# 소수란 1과 자신만을 소수로 가지는 수입니다.
num = 2
count = 0  # 소수의 개수를 세어줄 변수
while num <= 100:
    i = 2  # 2 ~ num 까지 증가할 변수
    while num % i:  # 나누어질 때까지 반복
        i += 1        # 1증가

    if i == num:  # 나누어진 수가 자기 자신이면 소수
        print('{0:5}'.format(num), end='')
        count += 1
        if not count % 5:  # 개수가 5의 배수면 줄바꿈
            print()
    num += 1  # 100까지 증가
```

실행 결과

```
    2    3    5    7   11
   13   17   19   23   29
   31   37   41   43   47
   53   59   61   67   71
   73   79   83   89   97
```

소수 구하기 3
굉장히 큰 수의 소수 판별에 위의 풀이방법은 너무 많은 반복 횟수를 가집니다. 반복 횟수를 줄여보도록 하겠습니다.
임의의 수 n은 n/2을 넘는 수로는 나누어 떨어질 수 없습니다.
Ex07.py

```
# 소수 구하기 3
```

```
while True:
    num = int(input('양의 정수를 입력하시오(0은종료)'))
    if not num:
        break  # 반복문을 탈출합니다.
    if num < 0:
        print('양의 정수로 입력하시오')
        continue  # 반복문의 처음으로 갑니다.

    i = 2
    flag = True
    while i <= num // 2:  # 절반까지만 반복
        if num % i == 0:    # 나누어지면 중간에 약수가 있으므로 소수 아님
            flag = False
            break
        i += 1

    if flag:  # flag 값이 참이면 나누어진적이 없다. 즉 소수
        print('{0}은 "소수입니다."'.format(num))
    else:
        print('{0}은 "소수가 아닙니다."'.format(num))
```

실행 결과

```
양의 정수를 입력하시오(0은종료)2
2은 "소수입니다."
양의 정수를 입력하시오(0은종료)99991
99991은 "소수입니다."
양의 정수를 입력하시오(0은종료)0
```

실행시간 측정하기

Ex08.py

```
import time
start_time = time.time()  # 시작 시간
# ----------------------------------------------------------------------
print('소수를 구하는 첫번째 방법')
count = 0
num = 9999991
i = 1  # 1 ~ num 까지 증가할 변수
while i <= num:
    if num % i == 0:  # 나누어지면 약수
        count += 1
    i += 1  # 1증가
```

```
if count == 2:  # 약수의 개수가 2개면 출력
    print('{0}의 약수가 {1}개이므로 "소수"입니다.'.format(num, count))
# ----------------------------------------------------------------------
end_time = time.time()    # 종료 시간
print('실행시간 : {0:.3f}초'.format(end_time-start_time))
print()

start_time = time.time()  # 시작 시간
# ----------------------------------------------------------------------
print('소수를 구하는 두번째 방법')
num = 9999991
i = 2  # 2 ~ num 까지 증가할 변수
while num % i:  # 나누어질 때까지 반복
    i += 1       # 1증가

if i == num:  # 나누어진 수가 자기 자신이면 소수
    print('{0:5}은 "소수"입니다.'.format(num))
# ----------------------------------------------------------------------
end_time = time.time()    # 종료 시간
print('실행시간 : {0:.3f}초'.format(end_time-start_time))
print()

start_time = time.time()  # 시작 시간
# ----------------------------------------------------------------------
print('소수를 구하는 세번째 방법')
num = 9999991
i = 2  # 2 ~ num//2 까지 증가할 변수
flag = True
while i <= num // 2:  # 절반까지만 반복
    if num % i == 0:  # 나누어지면 중간에 약수가 있으므로 소수 아님
        flag = False
        break
    i += 1

if flag:  # flag 값이 참이면 나누어진적이 없다. 즉 소수
    print('{0}은 "소수"입니다.'.format(num))
# ----------------------------------------------------------------------
end_time = time.time()    # 종료 시간
print('실행시간 : {0:.3f}초'.format(end_time-start_time))
print()
```

실행 결과

소수를 구하는 첫번째 방법
9999991의 약수가 2개이므로 "소수"입니다.
실행시간 : 1.943초

소수를 구하는 두번째 방법
9999991은 "소수"입니다.
실행시간 : 1.291초

소수를 구하는 세번째 방법
9999991은 "소수"입니다.
실행시간 : 1.412초
마지막 편집일시 : 2018년 7월 13일 12:26
---
03. for loops
1. for 반복문
for문의 기본 구조

```
for 변수명 in Collection_Data:
    반복 실행할 명령
    ....
else:
    데이터가 없을때 실행할 명령
    ....
```

in 뒤의 Collection_Data 값이 없을때 까지 하나씩 변수로 복사되어 반복됩니다.
Collection_Data로는 tuple, set, list, dictionary, string 등 데이터의 집합입니다.
else 구절은 선택 사항입니다.
기본사용
Ex01.py

```
for i in (1, 2, 3):
    print(i, end=' ')
print()

for i in {1, 2, 3}:
    print(i, end=' ')
print()

for i in [1, 2, 3]:
    print(i, end=' ')
print()
```

```
dict1 = {'이름': '한사람', '나이': 33}
for i in dict1:
    print(i, ':', dict1[i])
print()

for i in 'Python':
    print(i)
print()

for i in []:
    print(i)
else:
    print('데이터가 없습니다. ')
```

실행 결과

```
1 2 3
1 2 3
1 2 3
이름 : 한사람
나이 : 33

P
y
t
h
o
n

데이터가 없습니다.
```

for문의 기본 구조

```
for 변수명 in range함수 또는 enumerate 함수:      반복 실행할 명령
    ....
else:
    데이터가 없을때 실행할 명령
    ....
```

함수의 결과가 처음부터 끝날때까지 변수로 복사되어 반복합니다.

기본 사용

Ex02.py

```
# range 함수
for i in range(5):
    print(i, end=' ')
print()

for i in range(1, 11):
    print(i, end=' ')
print()

for i in range(1, 10, 3):
    print(i, end=' ')
print()

for i in range(10, -1, -1):
    print(i, end=' ')
print()
# enumerate 함수
data = enumerate([1, 2, 3])
for i, value in data:
    print(i, ":", value)
print()

dict1 = {'이름': '한사람', '나이': 33}
data = enumerate(dict1)
for i, key in data:
    print(i, ":", key, dict1[key])
print()

data = enumerate("재미있는 파이썬")
for i, value in data:
    print(i, ":", value)
print()
```

실행 결과

```
0 1 2 3 4
1 2 3 4 5 6 7 8 9 10
1 4 7
10 9 8 7 6 5 4 3 2 1 0
0 : 1
1 : 2
```

2 : 3

0 : 이름 한사람
1 : 나이 33

0 : 재
1 : 미
2 : 있
3 : 는
4 :
5 : 파
6 : 이
7 : 썬

2. for문을 이용한 리스트 만들기
[식 for 변수 in Collection_Data]
[식 for 변수 in Collection_Data if 조건]

기본 사용
Ex03.py

```
list1 = [i**2 for i in range(1,11)]
print(list1)

list2 = [i for i in range(1,11) if i%2]
print(list2)

list3 = [i for i in range(1,11) if not i%2]
print(list3)

# 중첩이 가능합니다. 구구단표입니다.
list4 = [i*j for i in range(1,10)
             for j in range(1,10)]


for i in range(0,len(list4)):
    print('{:3d}'.format(list4[i]), end='')
    if (i+1)%9==0:
        print()
```

실행 결과

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[1, 3, 5, 7, 9]
[2, 4, 6, 8, 10]
  1  2  3  4  5  6  7  8  9
  2  4  6  8 10 12 14 16 18
  3  6  9 12 15 18 21 24 27
  4  8 12 16 20 24 28 32 36
  5 10 15 20 25 30 35 40 45
  6 12 18 24 30 36 42 48 54
  7 14 21 28 35 42 49 56 63
  8 16 24 32 40 48 56 64 72
  9 18 27 36 45 54 63 72 81
```

3. for의 활용

while문으로 해결할 수 있는 문제는 for로도 해결이 가능합니다.

두가지 명령의 선택은 사용자 몫입니다.

사용자가 편리한 명령어를 사용하시기 바랍니다.

피보나치 수(Fibonacci Numbers)를 1줄에 5개씩 30개 출력하기

Ex04.py

```
first = 1
second = 2
for i in range(30):
    print('{0:10d}'.format(second), end='')
    second = first + second
    first = second - first
    if not (i+1)%5:
        print()
```

실행 결과

```
         2         3         5         8        13
        21        34        55        89       144
       233       377       610       987      1597
      2584      4181      6765     10946     17711
     28657     46368     75025    121393    196418
    317811    514229    832040   1346269   2178309
```

약수와 약수 개수 구하기

Ex05.py

```
# 약수와 약수 개수 구하기
while True:
```

```
    num = int(input('정수를 입력하시오(0은종료)'))
    if not num:
        break  # 반복문을 탈출합니다.
    if num < 0:
        print('양의 정수로 입력하시오')
        continue  # 반복문의 처음으로 갑니다.
    count = 0  # 약수의 개수를 세어줄 변수
    for i in range(1, num+1):
        if num % i == 0:  # 나누어지면 약수
            print('{0:5}'.format(i), end=' ')
            count += 1
    print()
    print('{0}의 약수의 개수 {1}개입니다.'.format(num, count))
```

실행 결과

```
정수를 입력하시오(0은종료)34
    1      2     17     34
34의 약수의 개수 4개입니다.
정수를 입력하시오(0은종료)78
    1      2      3      6     13     26     39     78
78의 약수의 개수 8개입니다.
정수를 입력하시오(0은종료)0
```

제곱수 구하기

Ex06.py

```
# 약수와 약수 개수 구하기
for num in range(1,101):
    count = 0  # 약수의 개수를 세어줄 변수
    for i in range(1, num+1):
        if num % i == 0:  # 나누어지면 약수
            count += 1
    if count%2:
        print('{0}는 {1}개의 약수를 가지므로 "제곱수"입니다.'.format(num, count))
```

실행 결과

```
1는 1개의 약수를 가지므로 "제곱수"입니다.
4는 3개의 약수를 가지므로 "제곱수"입니다.
9는 3개의 약수를 가지므로 "제곱수"입니다.
16는 5개의 약수를 가지므로 "제곱수"입니다.
25는 3개의 약수를 가지므로 "제곱수"입니다.
```

36는 9개의 약수를 가지므로 "제곱수"입니다.
49는 3개의 약수를 가지므로 "제곱수"입니다.
64는 7개의 약수를 가지므로 "제곱수"입니다.
81는 5개의 약수를 가지므로 "제곱수"입니다.
100는 9개의 약수를 가지므로 "제곱수"입니다.
소수 구하기
Ex07.py

```
# 소수 구하기
count = 0
for num in range(2, 101):
    flag = True
    for i in range(2, num//2+1):
        if num % i == 0:  # 나누어지면 중간에 약수가 있으므로 소수 아님
            flag = False
            break
    if flag:  # flag 값이 참이면 나누어진적이 없다. 즉 소수
        count += 1
        print('{0:5d}'.format(num), end='')
        if not count%5:
            print()
```

실행 결과

```
    2    3    5    7   11
   13   17   19   23   29
   31   37   41   43   47
   53   59   61   67   71
   73   79   83   89   97
```

마지막 편집일시 : 2018년 7월 16일 3:59

==========

제 03장 세번째 계단밟기

세부적인 항목을 차례대로 심도있게 연구해 보도록 하겠습니다.

마지막 편집일시 : 2018년 7월 9일 11:29

01. Python(파이썬) 자료형 심화

- Python Strings

- Python Tuple

- Python Set
- Python List
- Python Dictionary

마지막 편집일시 : 2018년 7월 9일 11:3

01. Python Strings
02. Python Tuple
작성중
03. Python Set
작성중
04. Python List
작성중
05. Python Dictionary
작성중

---
02. Python 함수
함수란? 자주 사용되는 코드를 별도로 만들어 놓고 필요시 불러서 재사용하는 방법입니다.

다음의 코드를 보면 무엇을 하는 코드일까요? 파이썬에 대한 소감을 출력하는 프로그램이라고 작성을 했지만 소감 사이사이에 줄을 넣는 코드가 더 많아 어떤것이 주가 되는지 알 수 없는 코드가 됩니다.

```
# 파이선에 대한 소감을 출력하는 프로그램 작성
print('재미있는 Python!!')
for i in range(0, 80):
    print('-', sep='', end='')
print()

print('정말 재미있는 Python!!')
for i in range(0, 80):
    print('-', sep='', end='')
print()
```

```
print('진짜 진짜 재미있는 Python!!')
for i in range(0, 80):
    print('-', sep='', end='')
print()
```

다음의 코드는 어떤가요? 위의 코드와 실행 결과는 같습니다. 어떤 코드가 이해하기 쉬울까요? 100의 100은 다음의 코드를 더 좋게 판단할것 입니다. 그렇습니다. 다음의 코드에 나오는 line()이 "함수(function)"입니다. 코드가 깔끔할 뿐만 아니라 가독력도 높여주고 있습니다. 이렇게 자주 사용되는 코드나 단위작업을 별도로 만들어 놓고 필요할때 호출할 수 있도록하는 것이 바로 함수 입니다.

```
# 파이선에 대한 소감을 출력하는 프로그램 작성
def line():
    for i in range(0, 80):
        print('-', sep='', end='')
    print()


print('재미있는 Python!!')
line()

print('정말 재미있는 Python!!')
line()

print('진짜 진짜 재미있는 Python!!')
line()
```

이제 함수에 대하여 하나하나 알아 보도록 하겠습니다.

마지막 편집일시 : 2018년 7월 23일 11:50

---

01. 리턴값이 없는 함수

리턴값이란 함수가 어떠한 기능을 수행하고 그 결과를 호출한 곳으로 돌려주는 값을 말합니다. 함수가 어떠한 기능을 수행하고 돌려줄 값이 없으면 return 명령은 사용하지 않습니다.

함수를 만들때는 def 예약어를 사용합니다.
함수의 내용은 들여쓰기를 합니다. 들여쓰기가 종료되면 함수 정의가 종료 됩니다.
함수의 이름은 변수명을 만들 듯이 사용자가 임의로 지정이 가능합니다.
필요에 따라 괄호안에 매개 변수를 지정할 수 있습니다.
[] (대괄호)는 생략가능하다는 표시 입니다.
매개 변수에 초기값을 지정할 수 있습니다.
초기값을 지정하지 않으면 호출 시 반드시 인수를 지정해야 합니다.

```
def 함수명([매개변수[=초기값], .....]):
    수행할 문장1
    수행할 문장2
    ....
    ....
    ....
```

다음의 예를 보도록 하겠습니다.

Ex03_function.py

함수를 정의할때 매개변수(parameter)를 두개 지정하였으며 기본 값도 지정하였습니다.
이렇게 함수를 지정하면 호출 할때 아주 다양하게 호출이 가능합니다.
함수를 호출할 때 인수를 생략하면 기본값이 적용됩니다. line()으로 호출하면 '-'를 80개 그립니다.
함수를 호출할 때 매개변수(parameter) 이름을 생략하면 인수값이 매개변수에 차례대로 전달됩니다.
함수를 호출할 때 매개변수(parameter) 이름을 지정하면 지정 이름에 값이 대입됩니다.

```
# 파이선에 대한 소감을 출력하는 프로그램 작성
def line(count=80, char='-'):  # 매개변수(parameter)를 두개 지정
    for i in range(0, count):
        print(char, sep='', end='')
    print()


print('재미있는 Python!!')
line(17)  # 인수(arguments)로 숫자 1개 지정. 첫번째 개변수(parameter)에 17이 대입

print('정말 재미있는 Python!!')
# 인수(arguments)로 숫자 1개와 문자 지정, 매개변수(parameter) 이름을 생략시 차례대로 대입
line(22, '~')

print('진짜 진짜 재미있는 Python!!')
line(char='^', count=27)  # 매개변수 이름을 지정하면 순서와 상관 없음

print('진짜 진짜 정말로 재미있는 Python!!')
line() # 인수(arguments) 생략 시 기본값 적용
```

실행 결과

```
재미있는 Python!!
```

정말 재미있는 Python!!
~~~~~~~~~~~~~~~~~~~~~~
진짜 진짜 재미있는 Python!!
^^^^^^^^^^^^^^^^^^^^^^^^^^^
진짜 진짜 정말로 재미있는 Python!!

마지막 편집일시 : 2018년 7월 23일 12:36
---
02. 리턴값이 있는 함수
리턴값이란 함수가 어떠한 기능을 수행하고 그 결과를 호출한 곳으로 돌려주는 값을 말합니다. 함수가 어떠한 기능을 수행하고 돌려줄 값이 있으면 return 명령을 사용합니다.

함수를 만들때는 def 예약어를 사용합니다.
함수의 내용은 들여쓰기를 합니다. 들여쓰기가 종료되면 함수 정의가 종료 됩니다.
함수의 이름은 변수명을 만들 듯이 사용자가 임의로 지정이 가능합니다.
필요에 따라 괄호안에 매개 변수를 지정할 수 있습니다.
[] (대괄호)는 생략가능하다는 표시 입니다.
매개 변수에 초기값을 지정할 수 있습니다.
초기값을 지정하지 않으면 호출 시 반드시 인수를 지정해야 합니다.
값을 되돌려 줄때 return 명령어를 사용합니다.
return 명령어를 만나면 함수가 종료됩니다.
return 명령어는 하나의 함수에 여러번 나올 수 있습니다. 하지만 1번만 사용하는것이 좋습니다.
```
def 함수명([매개변수[=초기값], .....]):
    수행할 문장1
    수행할 문장2
    ....
    ....
    ....
    return 돌려줄값
```
다음의 예를 보도록 하겠습니다.

Ex04_function.py

함수의 실행 결과가 또 다른 연산에 사용된다면 return 명령어를 이용하여 결과를 돌려줍니다.
함수를 작성할 때는 언제나 항상 올바른 값을 리턴하도록 작성해야 합니다.
```
# 첫번째 풀이
s = 1+2+3 + 1+2+3+4+5 + 1+2+3+4+5+6+7+8+9+10
```
~~~~~~~~~~~~~~~~~~~~~~

```
print("합계 :", s)

# n ~ m 사이의 합계 구하는 함수
def sum(n=1, m=100):
    if n > m:  # 앞의 값이 크면 교환합니다.
        n, m = m, n
    s = 0
    for i in range(n, m+1):
        s += i
    return s


# 두번째 풀이
s = sum(1, 3) + sum(1, 5) + sum(1, 10)
print("합계 :", s)

print("{0}~{1}까지 합 : {2}".format(1, 100, sum()))
print("{0}~{1}까지 합 : {2}".format(1, 10, sum(1, 10)))
print("{0}~{1}까지 합 : {2}".format(1, 1000, sum(1, 1000)))
# 이때도 올바른 값이 나와야 합니다.
print("{0}~{1}까지 합 : {2}".format(10, 1, sum(10, 1)))
print("{0}~{1}까지 합 : {2}".format(10, 10, sum(10, 10)))
```

실행 결과

```
합계 : 76
합계 : 76
1~100까지 합 : 5050
1~10까지 합 : 55
1~1000까지 합 : 500500
10~1까지 합 : 55
10~10까지 합 : 10
```

다음의 예를 보겠습니다.
같은 기능을 하는 함수를 4가지로 만들어 보았습니다.
is_even_number 함수는 "짝수"인지를 판단하는 함수 입니다.
is_even_number4 함수가 제일 보기가 좋습니다. 제일 추천하는 방식입니다.
return은 한번 만 사용하는 것이 함수를 이해하는데 좋습니다.

Ex05_function.py

```
def is_even_number1(number):
```

```
    if number % 2:
        return False
    else:
        return True


def is_even_number2(number):
    if number % 2:
        result = False
    else:
        result = True
    return result


def is_even_number3(number):
    return False if number % 2 else True


def is_even_number4(number):
    return number % 2 == 0


def is_odd_number(number):
    return number % 2 != 0  # return을 만나면 함수 종료
    print('홀수' if number % 2 != 0 else '짝수') # 실행 안됨


number = 12
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number1(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number2(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number3(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number4(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '홀' if is_odd_number(number) else '짝'))
print()

number = 11
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number1(number) else '홀
```

```
'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number2(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number3(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '짝' if is_even_number4(number) else '홀'))
print('{0}는(은) "{1}수"입니다.'.format(number, '홀' if is_odd_number(number) else '짝'))
print()
```

실행 결과

```
12는(은) "짝수"입니다.
12는(은) "짝수"입니다.
12는(은) "짝수"입니다.
12는(은) "짝수"입니다.
12는(은) "짝수"입니다.

11는(은) "홀수"입니다.
11는(은) "홀수"입니다.
11는(은) "홀수"입니다.
11는(은) "홀수"입니다.
11는(은) "홀수"입니다.
```

마지막 편집일시 : 2018년 7월 23일 6:25